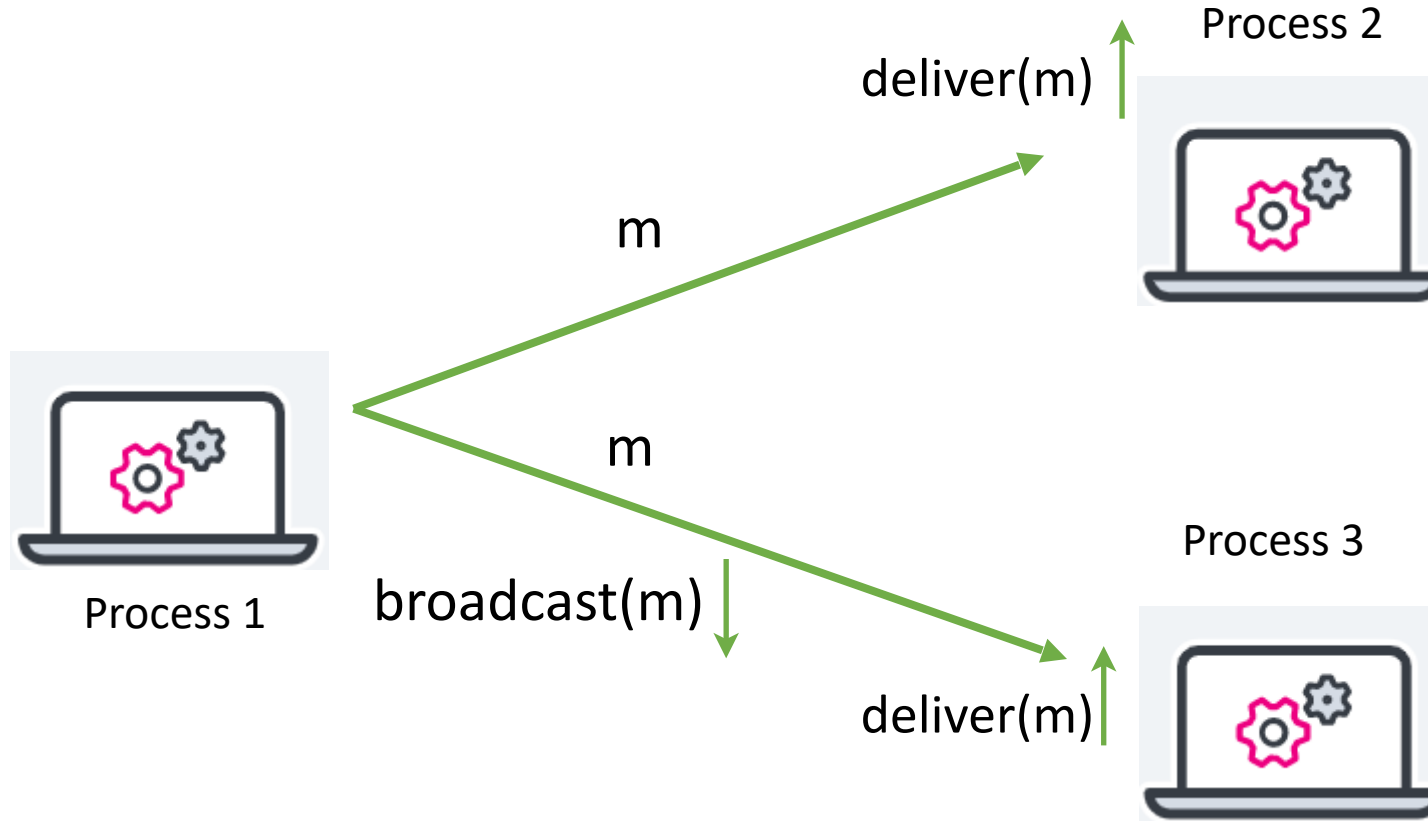


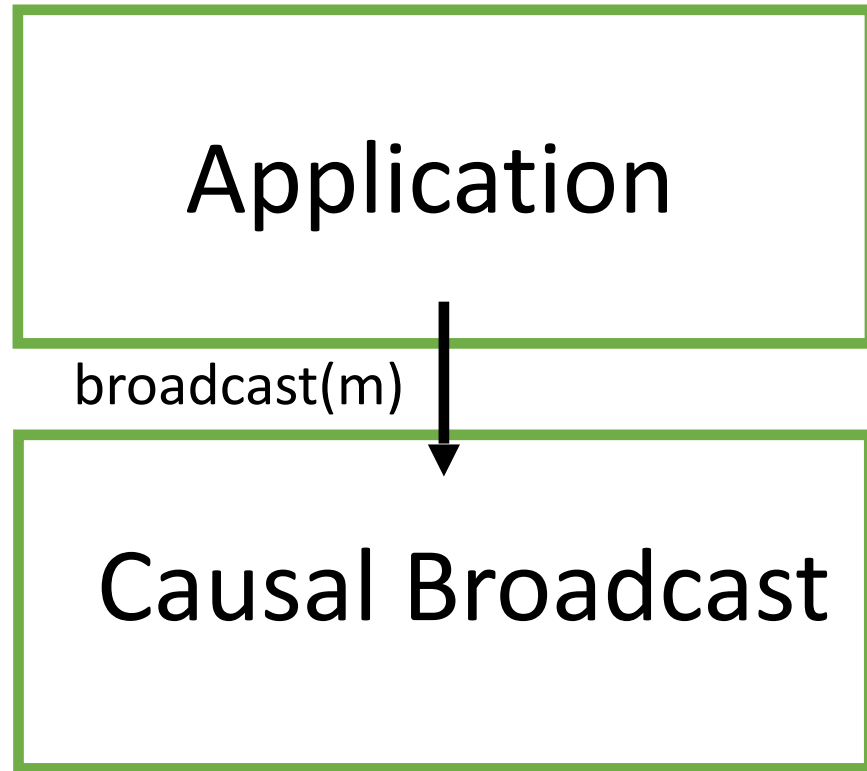
Causal Broadcast

Mohsen Lesani

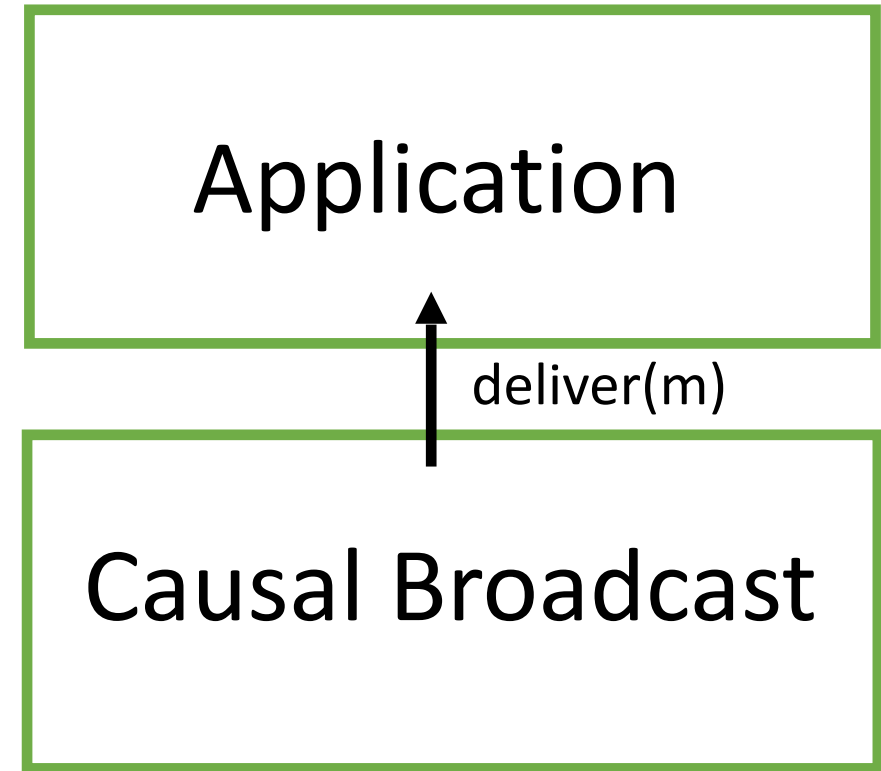
Broadcast Abstraction



Modular Design



Process 1



Process 2

Broadcast Execution Diagram

broadcast(m1)

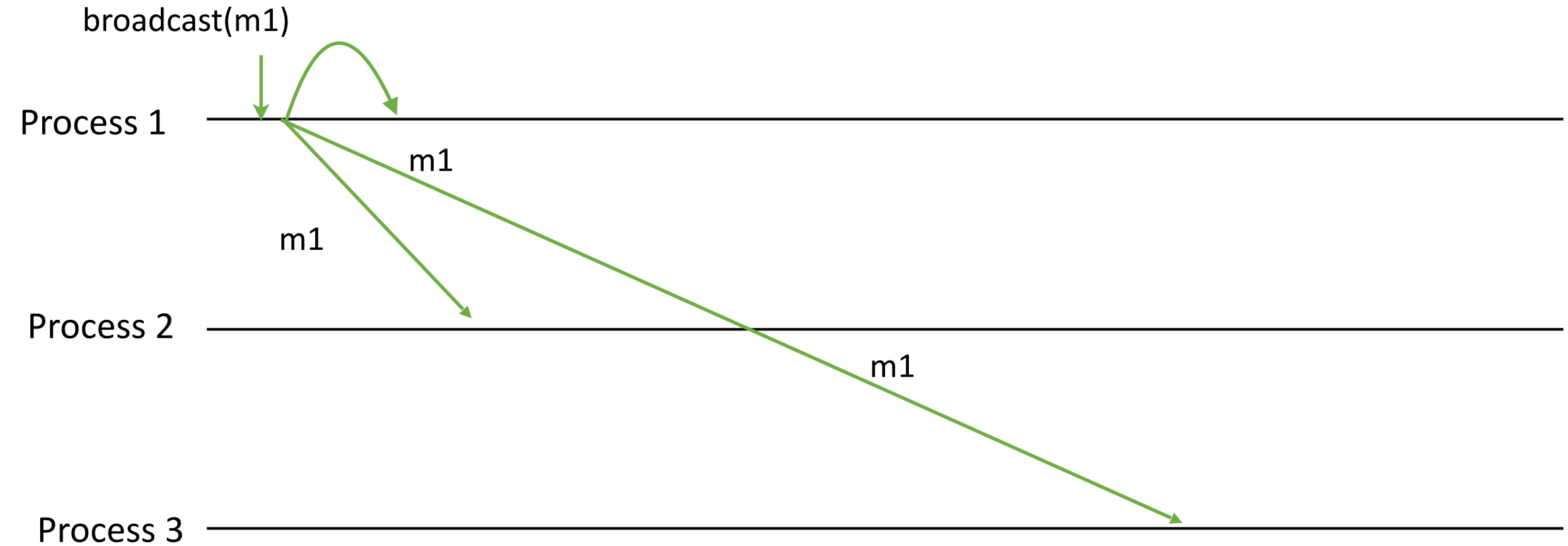


Process 1

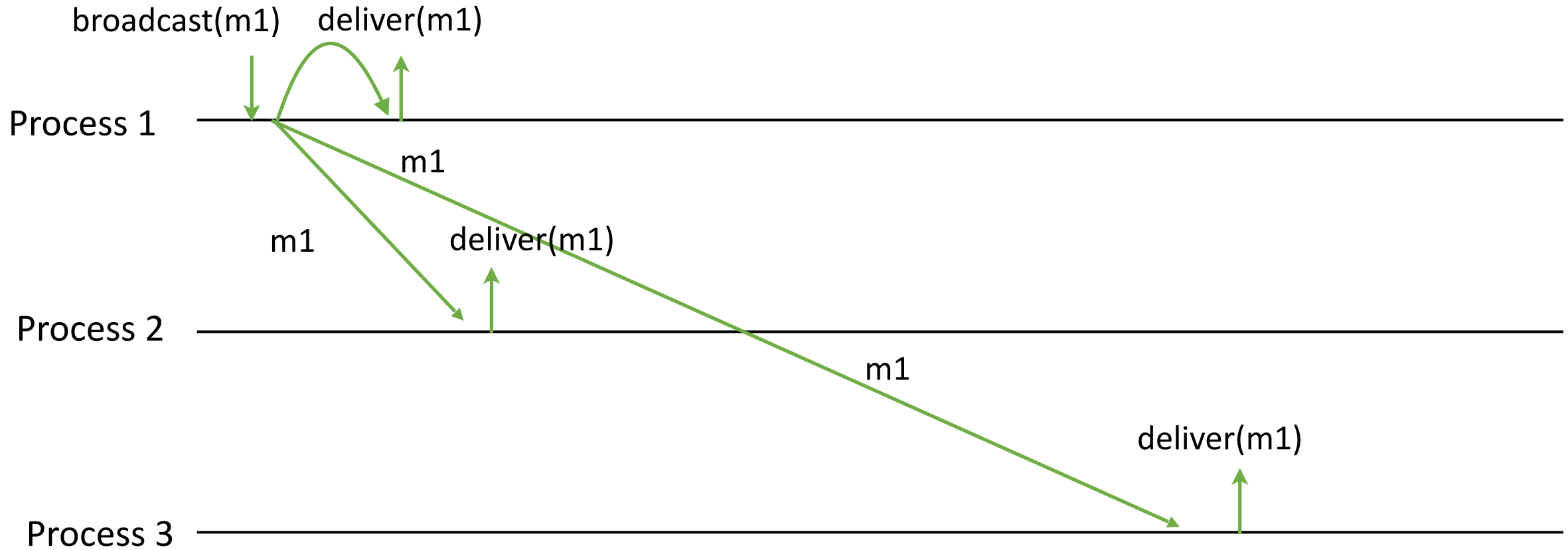
Process 2

Process 3

Broadcast Execution Diagram



Broadcast Execution Diagram



- **Motivation: why causal broadcast?**
- Properties of causal broadcast
- Protocols

Intuition

- So far, we did not consider ordering among messages; In particular, we considered messages to be independent
- Two messages from the same process might not be delivered in the order they were broadcast
- A message m_1 that causes a message m_2 might be delivered by some process after m_2
- Consider a news or social network where every new event contains a reference to the event that caused it.

Hey, check out my
cool picture.

Hey, check out my cool picture.



No image available

Photo and Comment

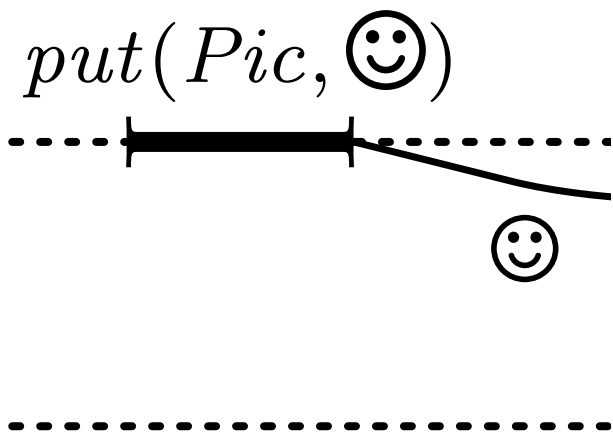


Photo and Comment

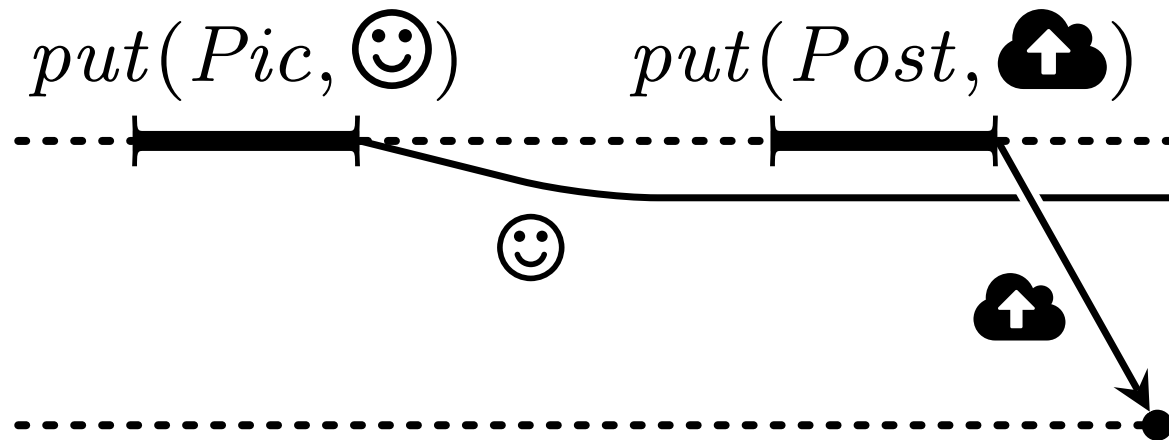


Photo and Comment

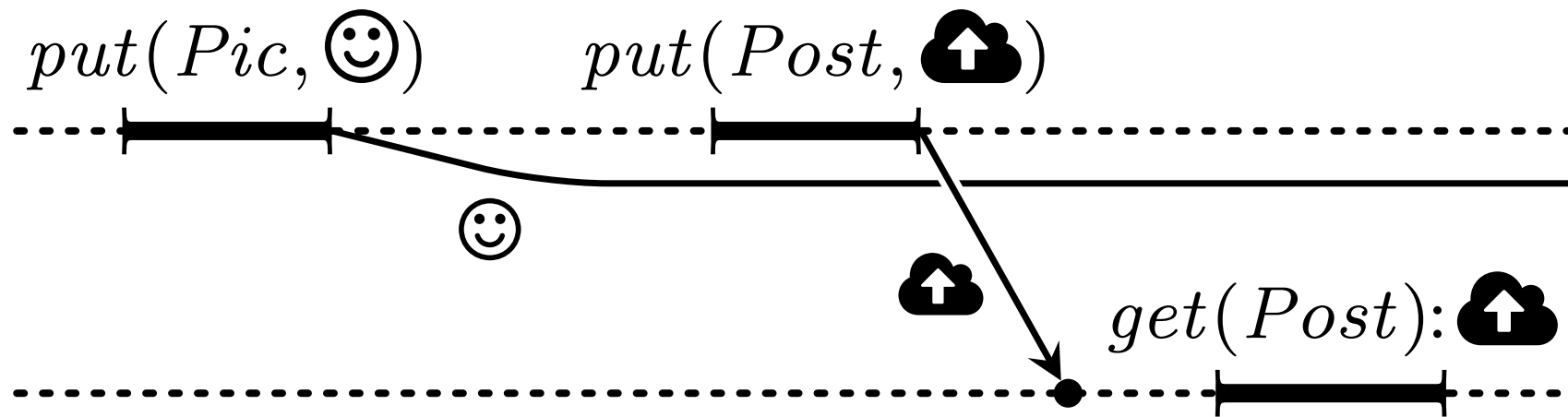
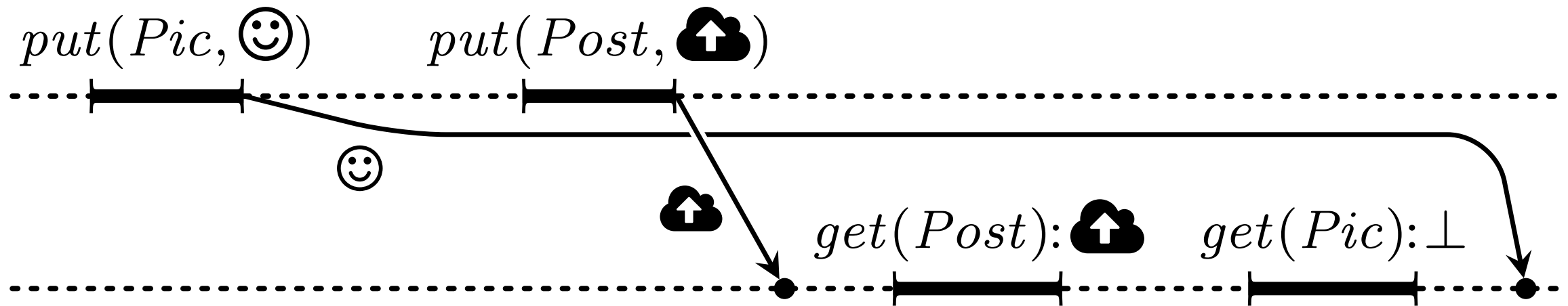
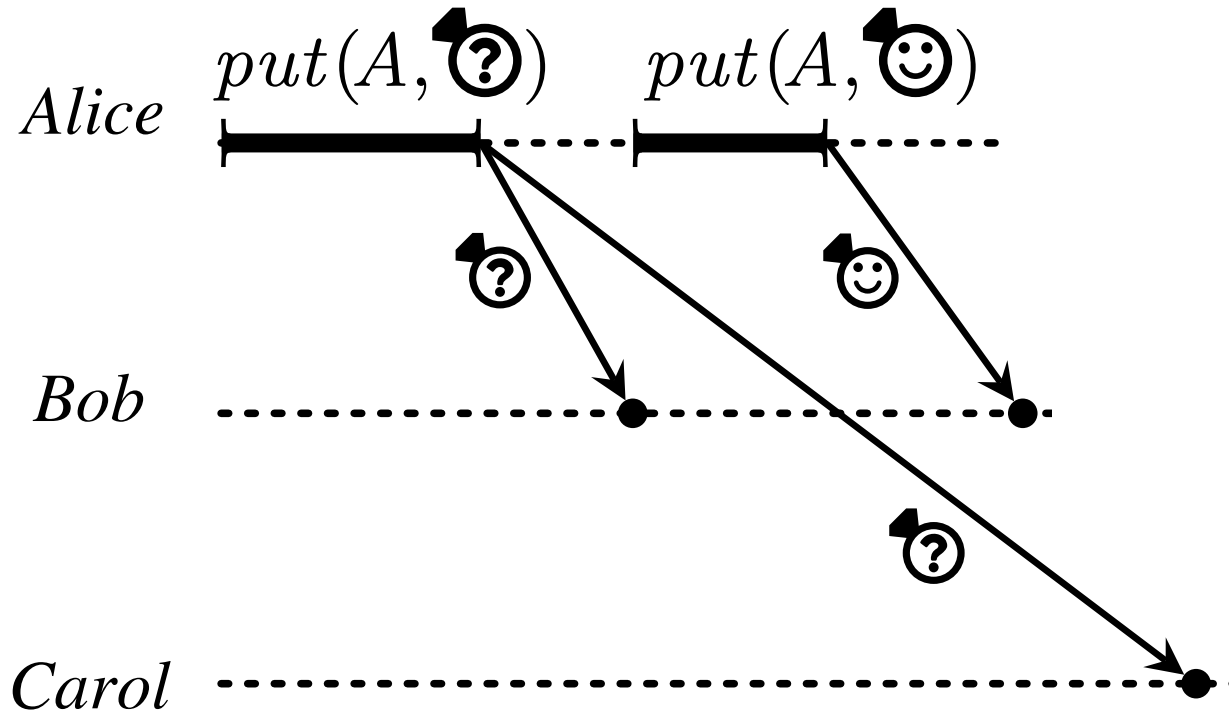


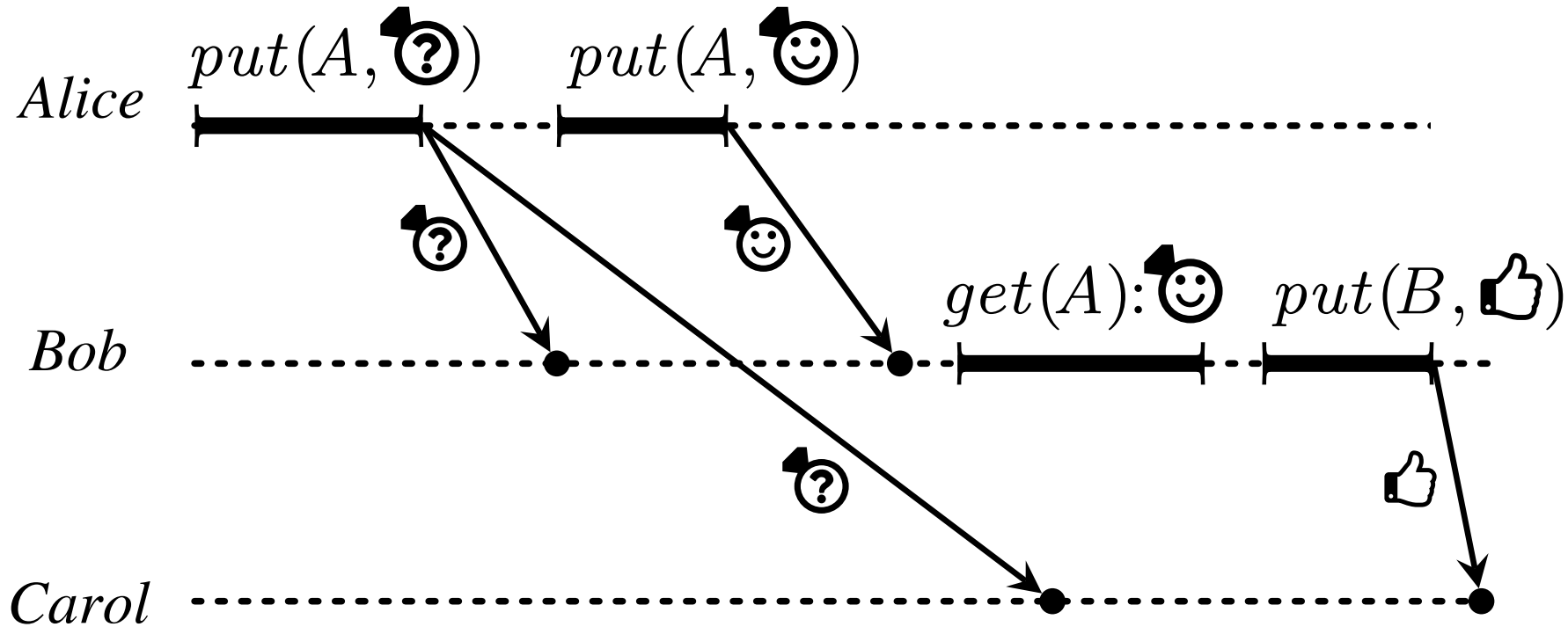
Photo and Comment



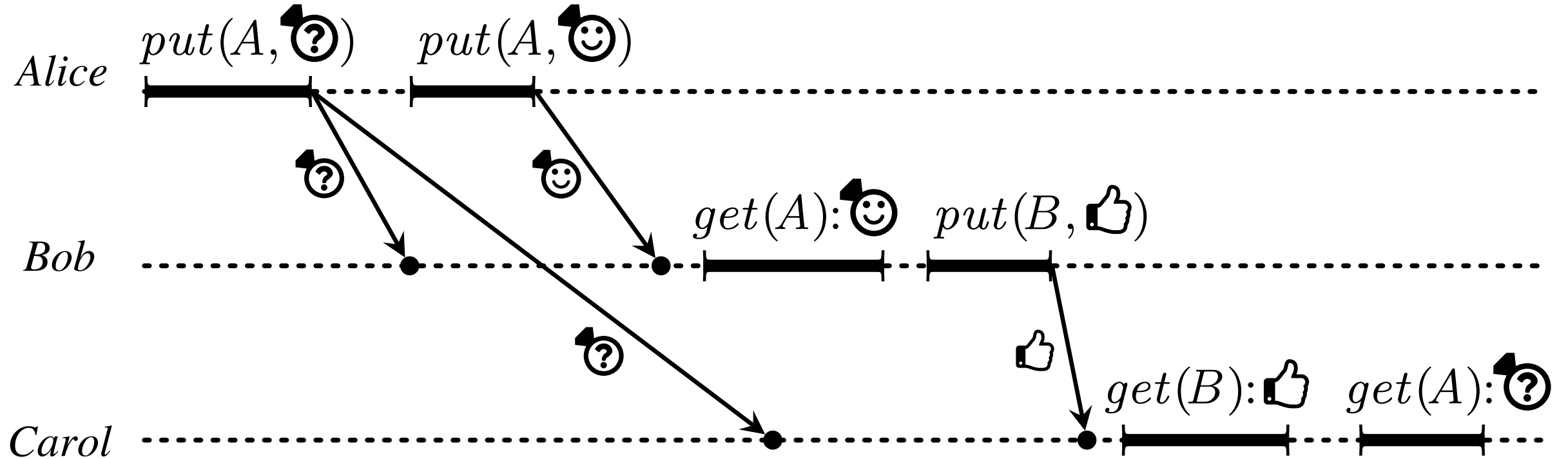
Lost Ring



Lost Ring



Lost Ring



Overview

- Motivation: why causal broadcast?
- **Properties of causal broadcast**
- Protocols

Causal Order Property

If any process p_i delivers a message m_2 ,
then p_i must have delivered every message m_1 that m_2 is **dependent** on.

Causal Relation (Dependency)

Let m_1 and m_2 be any two messages.

$m_1 < m_2$ (m_1 is causally before m_2 , or m_2 depends on m_1) iff

- **FIFO order:**

A process p_i broadcasts m_1 before broadcasting m_2 .

- ...

- ...

Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.

P1

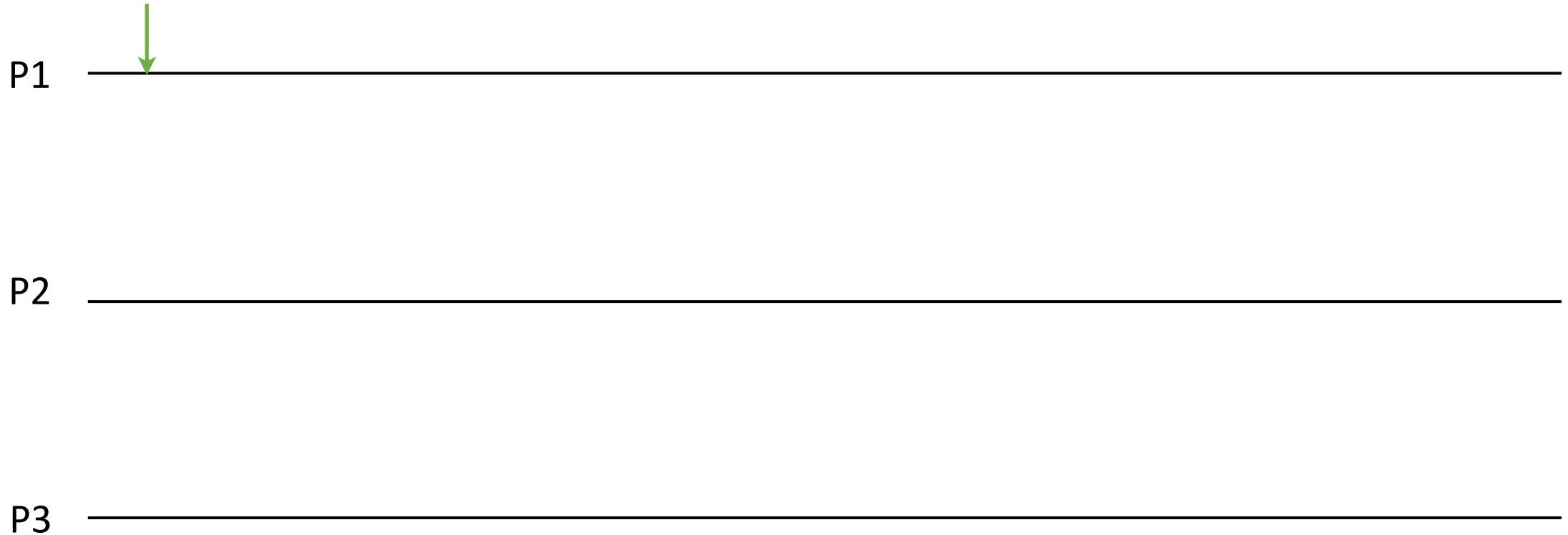
P2

P3

Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.

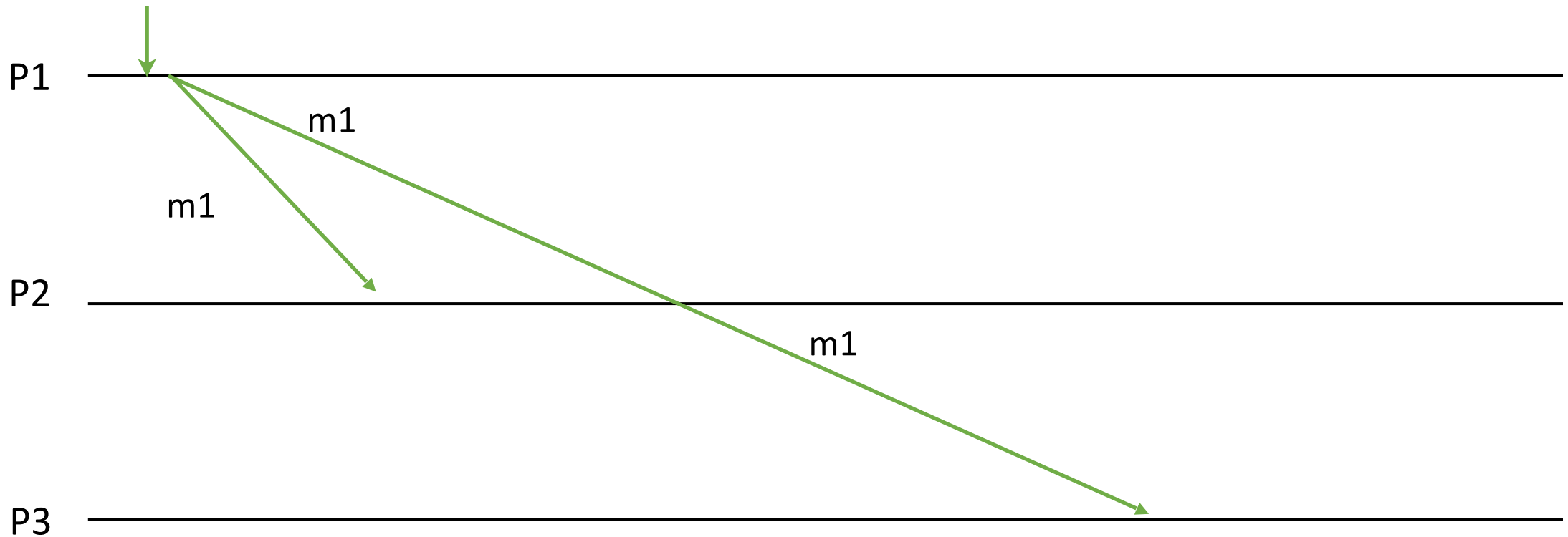
broadcast(m_1)



Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.

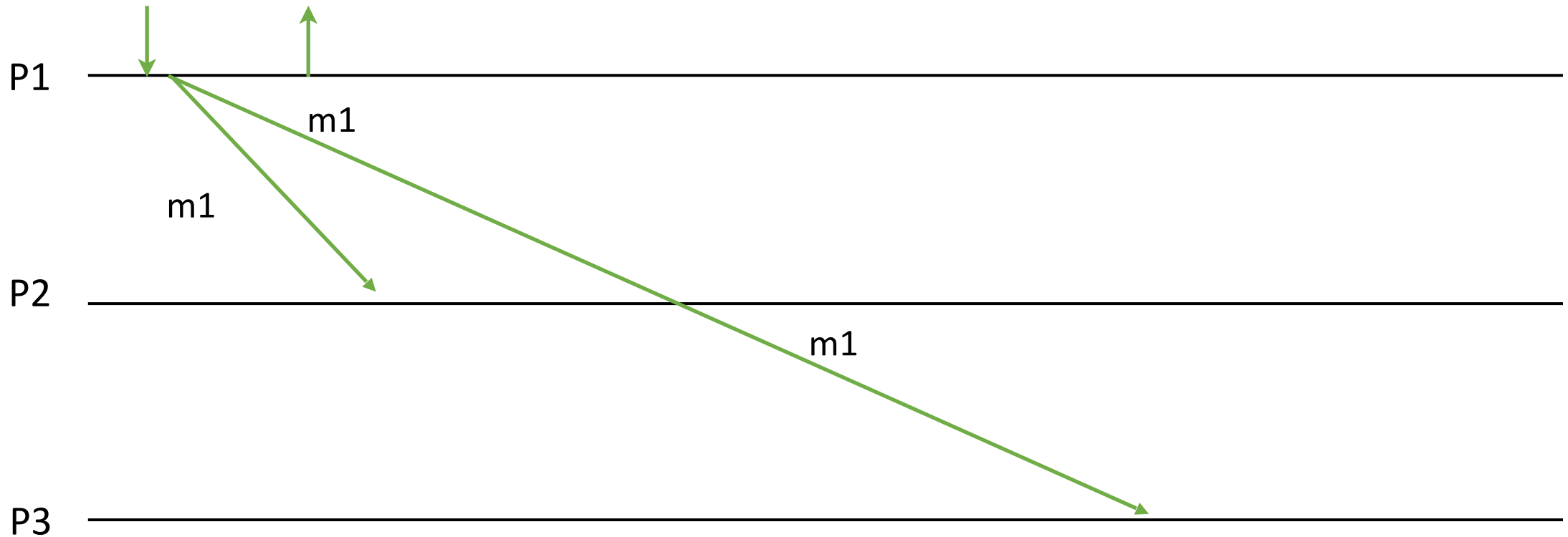
broadcast(m_1)



Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.

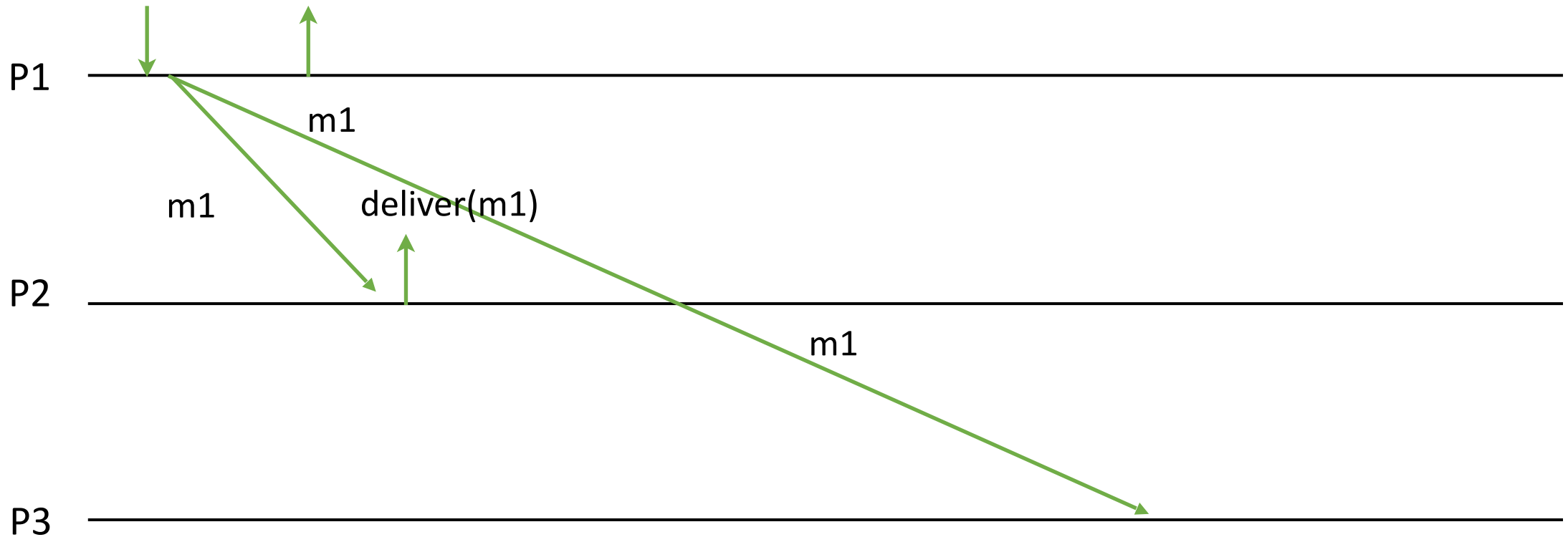
broadcast(m1) deliver(m1)



Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.

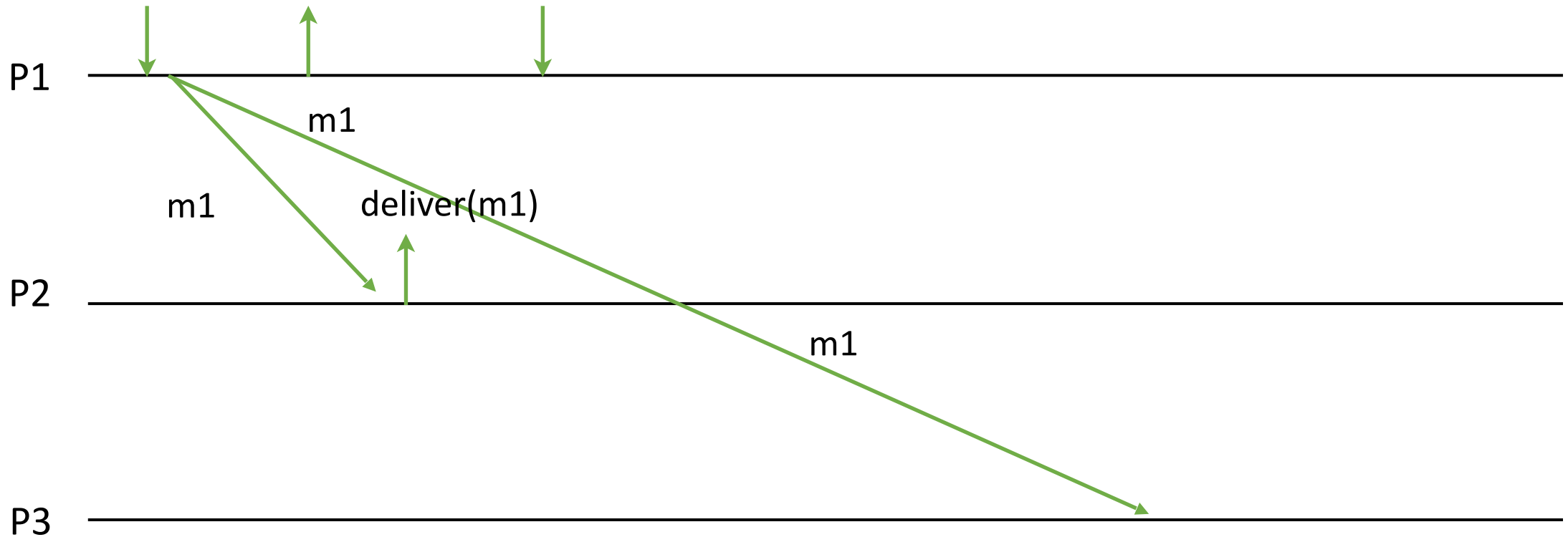
broadcast(m1) deliver(m1)



Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.

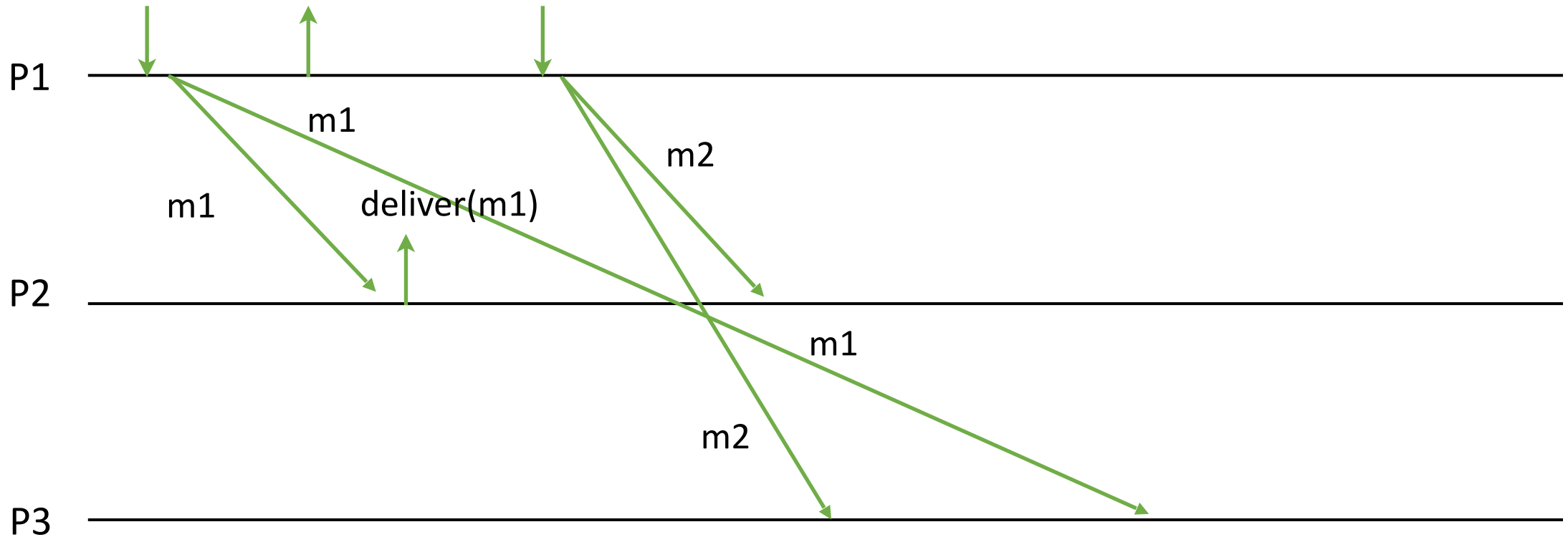
broadcast(m1) deliver(m1) **broadcast(m2)**



Example 1: FIFO Order

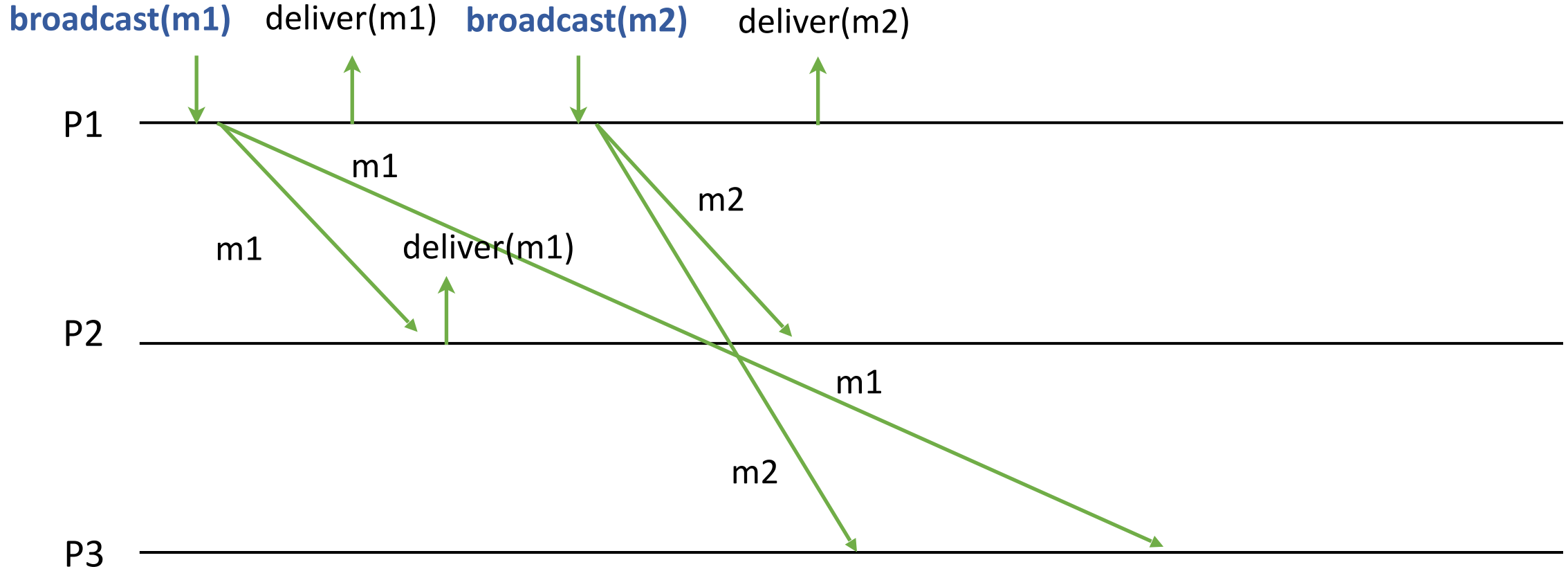
If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.

broadcast(m1) **deliver(m1)** **broadcast(m2)**



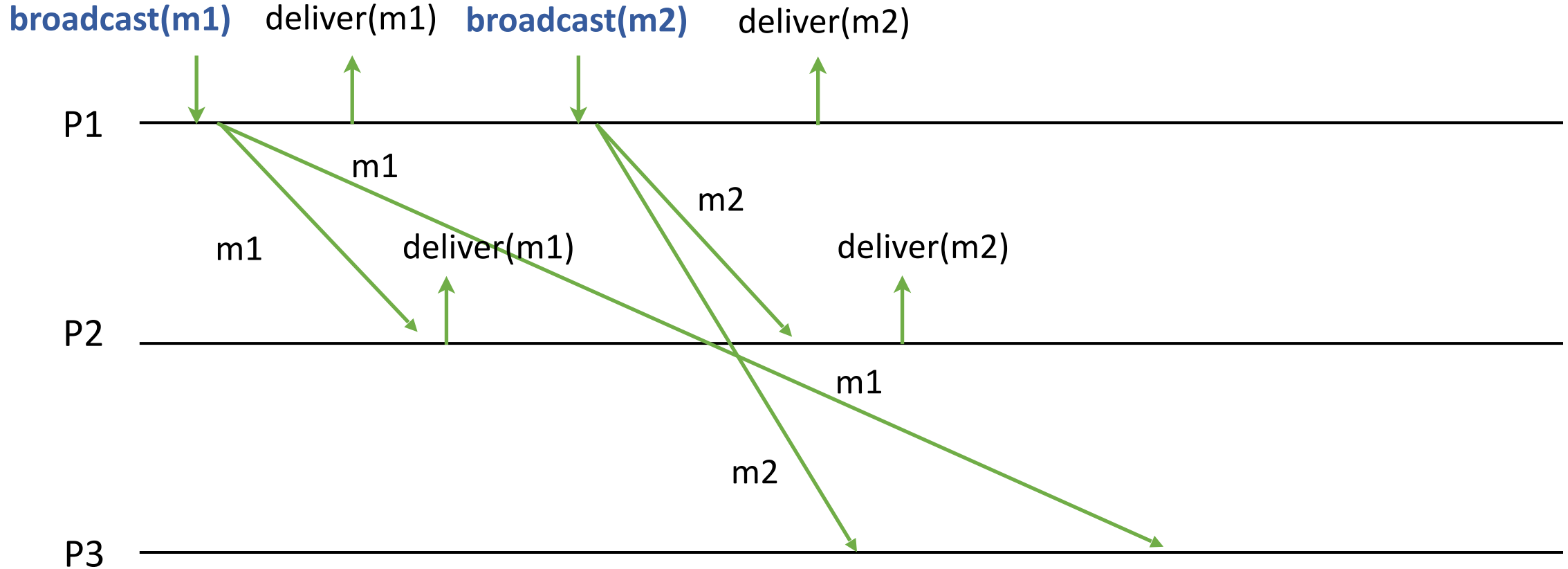
Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.



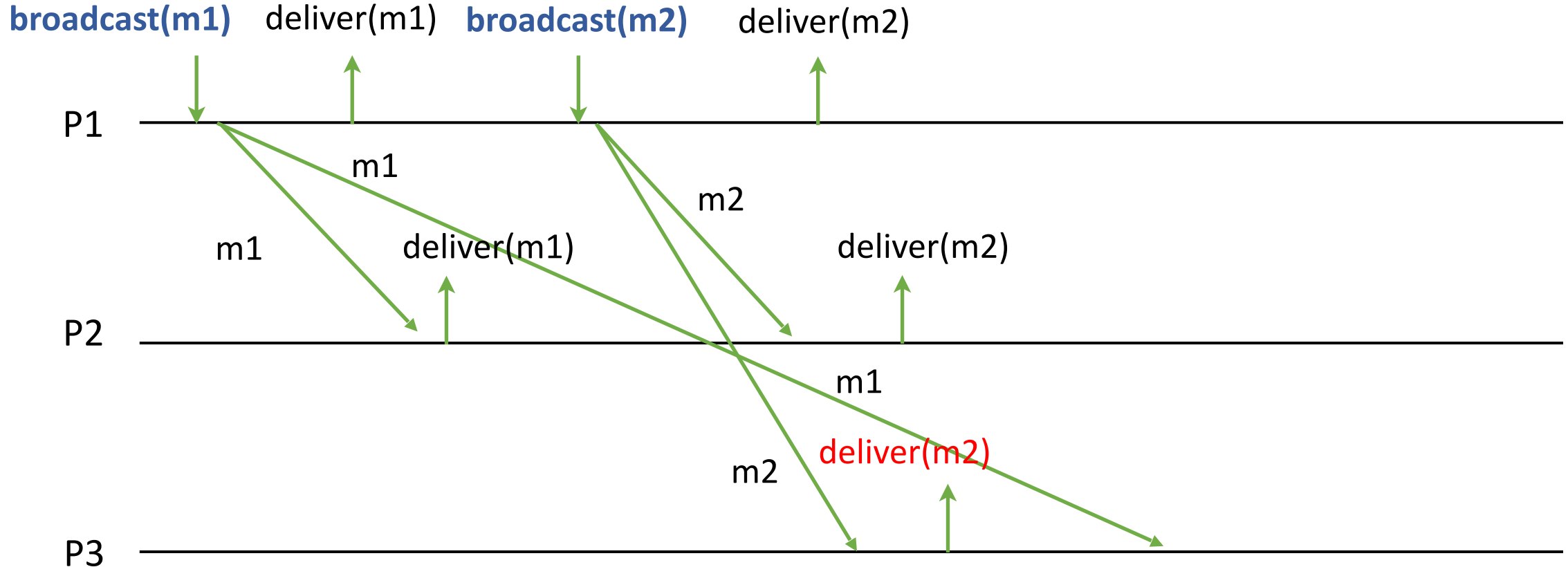
Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.



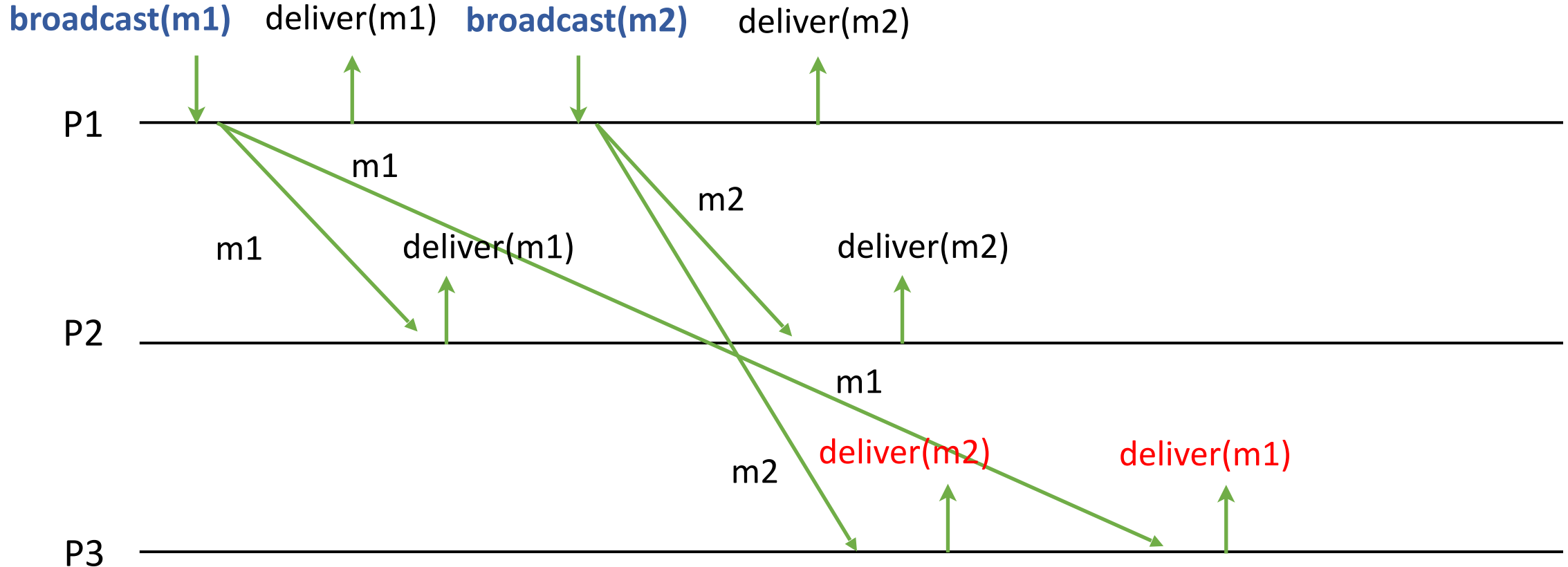
Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.



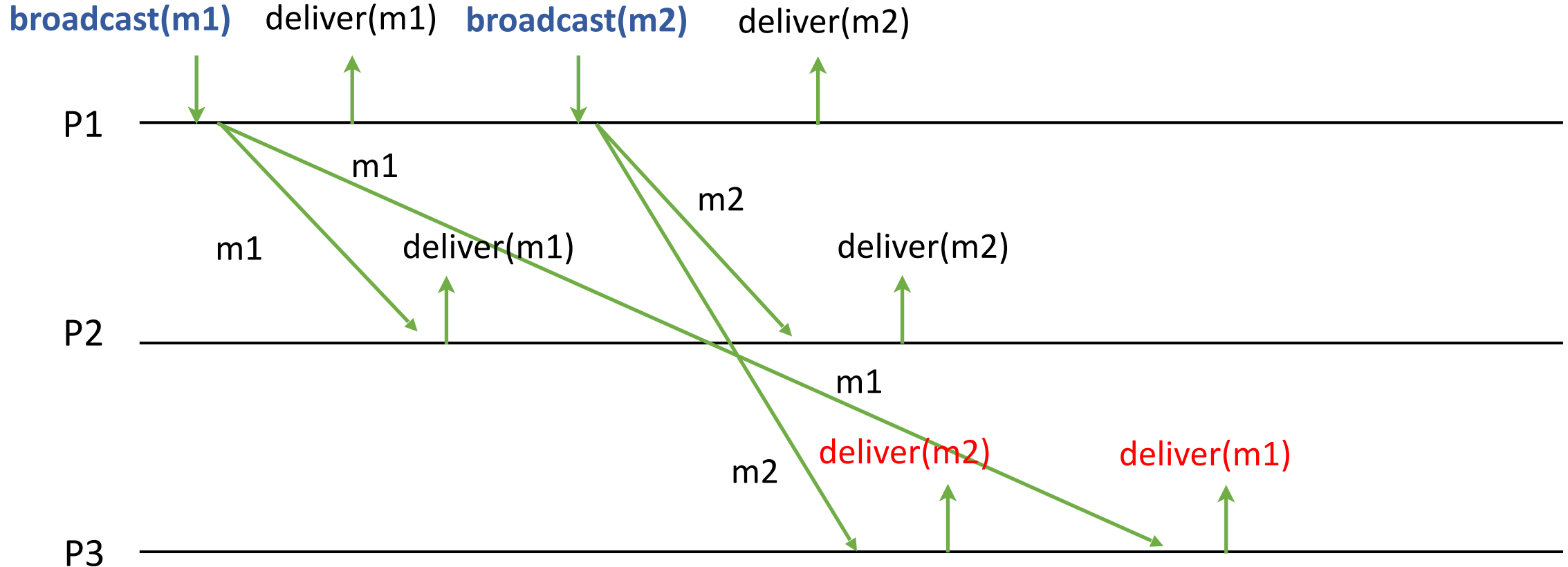
Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.



Example 1: FIFO Order

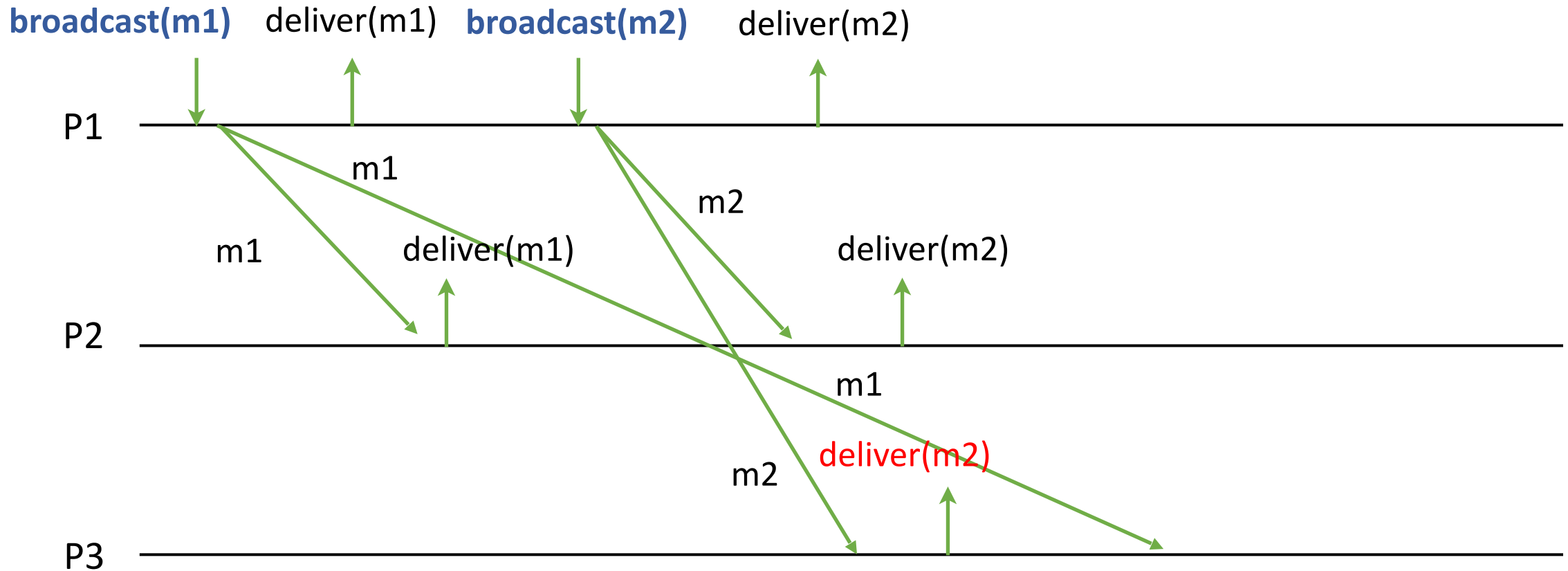
If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.



FIFO order of p_1 , $m_1 < m_2$, is not preserved.

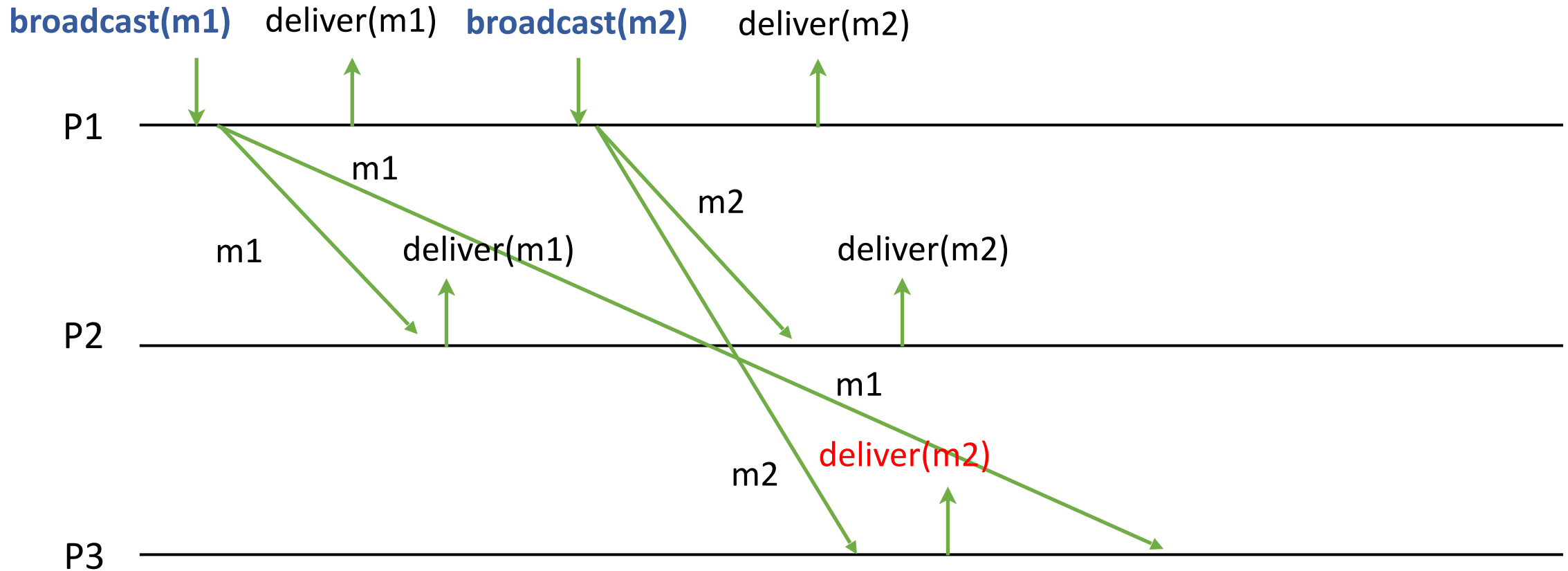
Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.



Example 1: FIFO Order

If a process p_i broadcasts m_1 before broadcasting m_2 then $m_1 < m_2$.



m_2 delivered but m_1 not delivered.
FIFO order of p_1 , $m_1 < m_2$, is not preserved.

Causal Relation (Dependency)

Let m_1 and m_2 be any two messages.

$m_1 < m_2$ (m_1 is causally before m_2 , or m_2 depends on m_1) iff

- **FIFO order:**

A process p_i broadcasts m_1 before broadcasting m_2 .

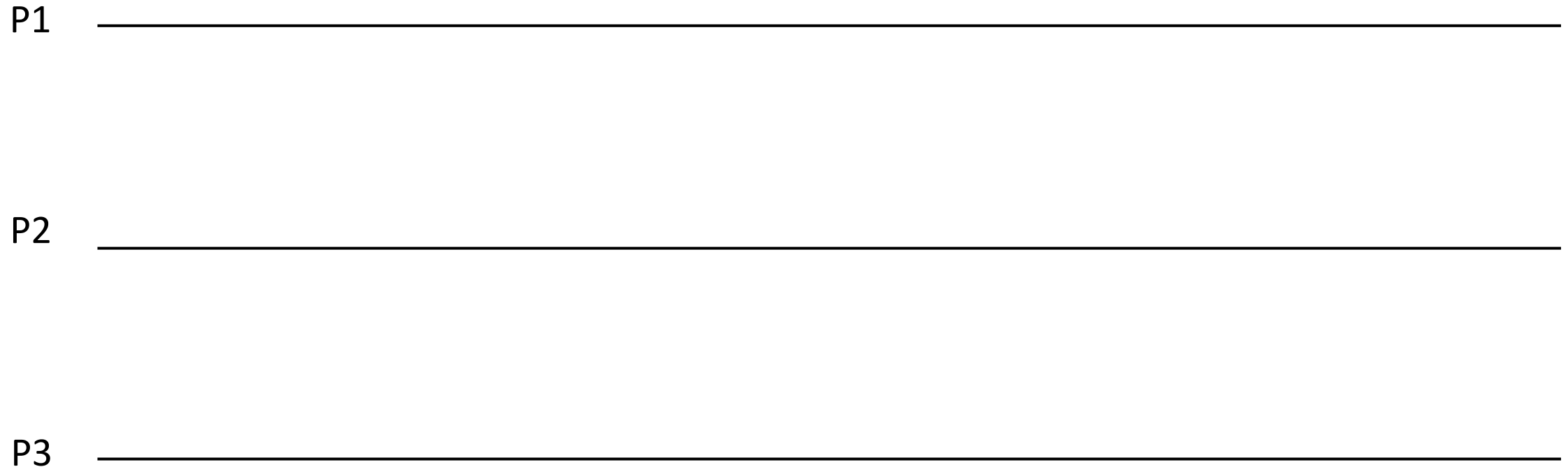
- **InOut order:**

A process p_i delivers m_1 and then broadcasts m_2 .

- ...

Example 2: InOut Order

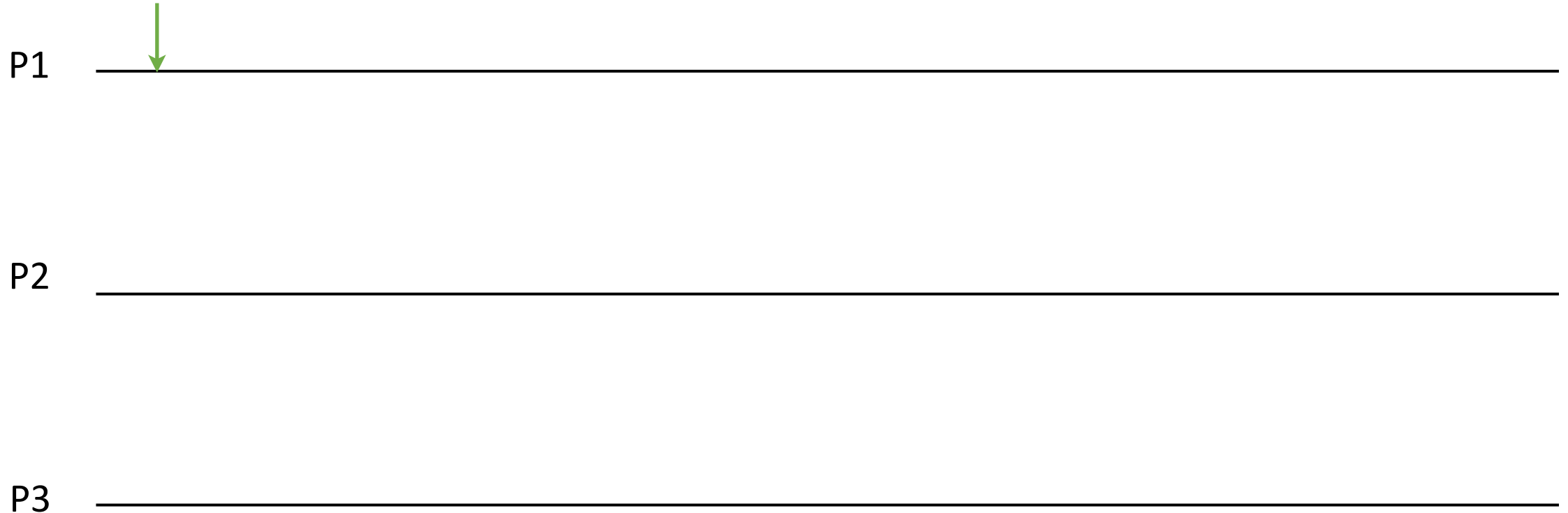
If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



Example 2: InOut Order

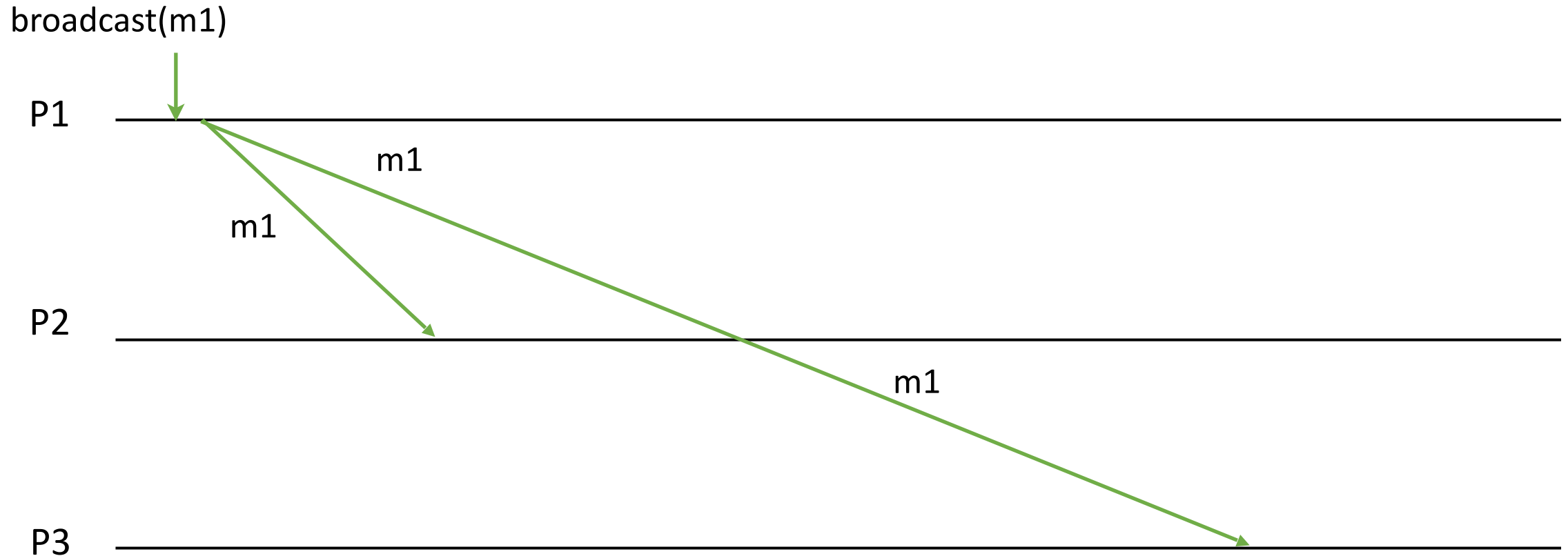
If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.

broadcast(m_1)



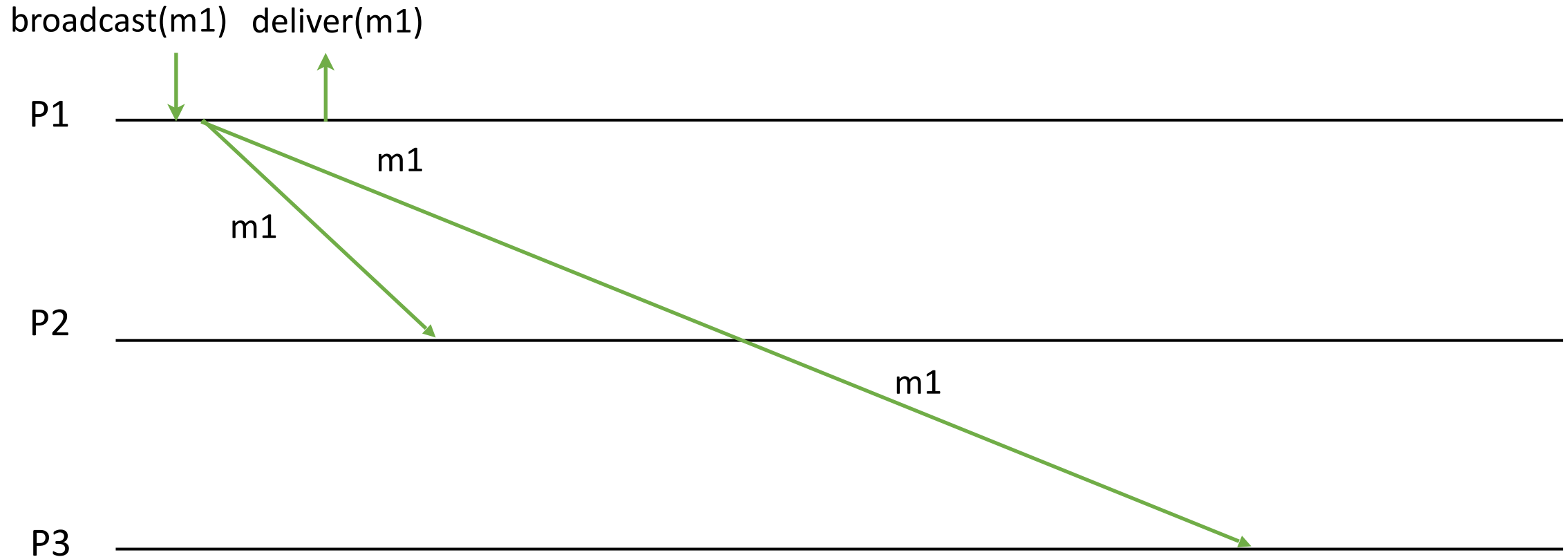
Example 2: InOut Order

If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



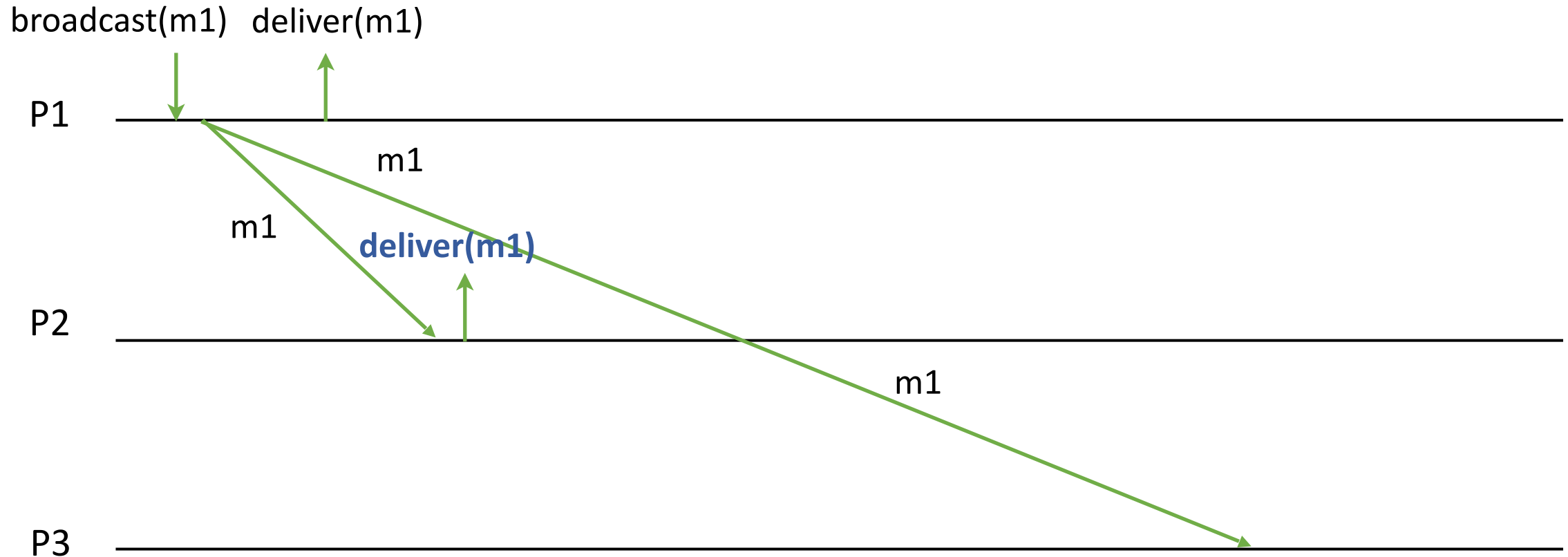
Example 2: InOut Order

If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



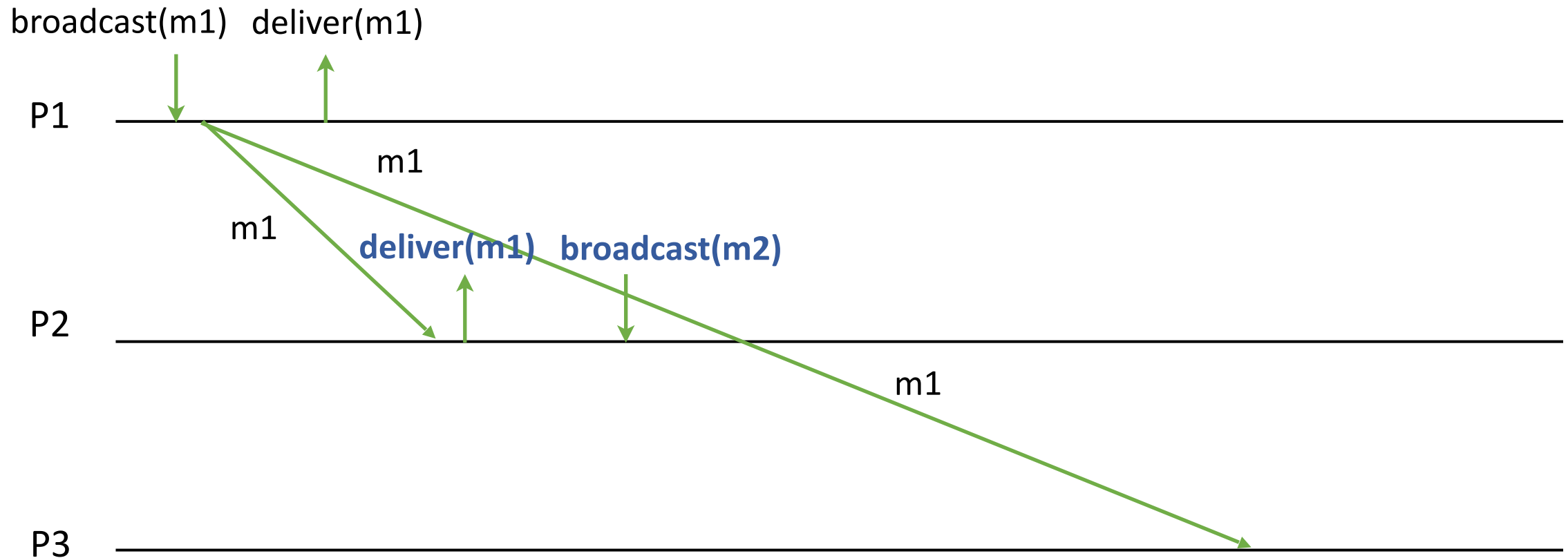
Example 2: InOut Order

If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



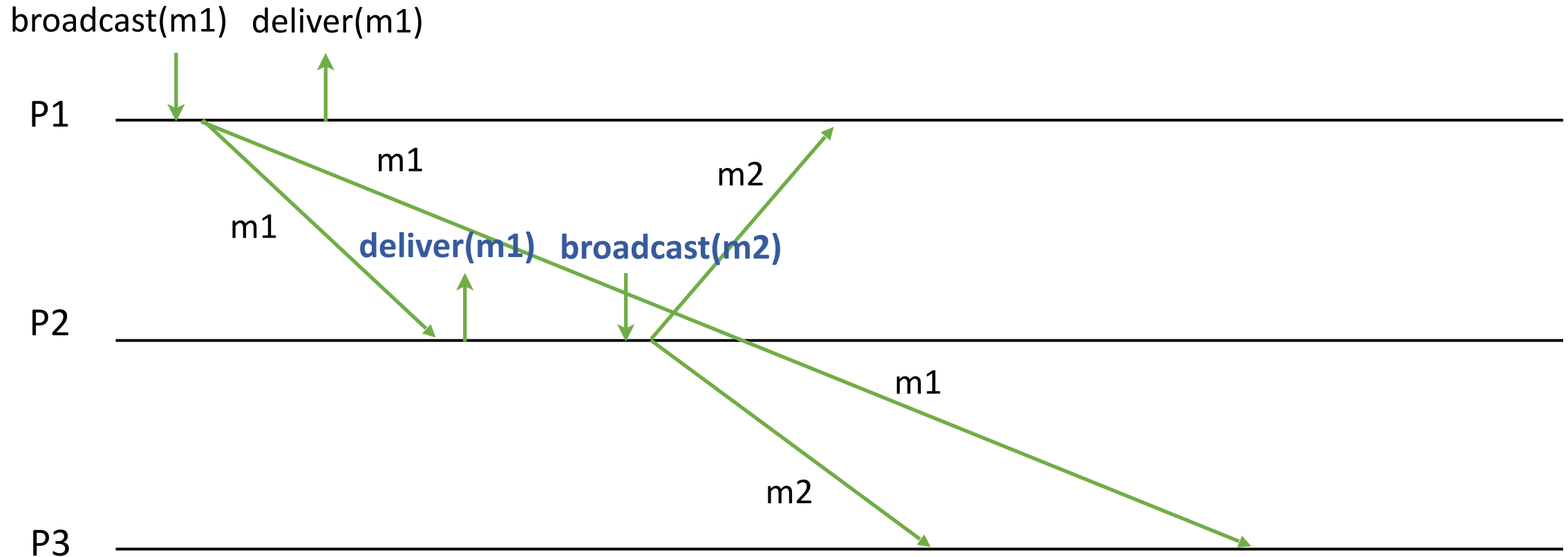
Example 2: InOut Order

If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



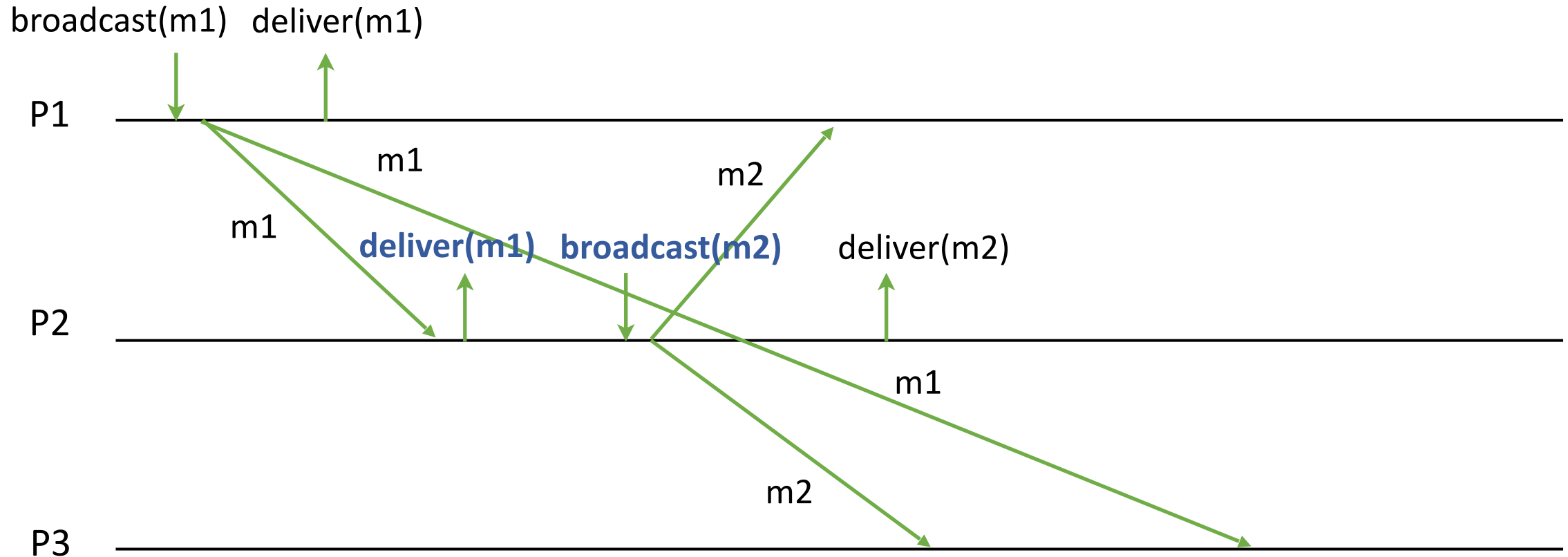
Example 2: InOut Order

If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



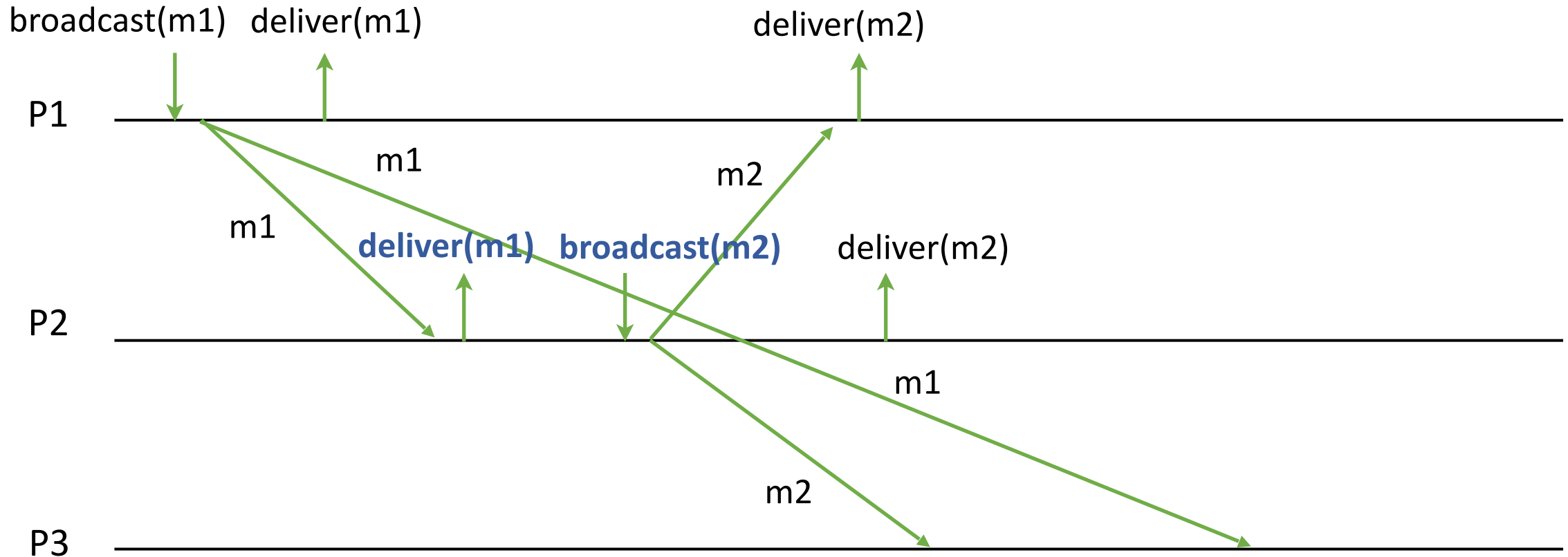
Example 2: InOut Order

If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



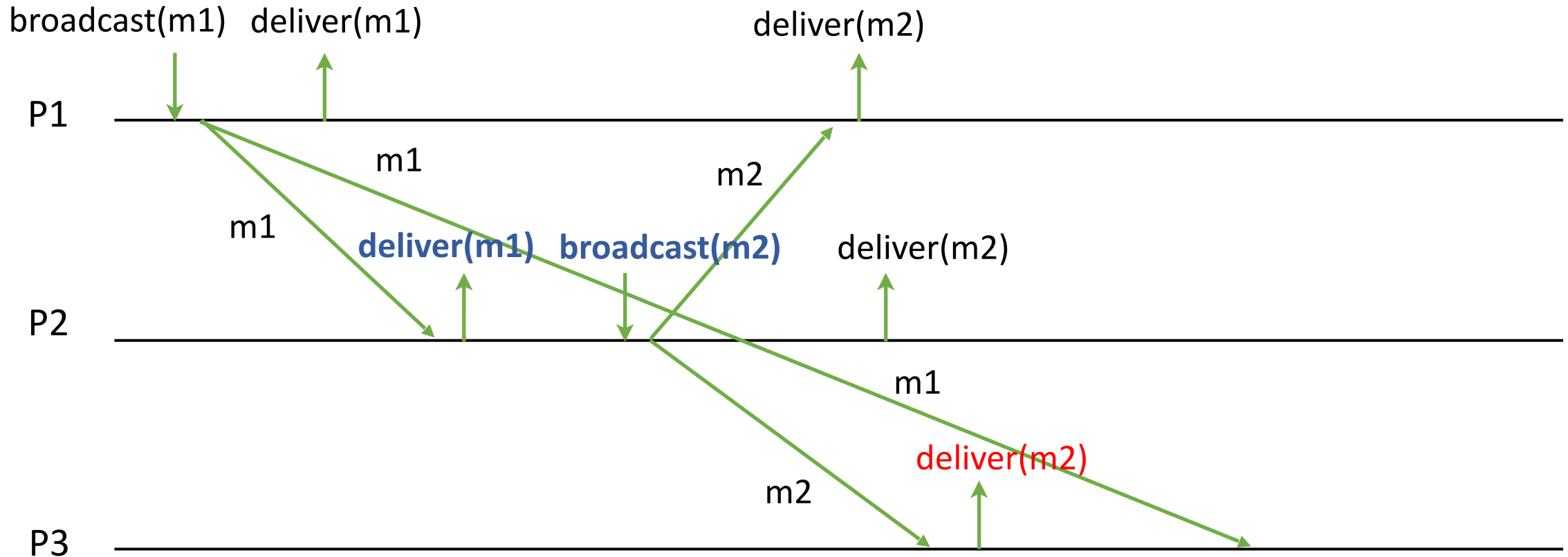
Example 2: InOut Order

If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



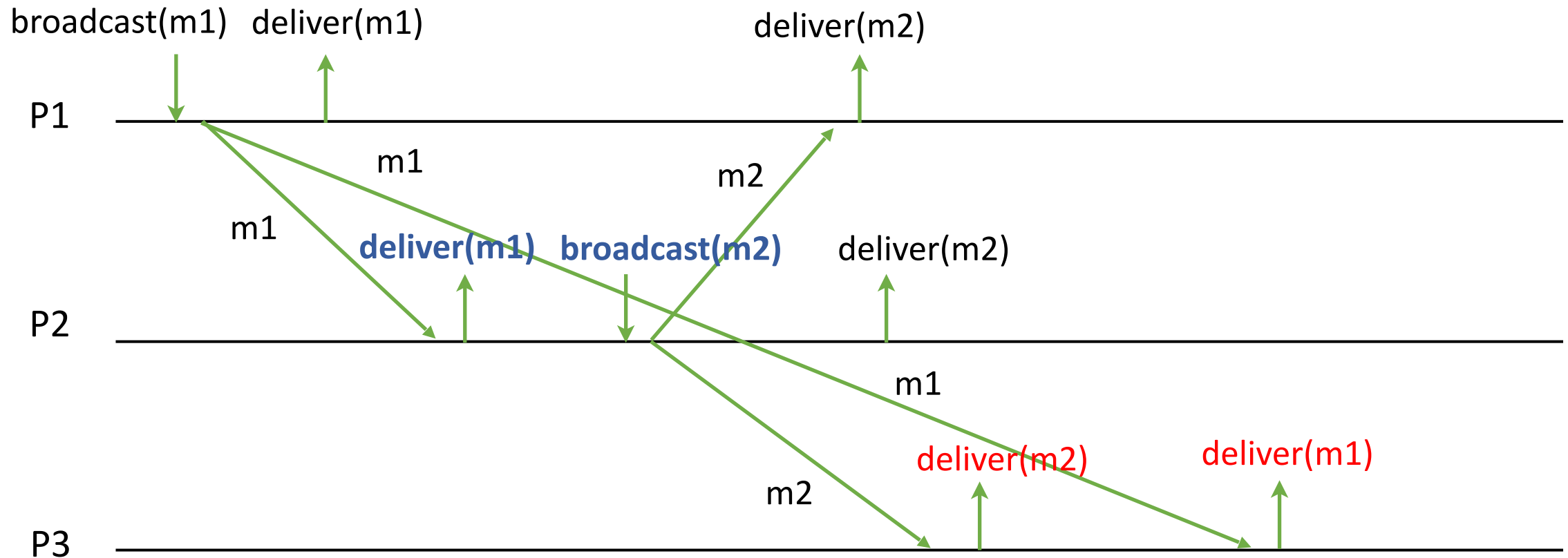
Example 2: InOut Order

If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



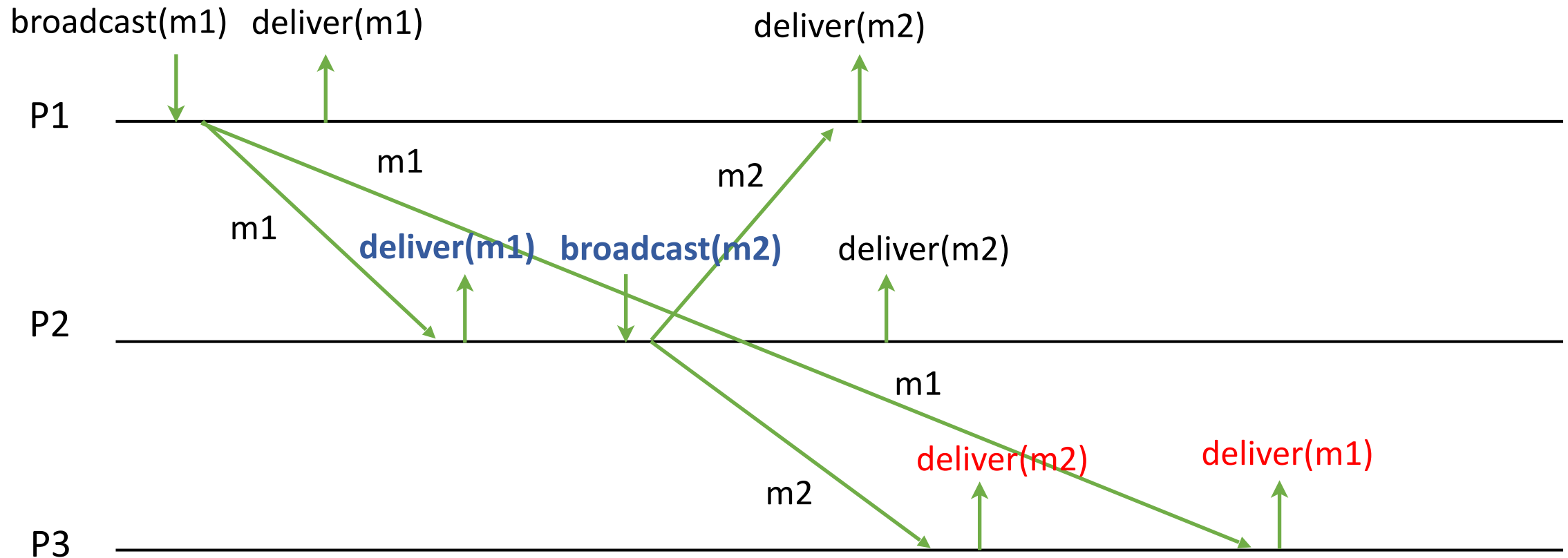
Example 2: InOut Order

If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



Example 2: InOut Order

If a process p_i delivers m_1 and then broadcasts m_2 then $m_1 < m_2$.



Local order of p_2 , $m_1 < m_2$, is not preserved.

Causal Relation (Dependency)

Let m_1 and m_2 be any two messages.

$m_1 < m_2$ (m_1 is causally before m_2 , or m_2 depends on m_1) iff

- **FIFO order:**

A process p_i broadcasts m_1 before broadcasting m_2 .

- **InOut order:**

A process p_i delivers m_1 and then broadcasts m_2 .

- **Transitivity:**

There is a message m_3 such that $m_1 < m_3$ and $m_3 < m_2$.

Uniform Causal Broadcast (UCB)

- **Events**

- Request: $\langle \text{broadcast } (m) \rangle$
- Indication: $\langle \text{deliver } (\text{src}, m) \rangle$

also called ucbBroadcast and ucbDeliver.

- **Properties:**

- URB1, URB2, URB3, URB4 +
- CO

Overview

- Motivation: why causal broadcast?
- Properties of causal broadcast
- **Protocols**

Protocols

How do we preserve the causal order?

Protocol 1

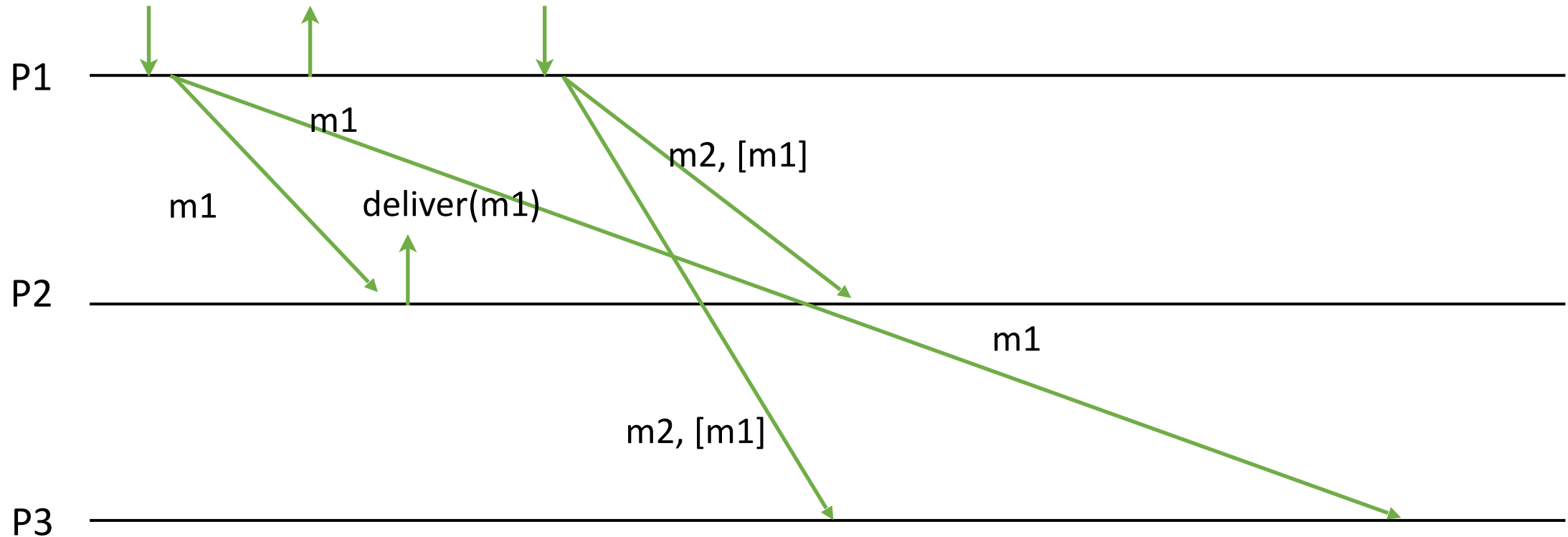


Idea:

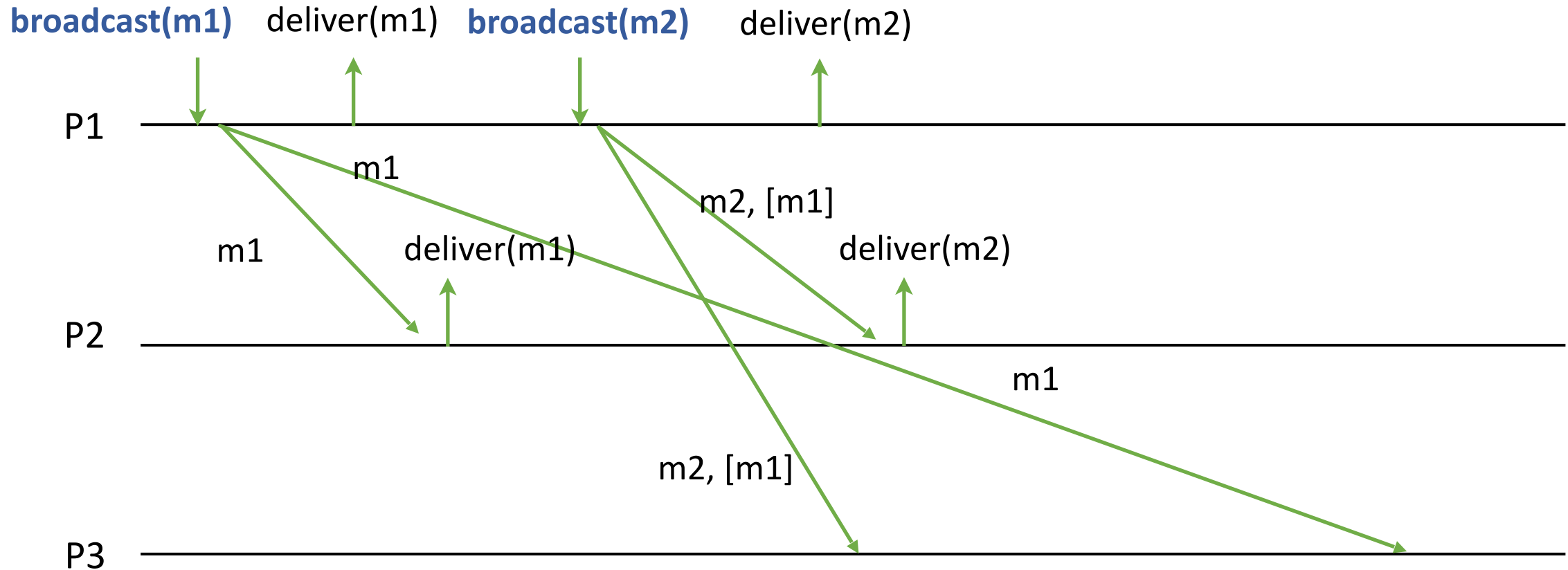
Remember the past messages and
sent them together with every new message.

Example 1: FIFO Order

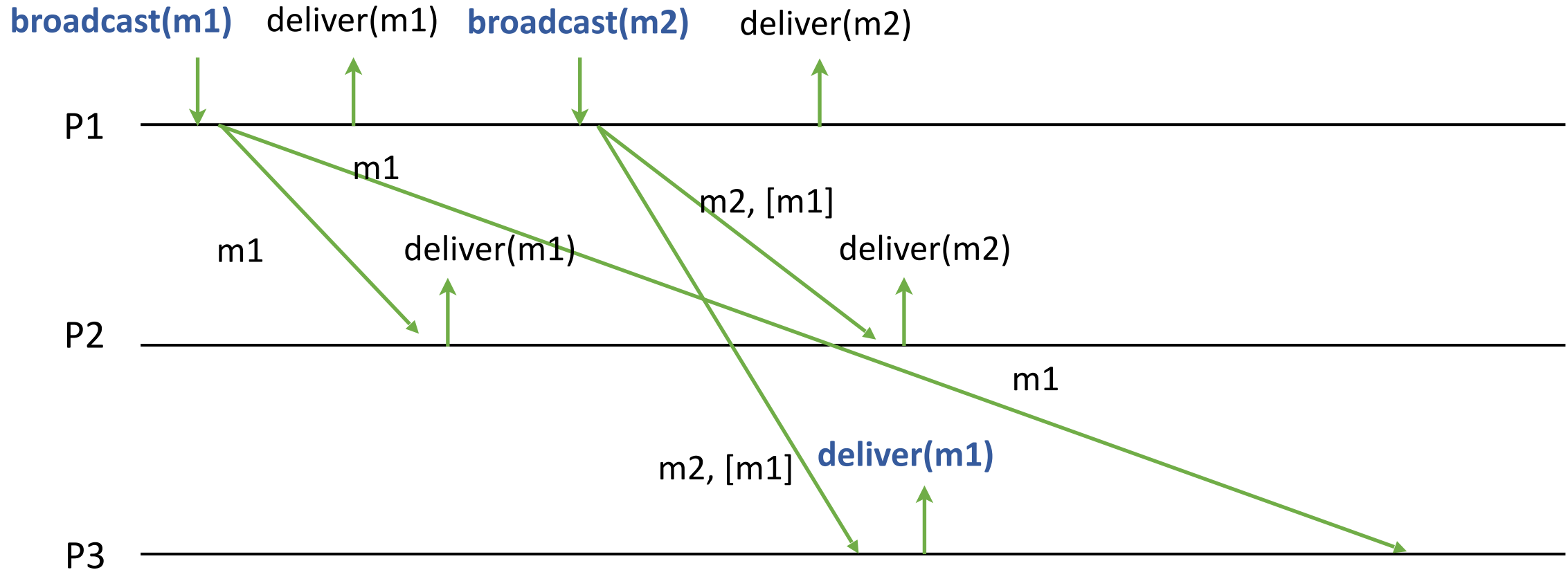
broadcast(m1) **deliver(m1)** **broadcast(m2)**



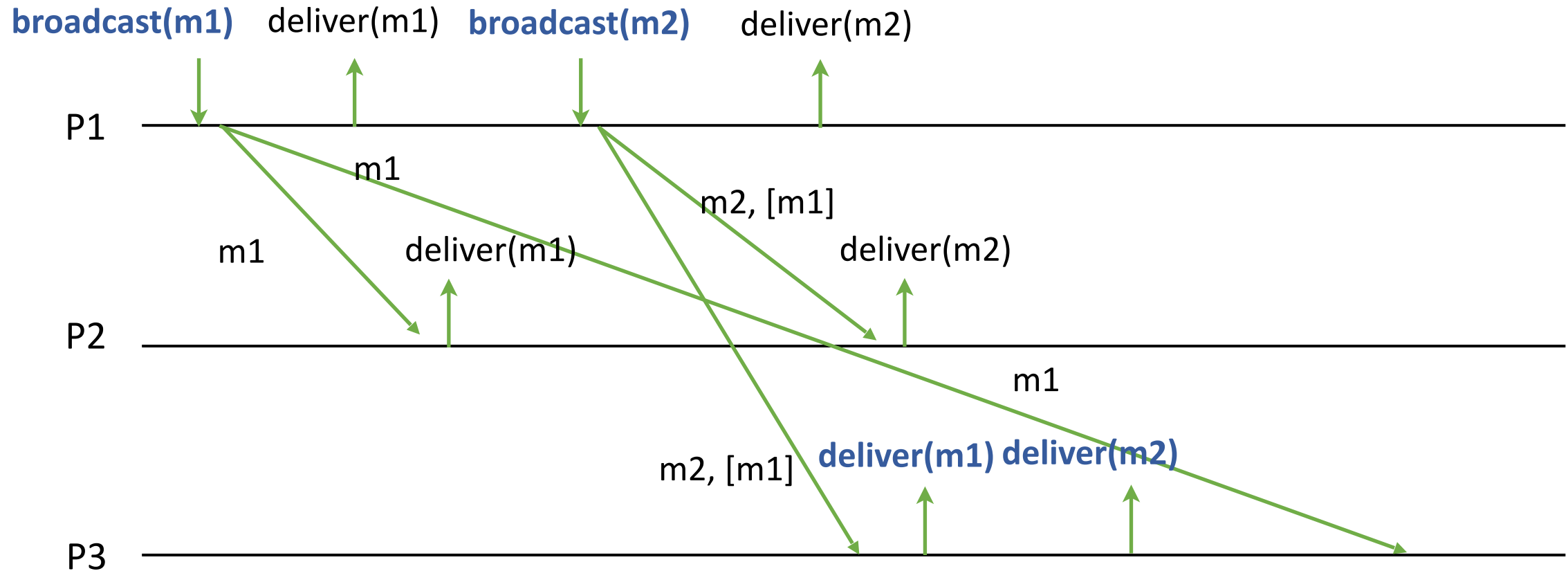
Example 1: FIFO Order



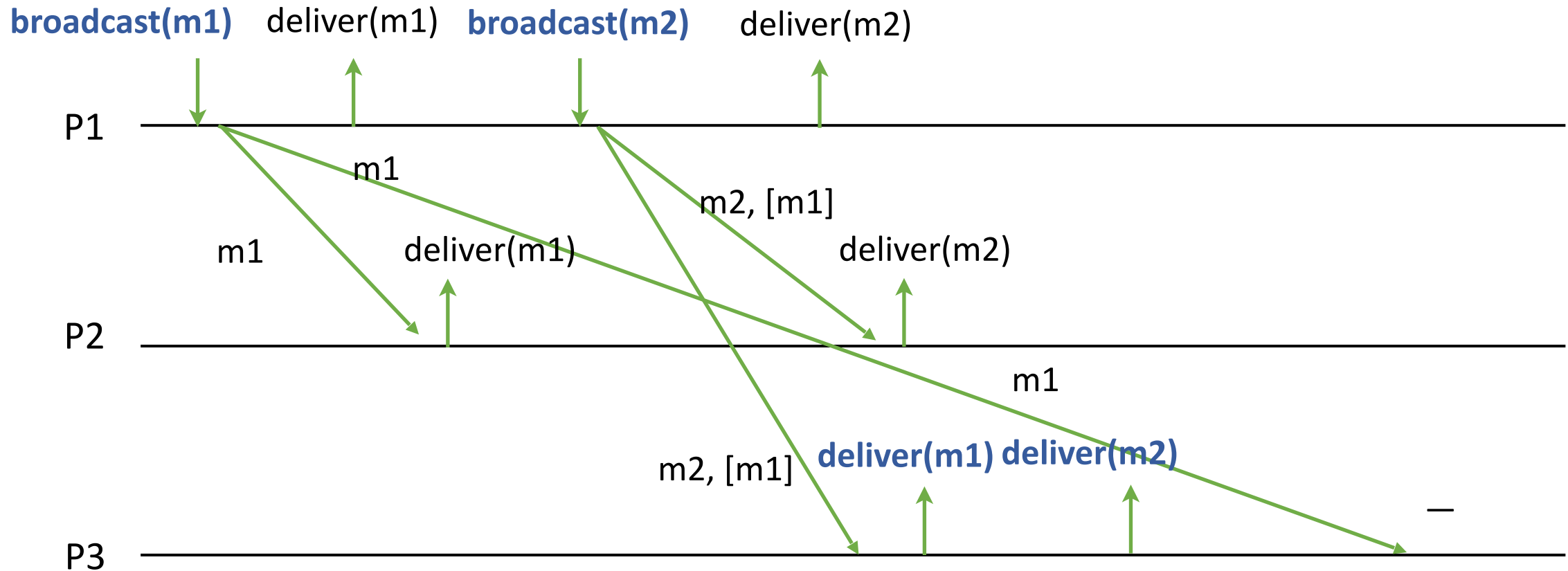
Example 1: FIFO Order



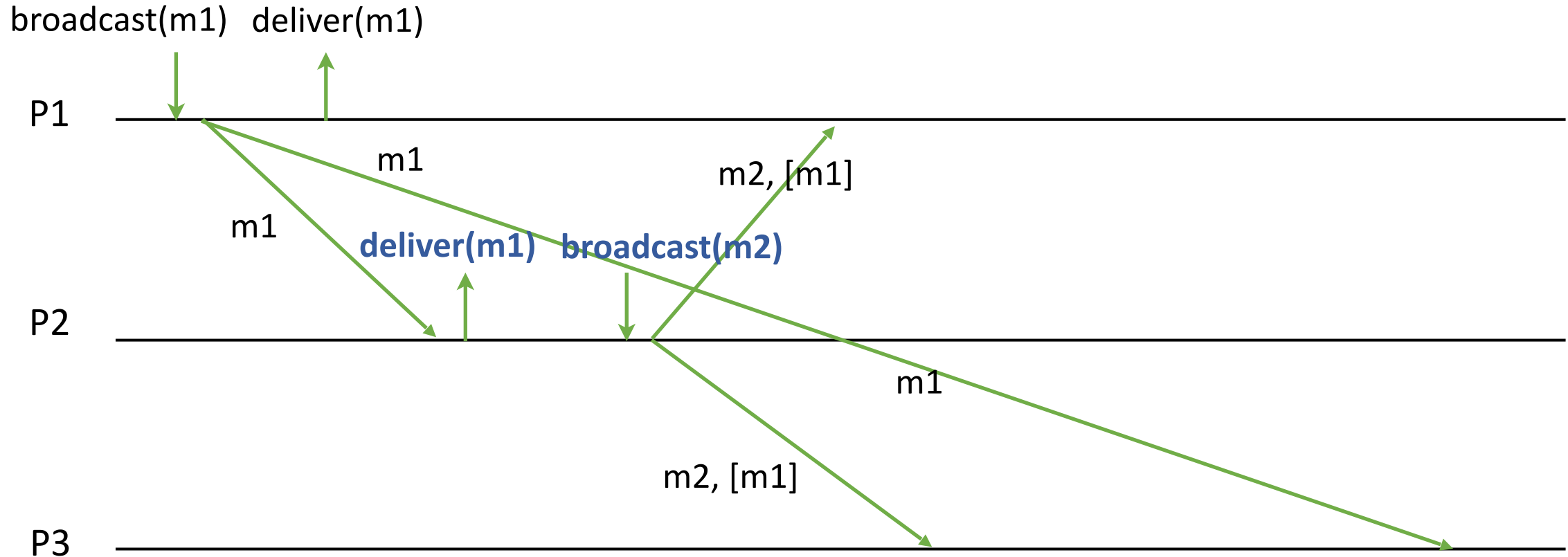
Example 1: FIFO Order



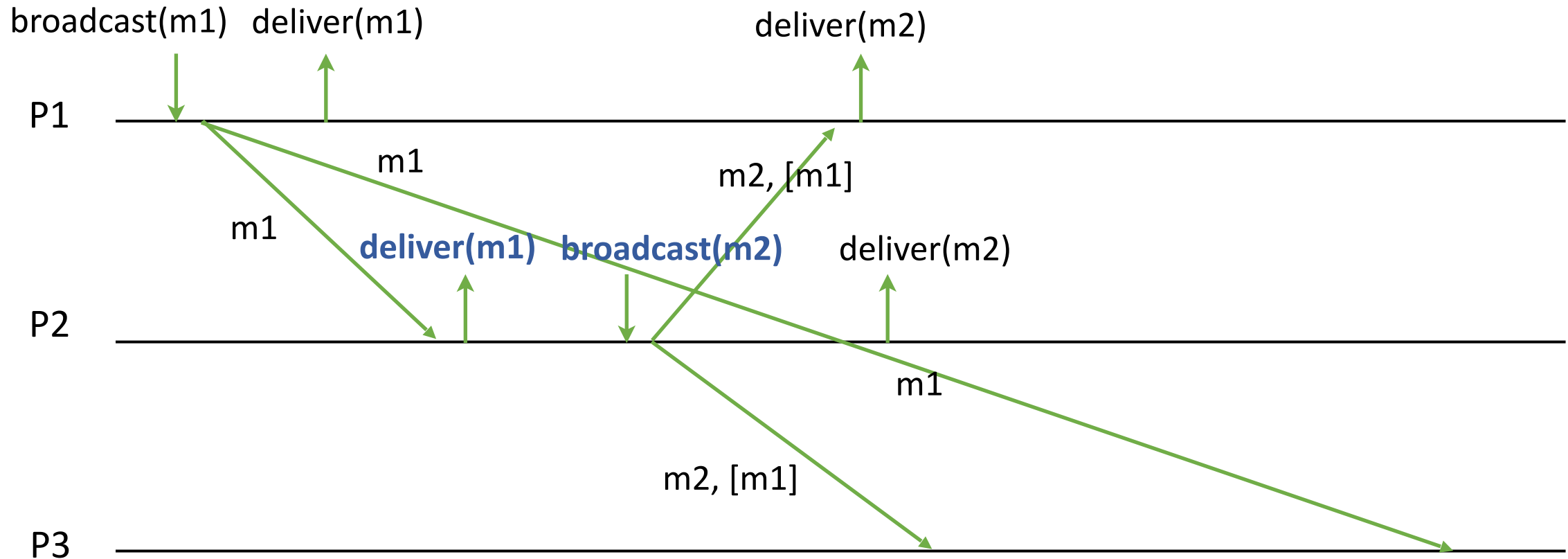
Example 1: FIFO Order



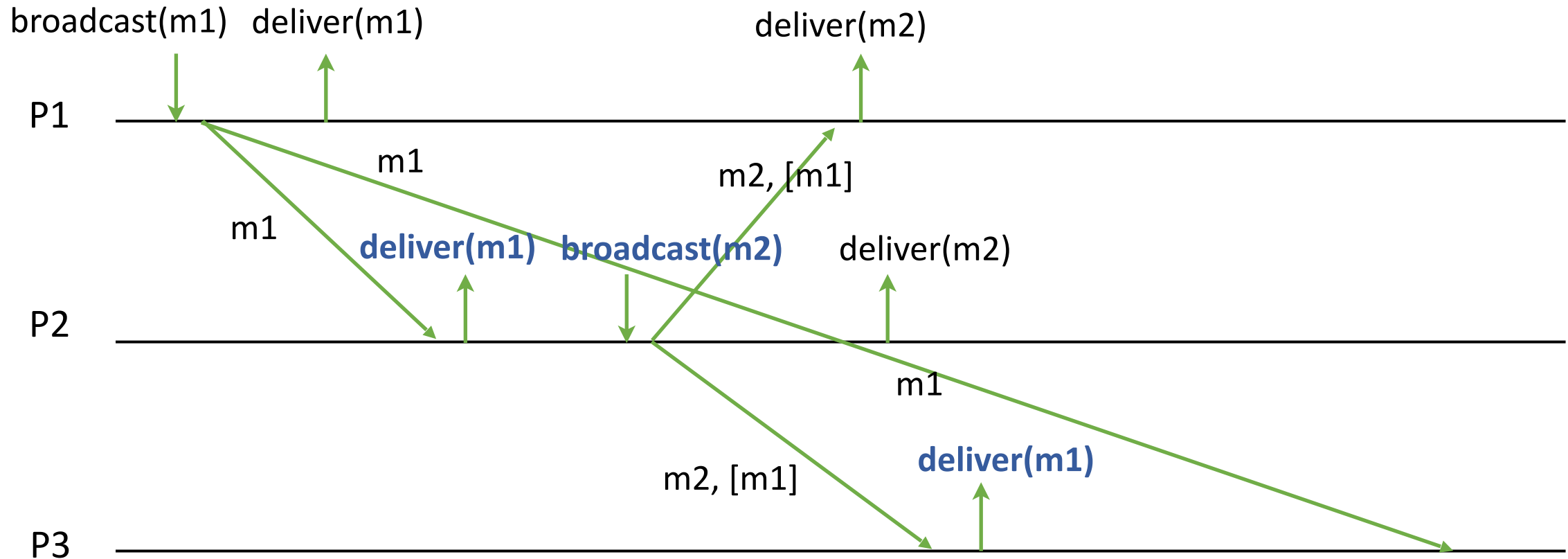
Example 2: InOut Order



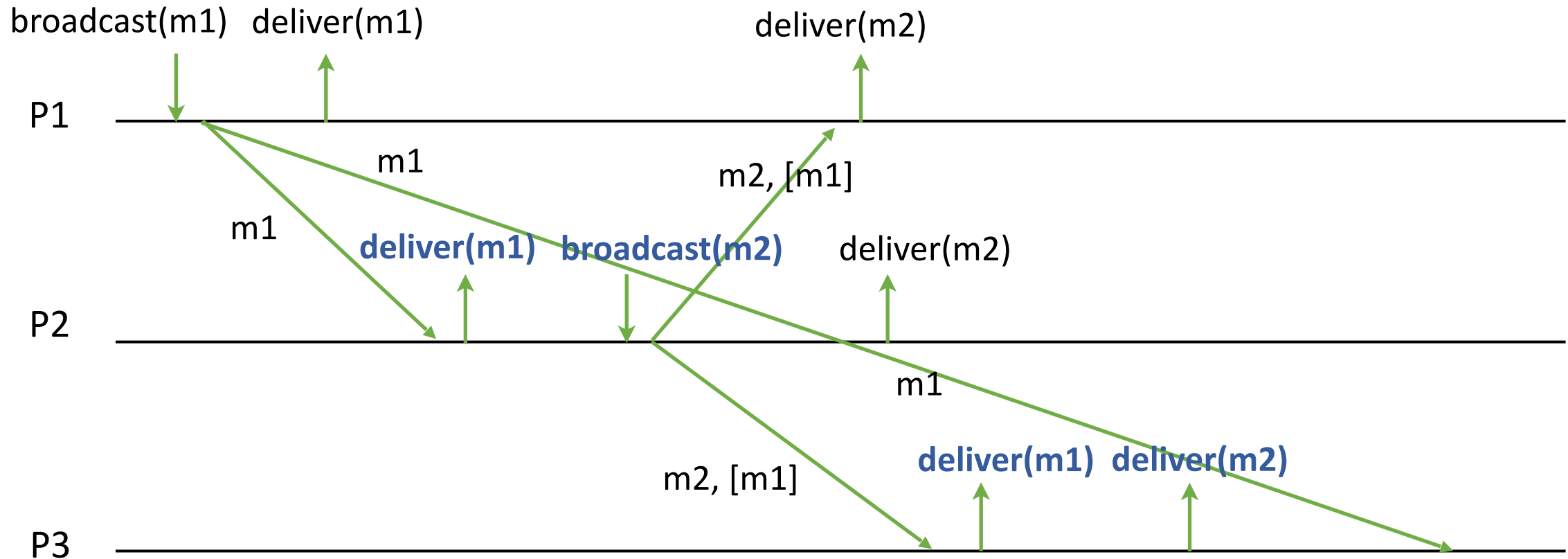
Example 2: InOut Order



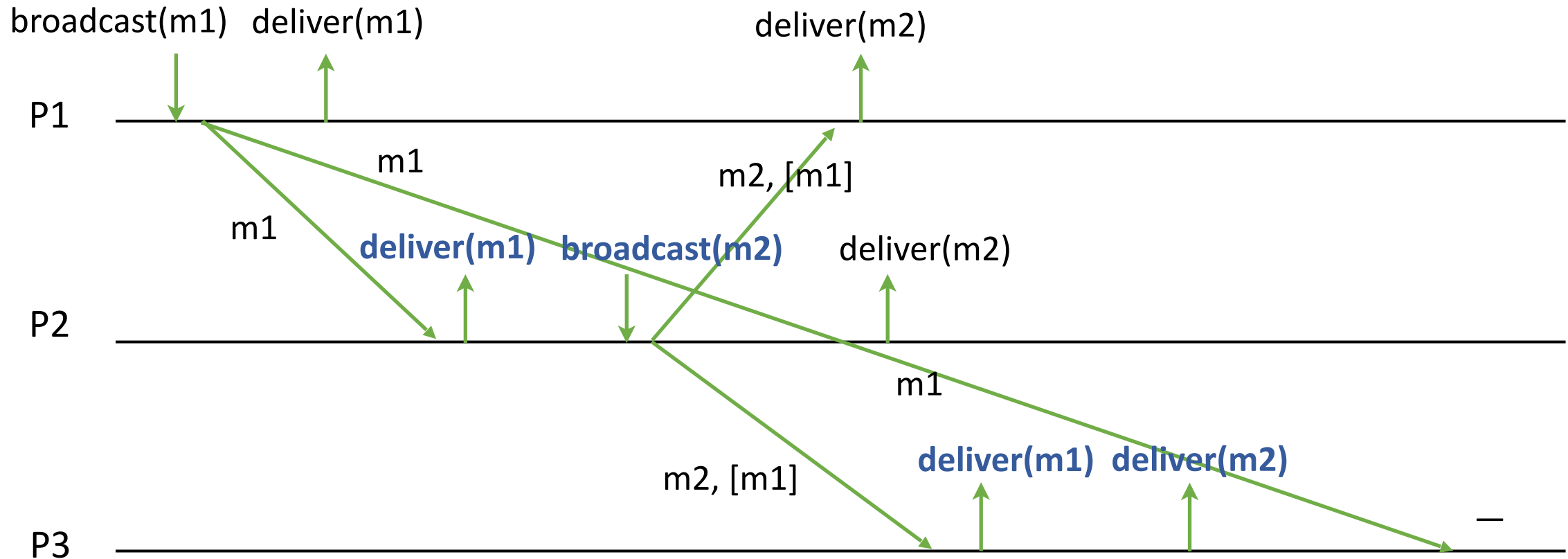
Example 2: InOut Order



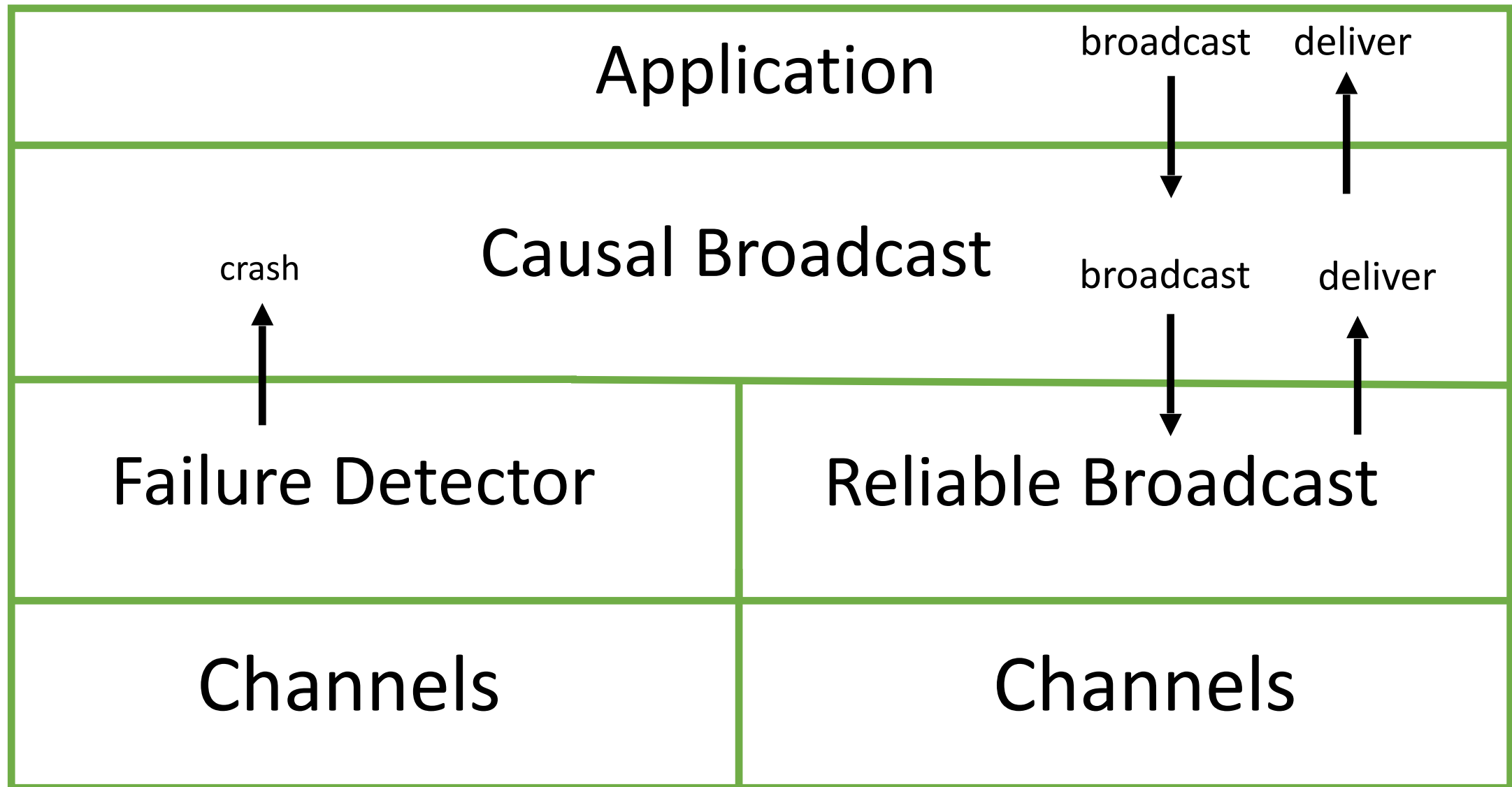
Example 2: InOut Order



Example 2: InOut Order



Modular Design



Protocol 1

Implements: UniformCausalBroadcast (ucb).

Uses: UniformReliableBroadcast (urb).

Protocol 1

Implements: UniformCausalBroadcast (ucb).

Uses: UniformReliableBroadcast (urb).

Similarly Reliable broadcast can be used to get Reliable causal broadcast.

Protocol 1

Implements: UniformCausalBroadcast (ucb).

Uses: UniformReliableBroadcast (urb).

upon event < Init > **do**

delivered := past := $\emptyset$

Similarly Reliable broadcast can be used to get Reliable causal broadcast.

Protocol 1

Implements: UniformCausalBroadcast (ucb).

Uses: UniformReliableBroadcast (urb).

upon event < Init > **do**

delivered := past := $\emptyset$

upon event < broadcast (m) > **do**

trigger < urb, broadcast ([past, m]) >

past := past $\cup$ {[self, m]}

Similarly Reliable broadcast can be used to get Reliable causal broadcast.

Protocol 1

Implements: UniformCausalBroadcast (ucb).

Uses: UniformReliableBroadcast (urb).

upon event < Init > **do**

delivered := past := $\emptyset$

upon event < broadcast (m) > **do**

trigger < urb, broadcast ([past, m]) >

past := past $\cup$ { [self, m] }

Similarly Reliable broadcast can be used to get Reliable causal broadcast.

Every new message is added to past. This preserves the FIFO order property.

Protocol 1

```
upon event <urb, deliver (pi, [pastm, m])> do  
  if  $m \notin \text{delivered}$  then  
    forall  $[sn, n] \in \text{pastm}$  do  
      if  $n \notin \text{delivered}$  then  
        trigger < deliver (sn, n) >  
        delivered := delivered  $\cup$  {n}  
        past := past  $\cup$  {[sn, n]}
```

Protocol 1

```
upon event <urb, deliver (pi, [pastm, m])> do  
  if  $m \notin \text{delivered}$  then  
    forall  $[sn, n] \in \text{pastm}$  do  
      if  $n \notin \text{delivered}$  then  
        trigger < deliver (sn, n) >  
        delivered := delivered  $\cup \{n\}$   
        past := past  $\cup \{[sn, n]\}$ 
```

The set pastm is added to past.
This preserves the transitivity property.

Protocol 1

```
upon event <urb, deliver (pi, [pastm, m])> do  
  if  $m \notin \text{delivered}$  then  
    forall  $[sn, n] \in \text{pastm}$  do  
      if  $n \notin \text{delivered}$  then  
        trigger < deliver (sn, n) >  
        delivered := delivered  $\cup$  {n}  
        past := past  $\cup$  {[sn, n]}  
  
  trigger < deliver (pi, m) >  
  delivered := delivered  $\cup$  {m}  
  past := past  $\cup$  {[pi, m]}
```

The set pastm is added to past.
This preserves the transitivity property.

Protocol 1

upon event $\langle \text{urb}, \text{deliver}(\text{pi}, [\text{pastm}, \text{m}]) \rangle$ **do**

if $\text{m} \notin \text{delivered}$ **then**

forall $[\text{sn}, \text{n}] \in \text{pastm}$ **do**

if $\text{n} \notin \text{delivered}$ **then**

trigger $\langle \text{deliver}(\text{sn}, \text{n}) \rangle$

$\text{delivered} := \text{delivered} \cup \{\text{n}\}$

$\text{past} := \text{past} \cup \{[\text{sn}, \text{n}]\}$

trigger $\langle \text{deliver}(\text{pi}, \text{m}) \rangle$

$\text{delivered} := \text{delivered} \cup \{\text{m}\}$

$\text{past} := \text{past} \cup \{[\text{pi}, \text{m}]\}$

The set pastm is added to past .
This preserves the transitivity property.

Every delivered message is added to past .
This preserves the InOut property.

Observation

If we keep remembering the past,
we eventually run out of space!

Protocol 1 + Garbage Collection



Idea:

- Broadcast an ack when a message is delivered.
- Forget a message after receiving acks from all correct processes.

Protocol 1 + Garbage Collection

Implements: CausalOrderBroadcast (co).

Uses:

ReliableBroadcast (rb).

PerfectFailureDetector (P).

upon event < Init > **do**

...

correct := Π

ack(m) := $\emptyset$ (for all m)

Protocol 1 + Garbage Collection

upon event $\langle P, \text{crash}(p_i) \rangle$ **do**
 $\text{correct} := \text{correct} \setminus \{p_i\}$

Protocol 1 + Garbage Collection

upon event < P, crash (pi) > **do**

correct := correct \ {pi}

upon event <urb, deliver (p, Msg[pastm, m])> **do**

...

trigger <urb, broadcast (Ack[p, m])>

Protocol 1 + Garbage Collection

upon event $\langle P, \text{crash}(p_i) \rangle$ **do**

$\text{correct} := \text{correct} \setminus \{p_i\}$

upon event $\langle \text{urb}, \text{deliver}(p, \text{Msg}[\text{pastm}, m]) \rangle$ **do**

...

trigger $\langle \text{urb}, \text{broadcast}(\text{Ack}[p, m]) \rangle$

upon event $\langle \text{urb}, \text{deliver}(p, \text{Ack}[s, m]) \rangle$ **do**

$\text{ack}(m) := \text{ack}(m) \cup \{p\}$

if $\text{correct} \subseteq \text{ack}(m)$ **then**

$\text{past} := \text{past} \setminus \{[s, m]\}$

Observation

Messages that carry the past are large!

Protocol 2



Idea:

- Keep the number of messages delivered from each process as a vector (clock) of numbers.*

p_1	p_2	p_3
2	1	0

- Send the vector clock (VC) together with new messages.
(*except for the current process that is updated with the number of broadcast messages.)

Protocol 2



- Deliver a message only if the local vector clock is larger than (or equal to) the vector clock of the message.

p_1	p_2	p_3
2	1	0

Message's VC

$\leq$

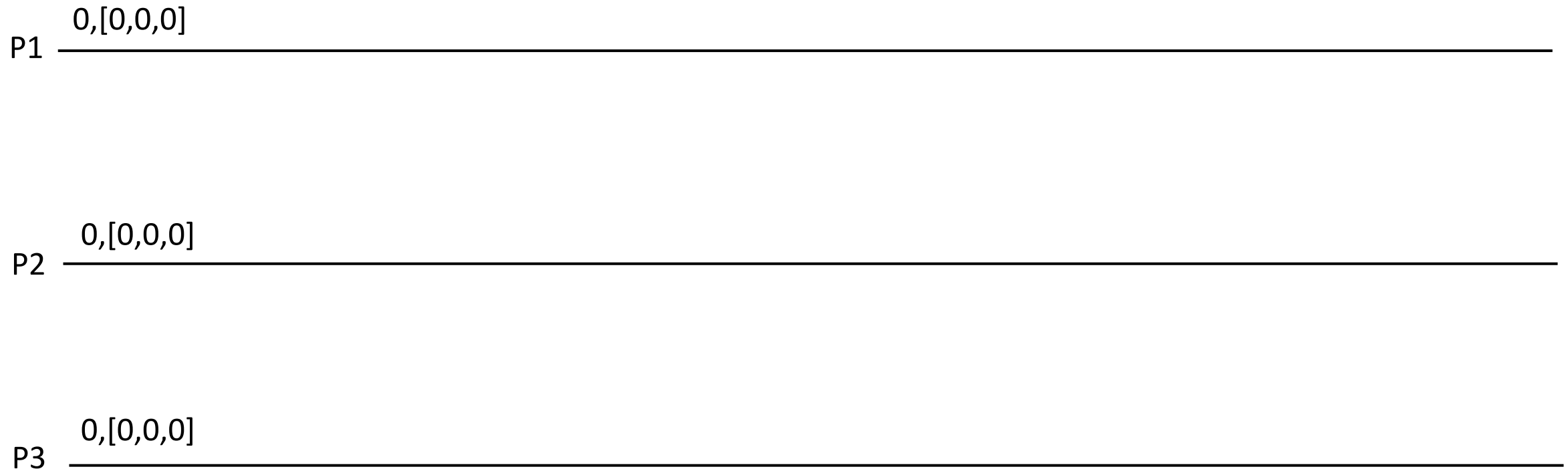
p_1	p_2	p_3
3	2	0

Receiver's VC

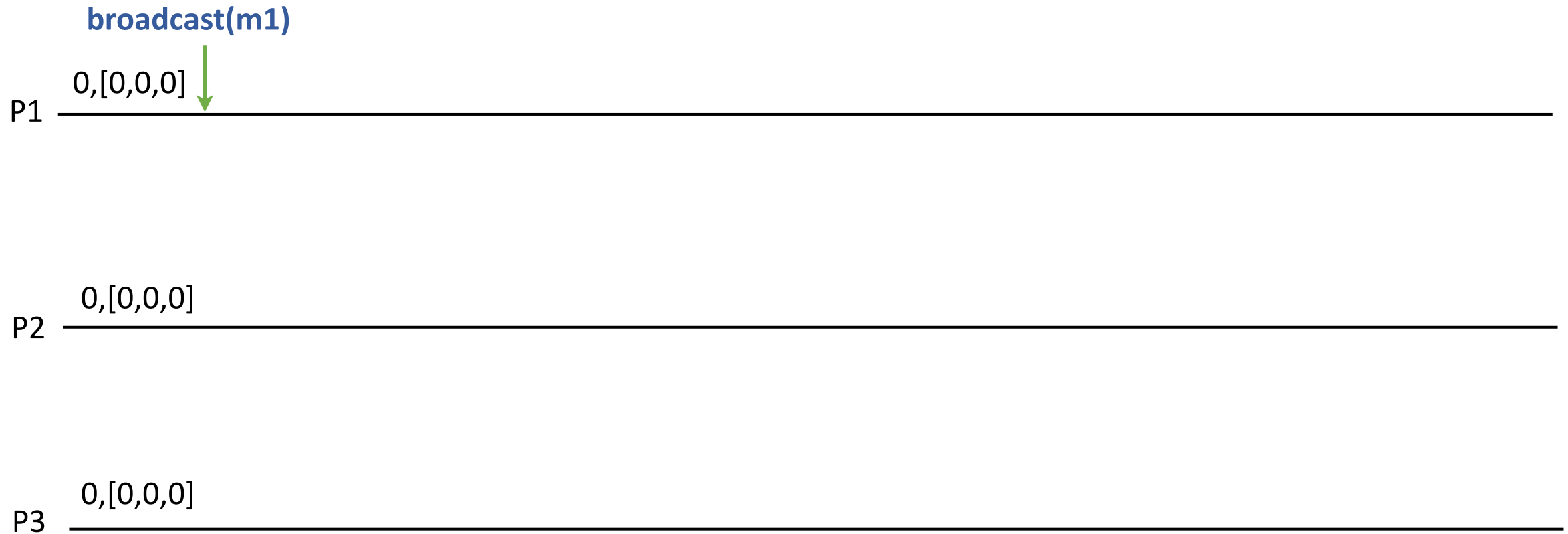
Example 1: FIFO Order



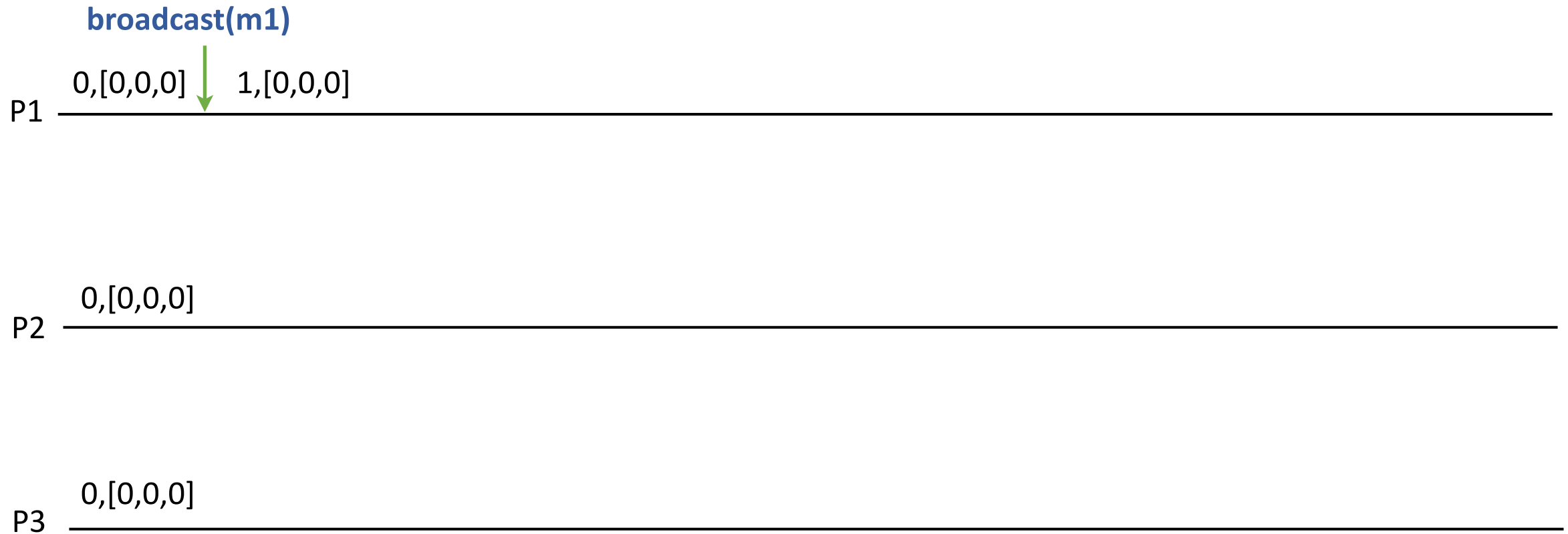
Example 1: FIFO Order



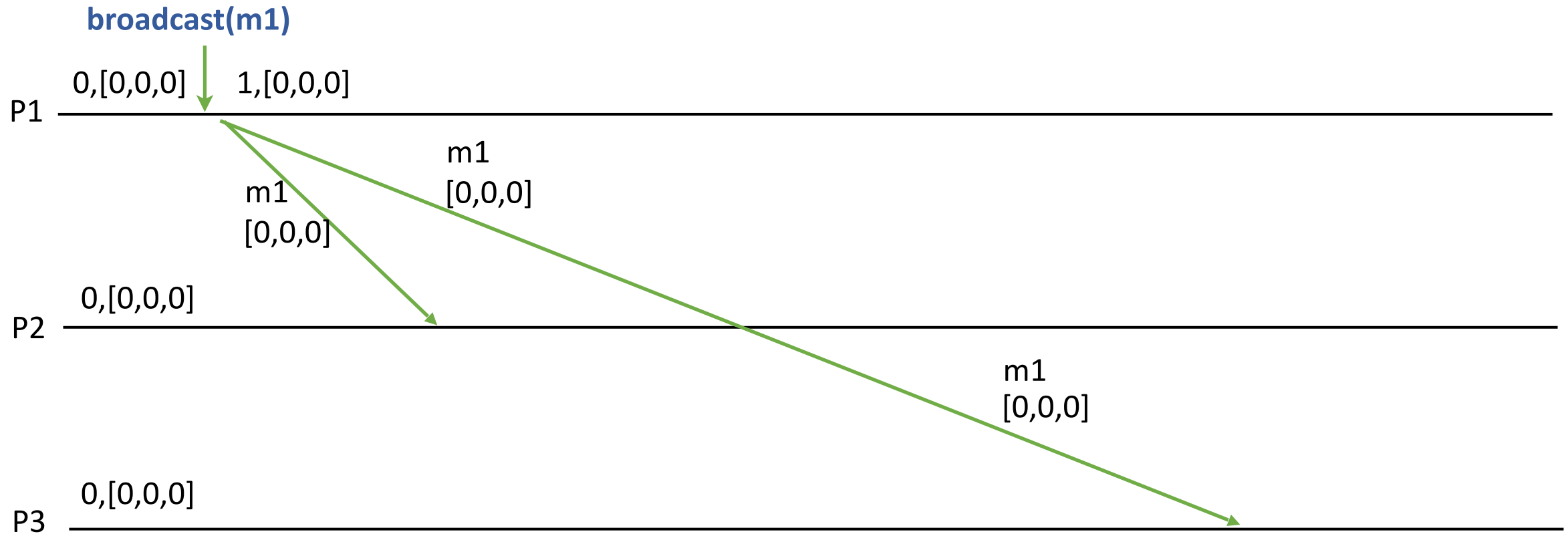
Example 1: FIFO Order



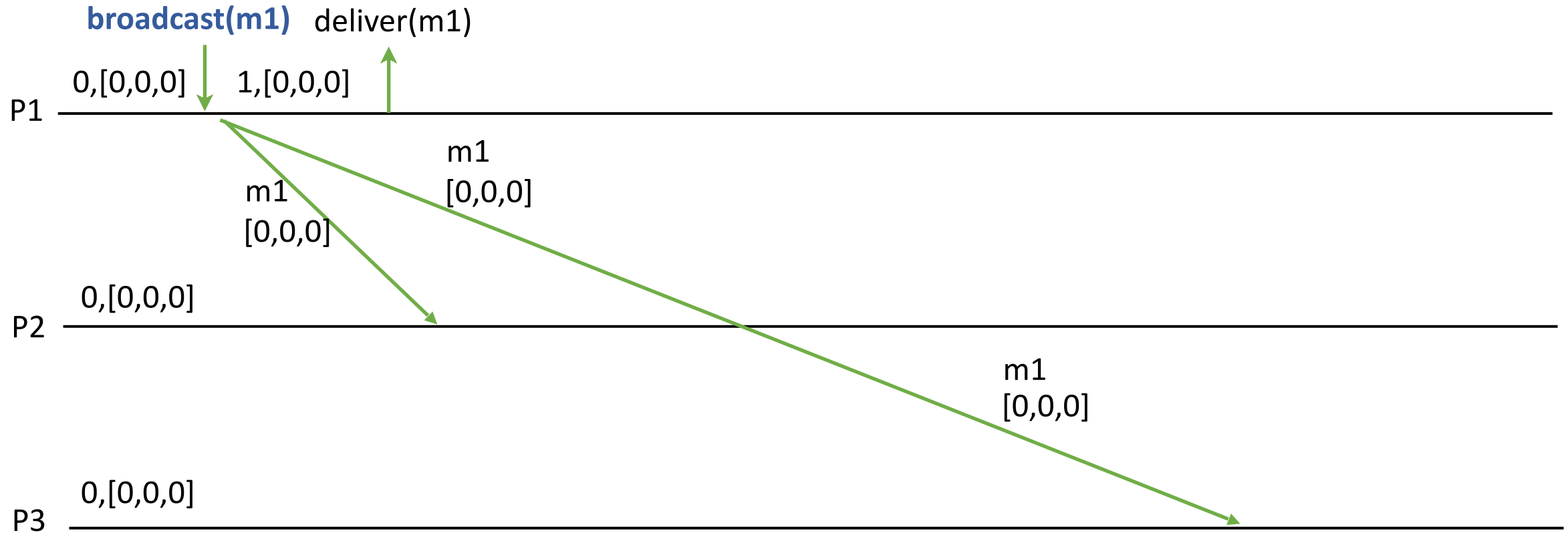
Example 1: FIFO Order



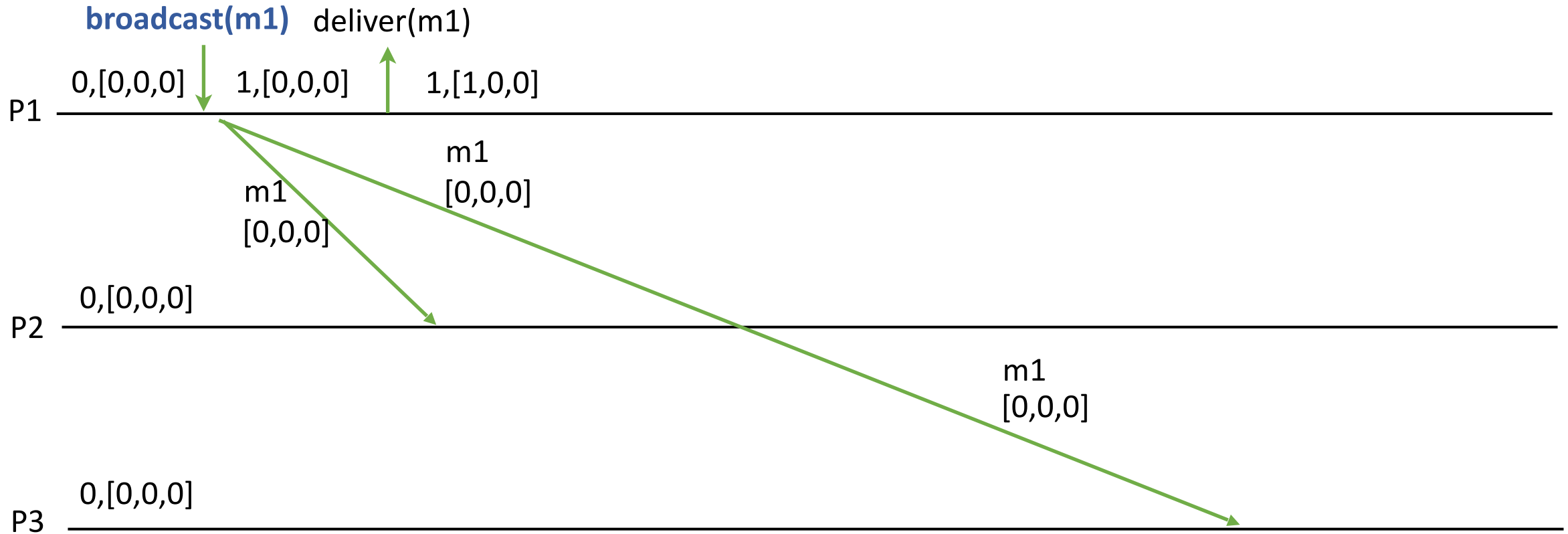
Example 1: FIFO Order



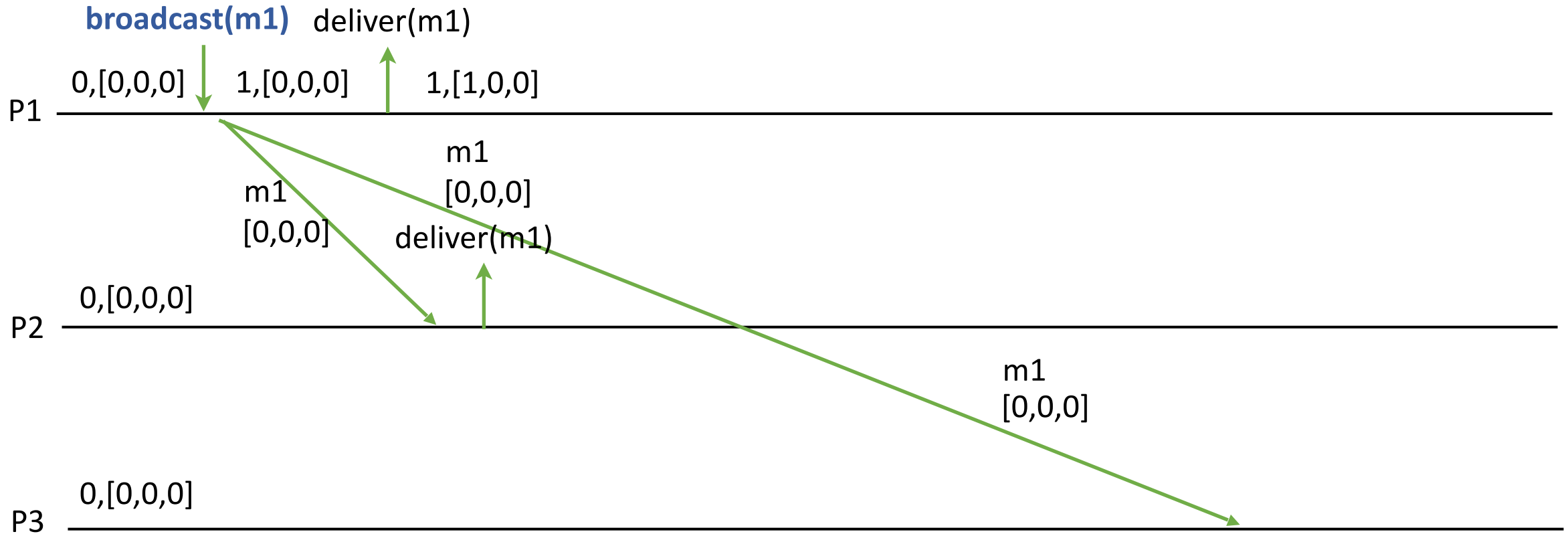
Example 1: FIFO Order



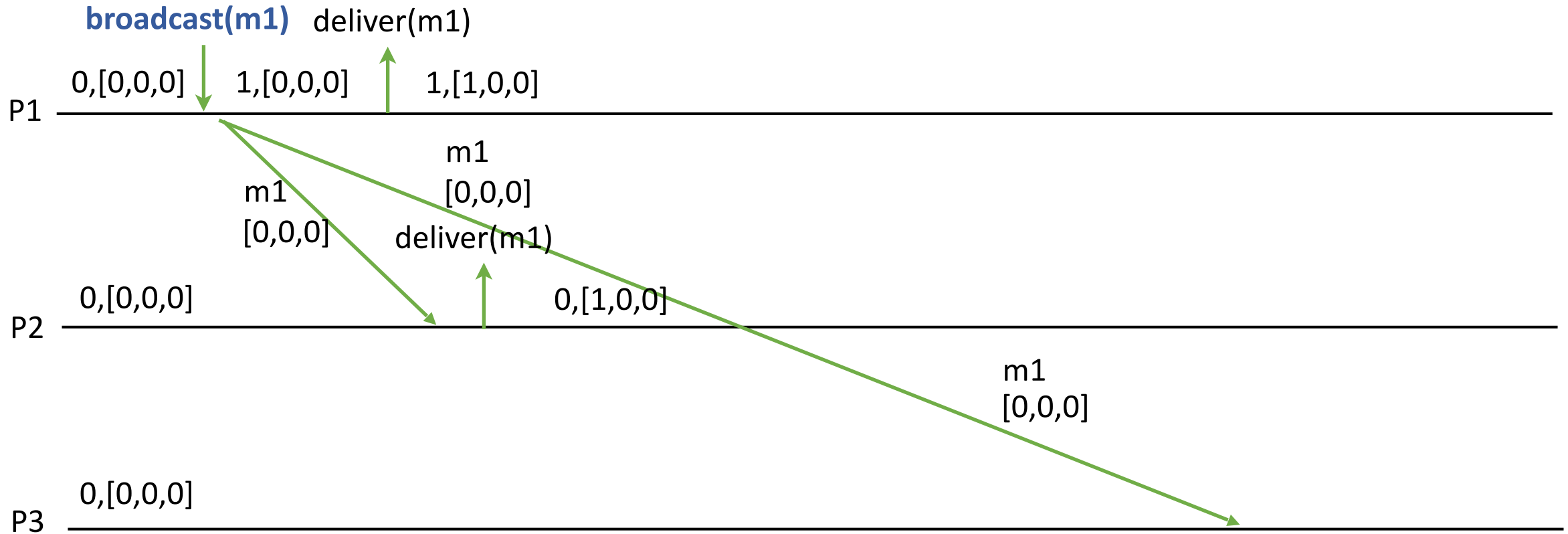
Example 1: FIFO Order



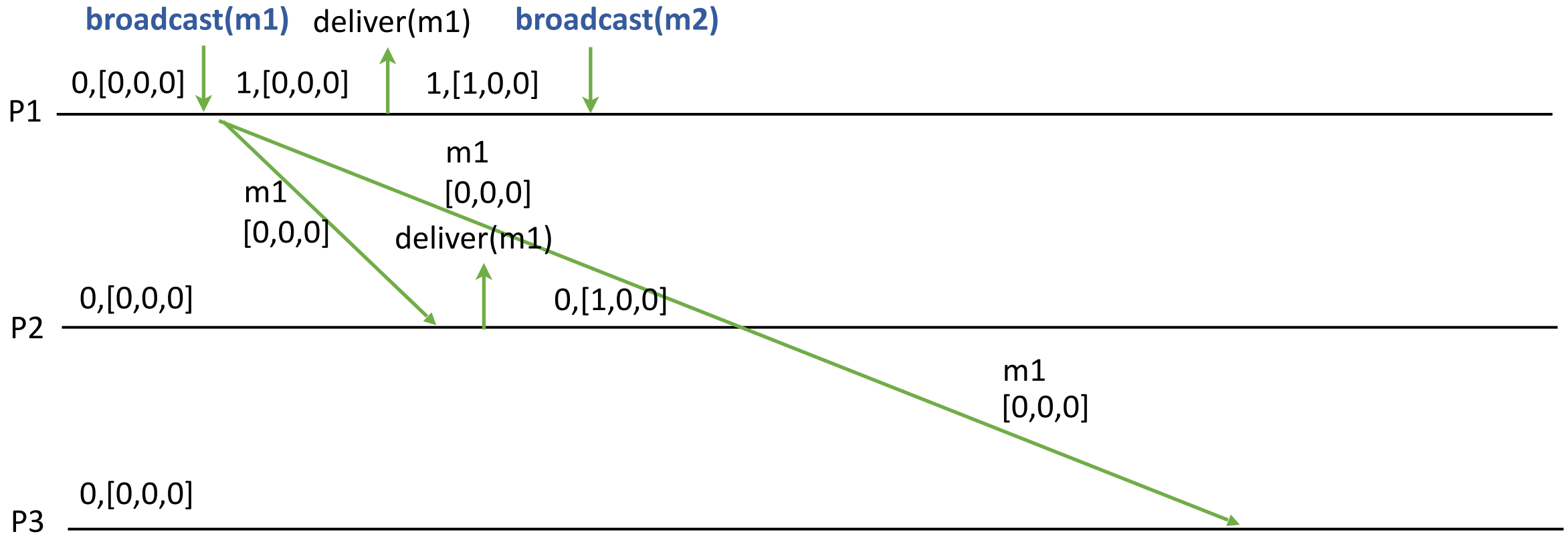
Example 1: FIFO Order



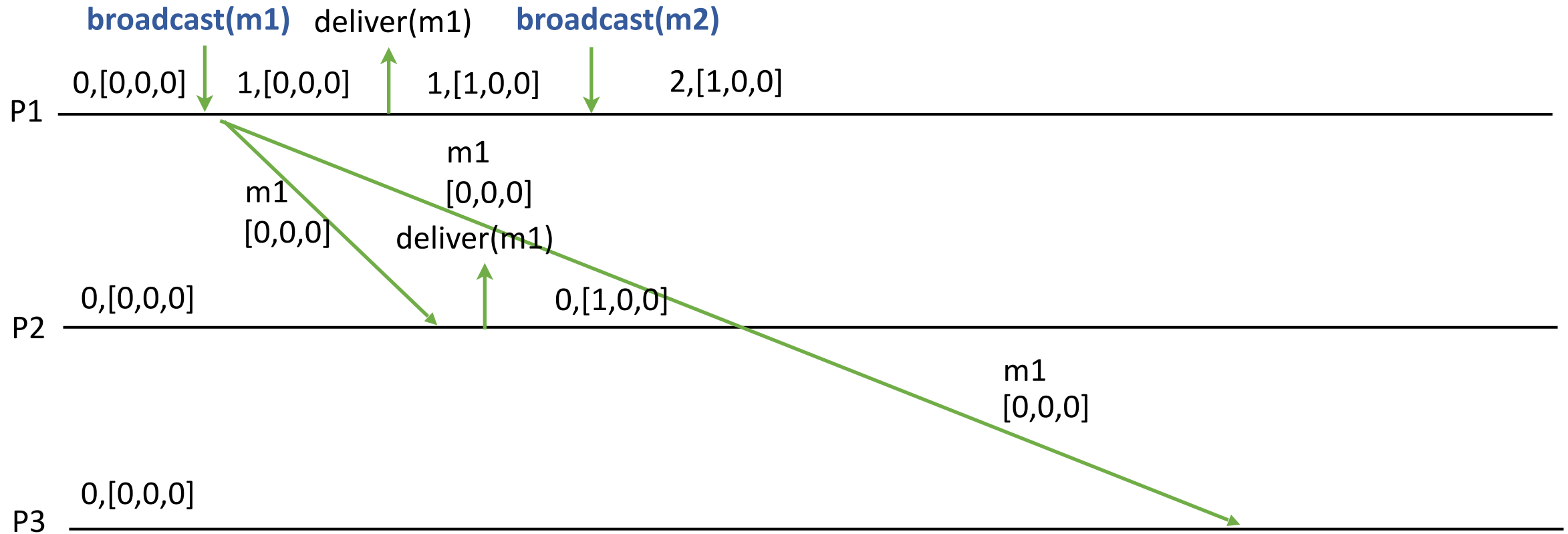
Example 1: FIFO Order



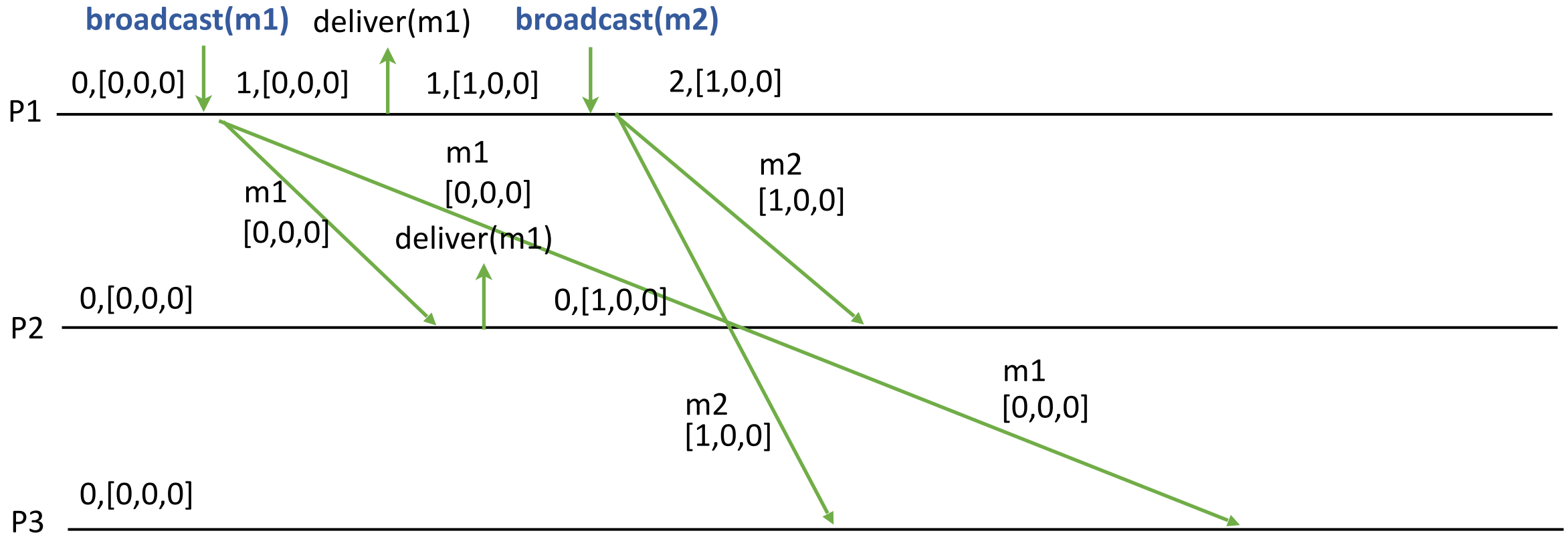
Example 1: FIFO Order



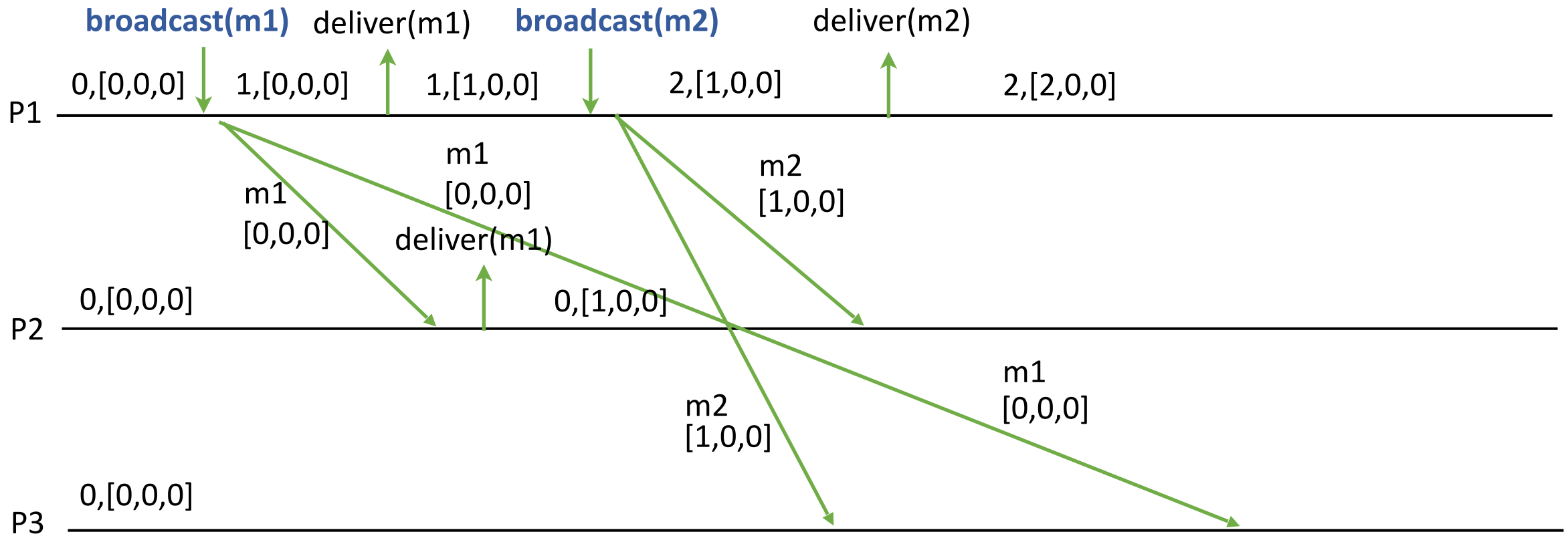
Example 1: FIFO Order



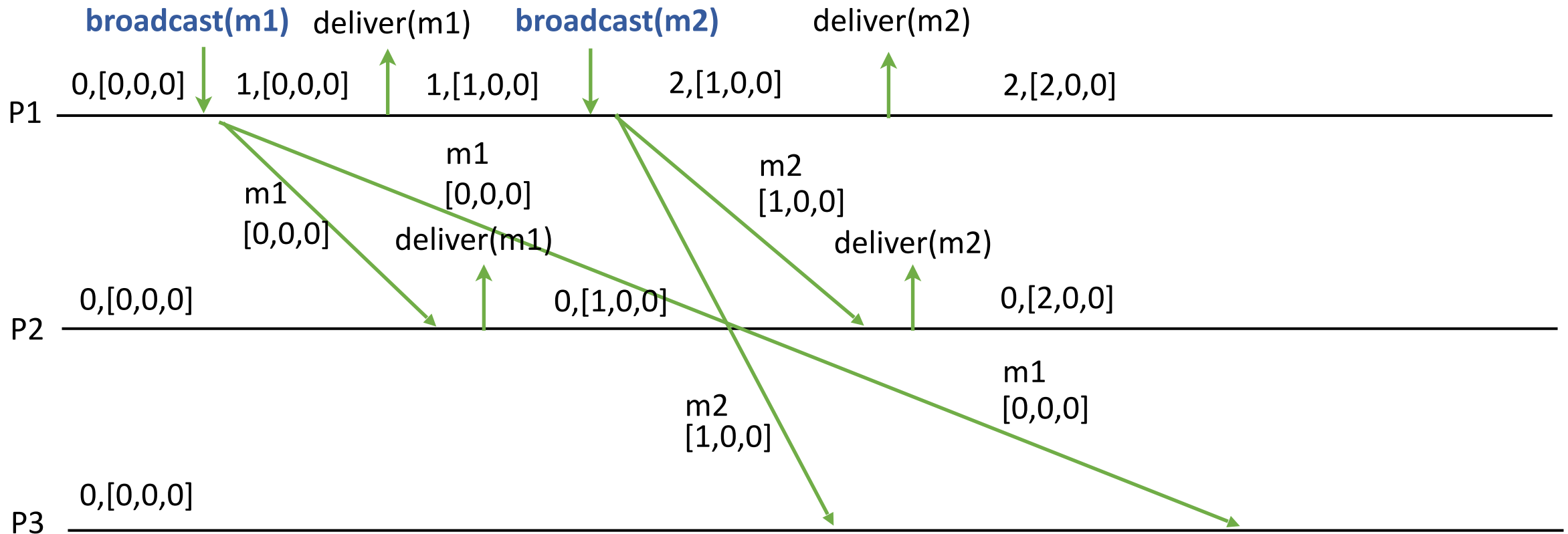
Example 1: FIFO Order



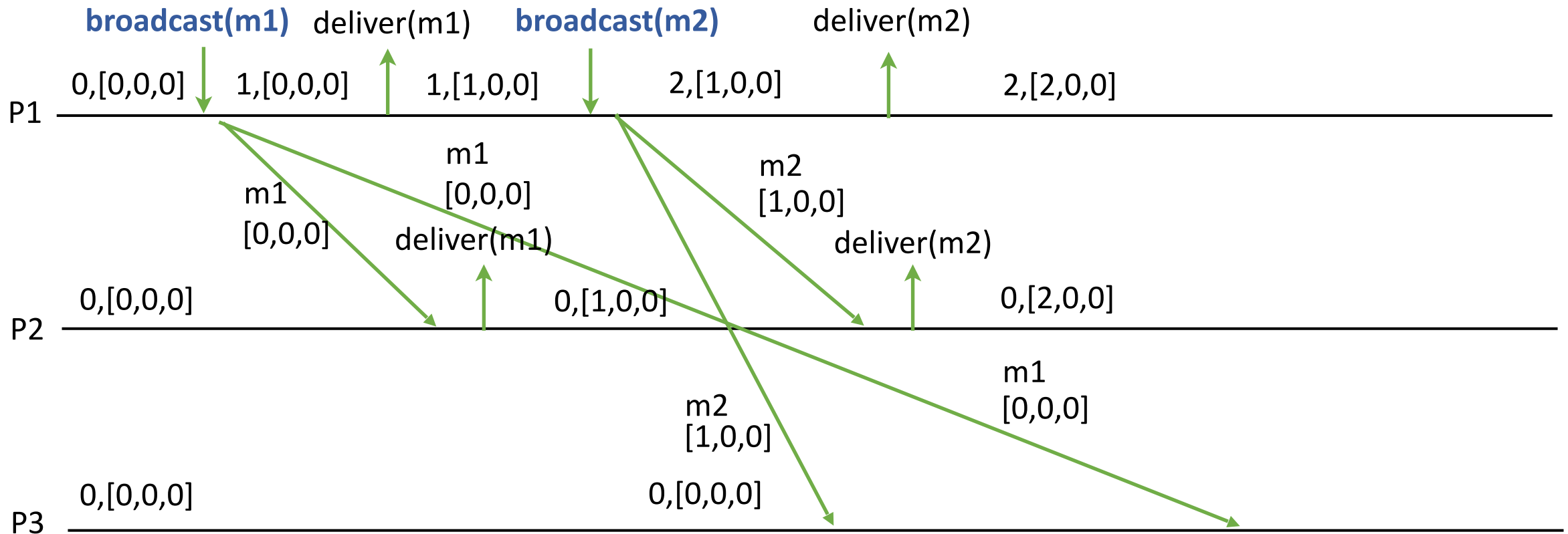
Example 1: FIFO Order



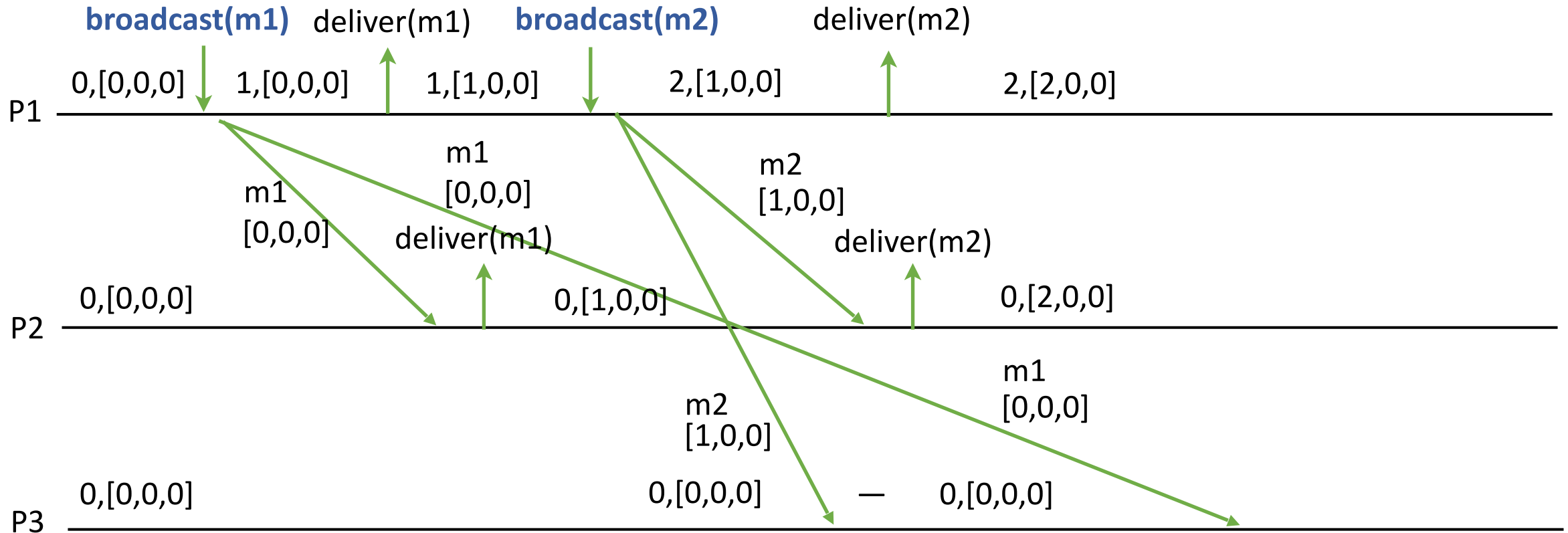
Example 1: FIFO Order



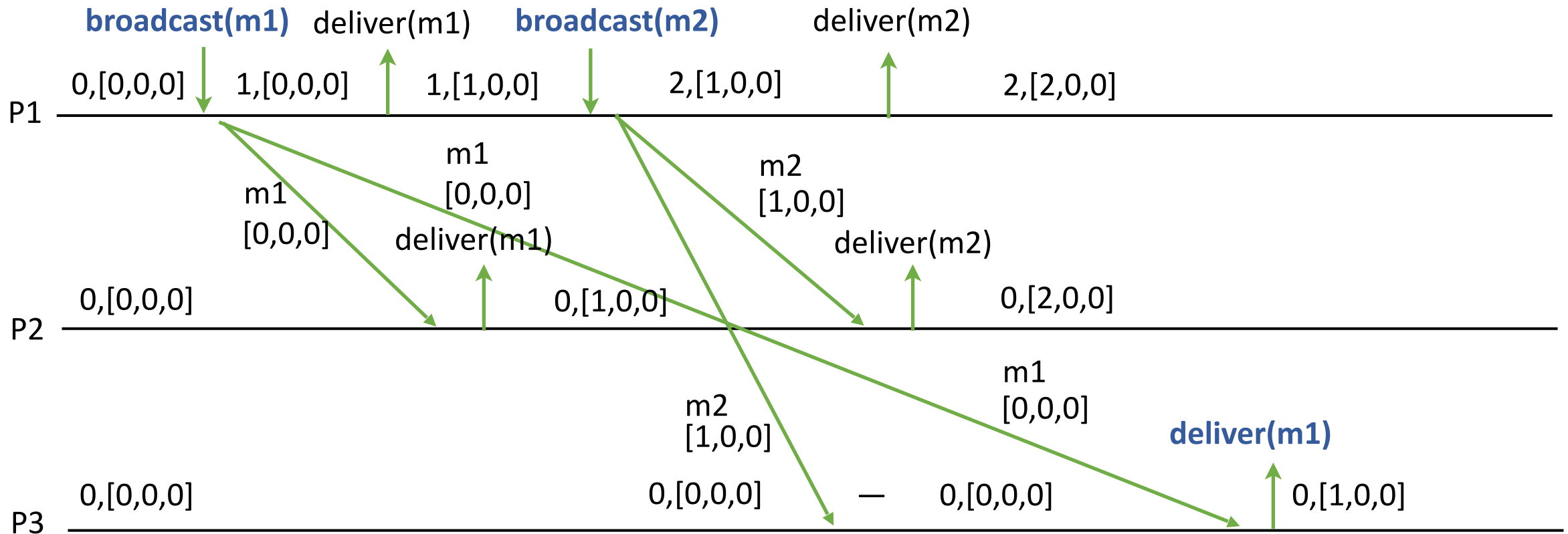
Example 1: FIFO Order



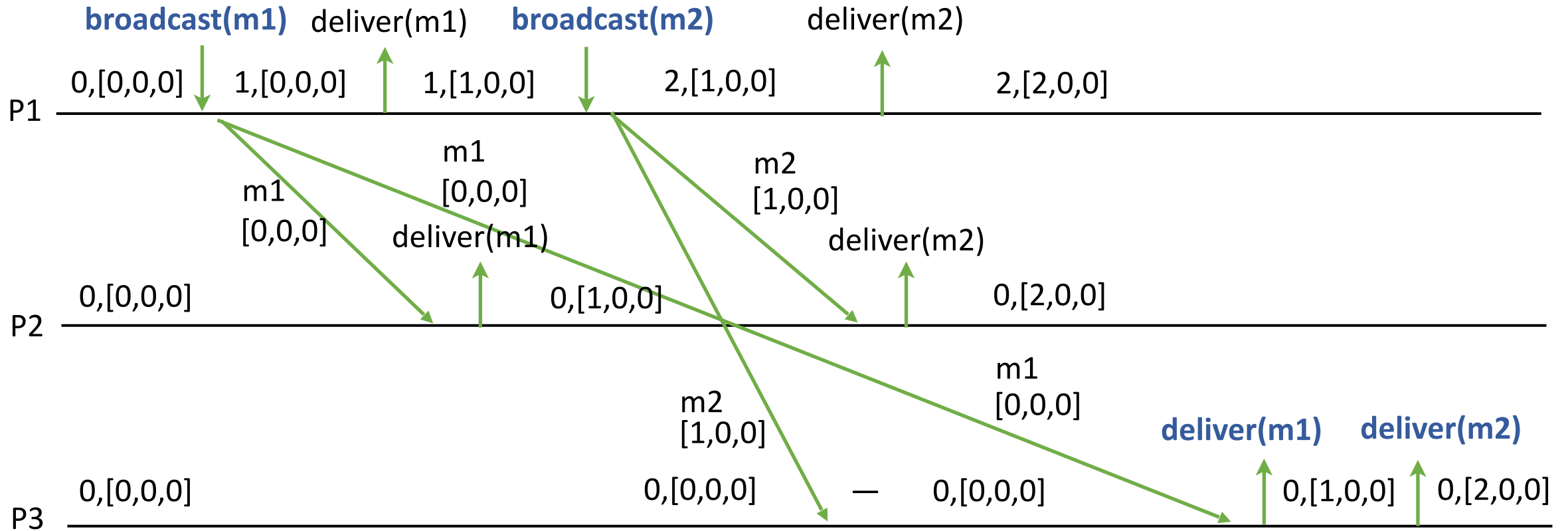
Example 1: FIFO Order



Example 1: FIFO Order



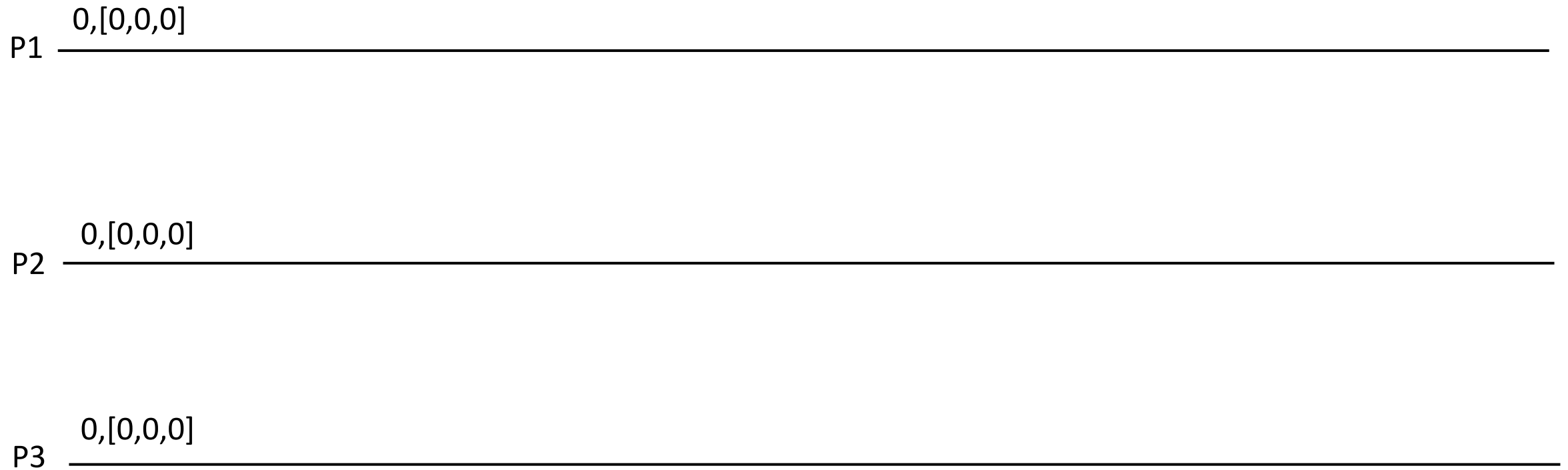
Example 1: FIFO Order



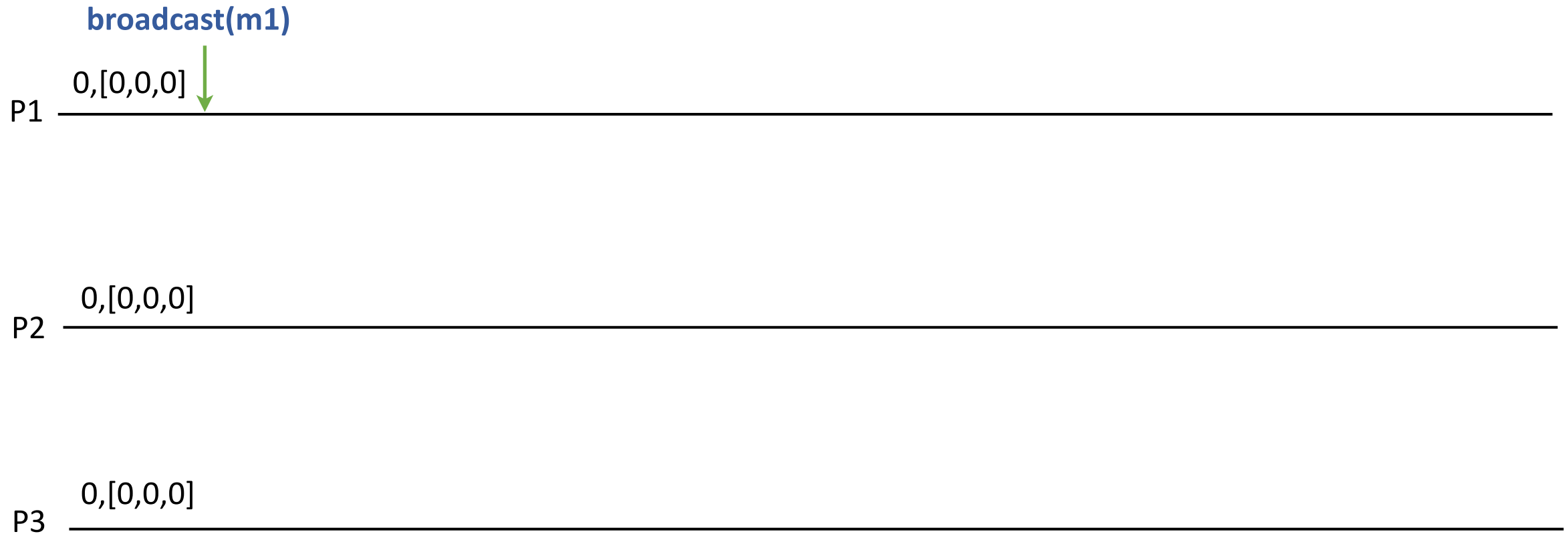
Example 2: FIFO Order



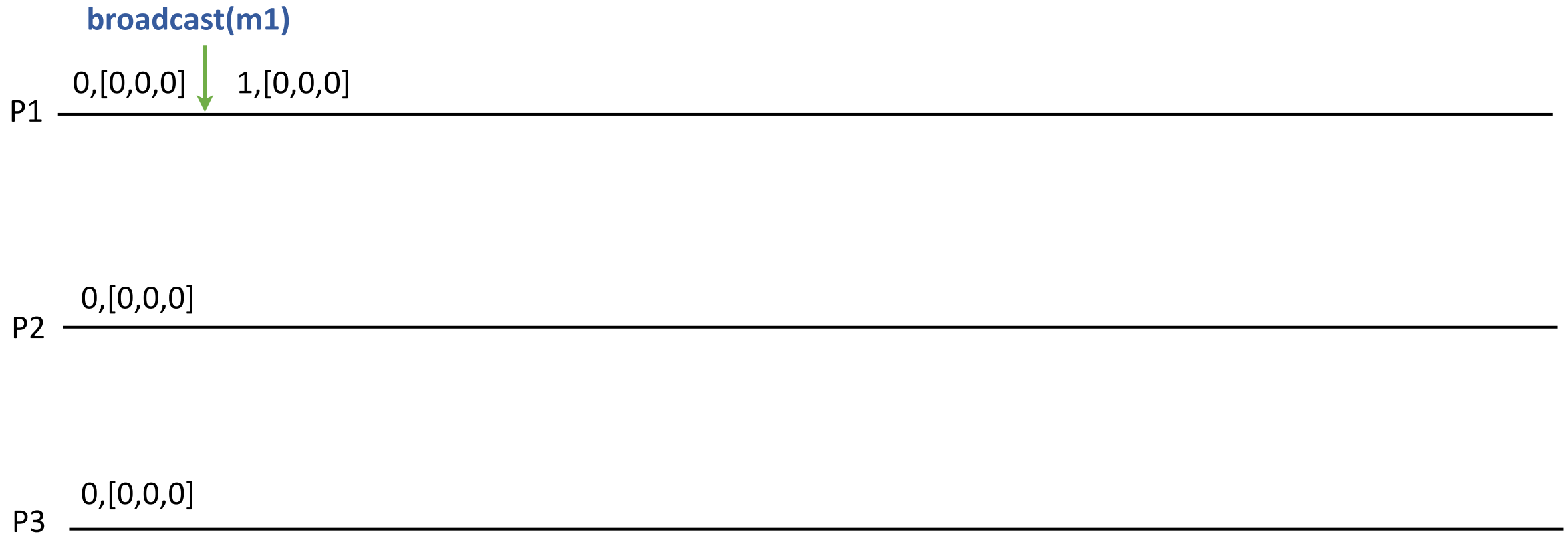
Example 2: FIFO Order



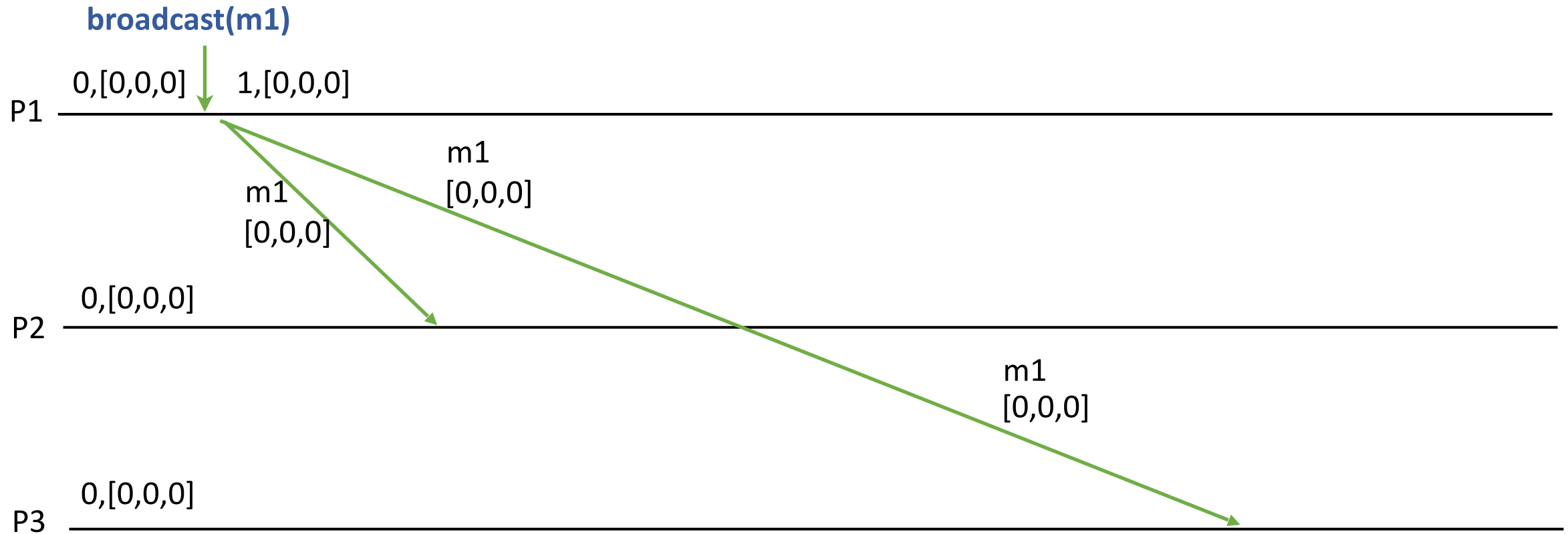
Example 2: FIFO Order



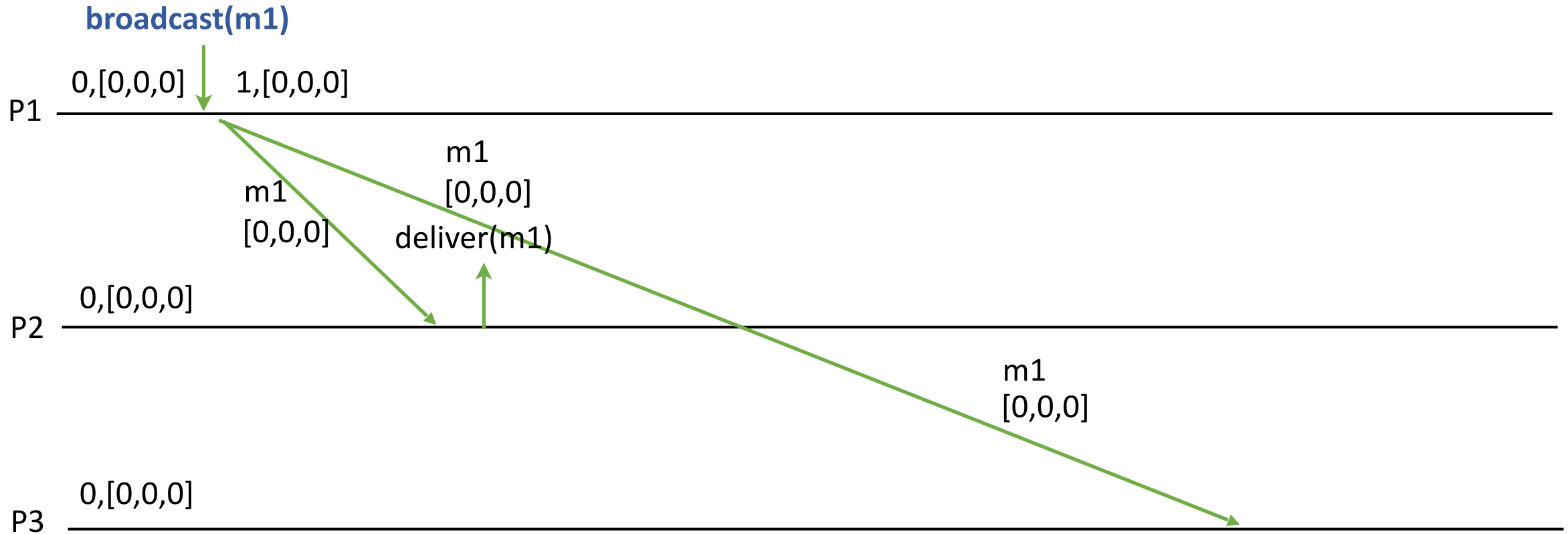
Example 2: FIFO Order



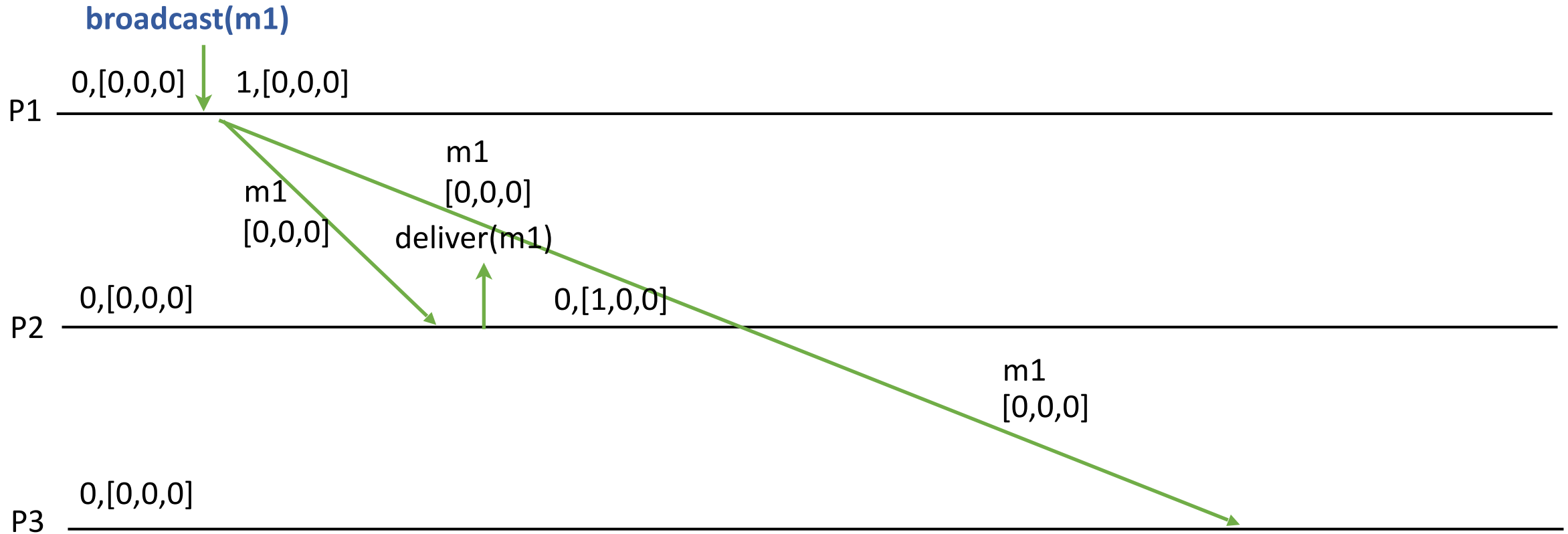
Example 2: FIFO Order



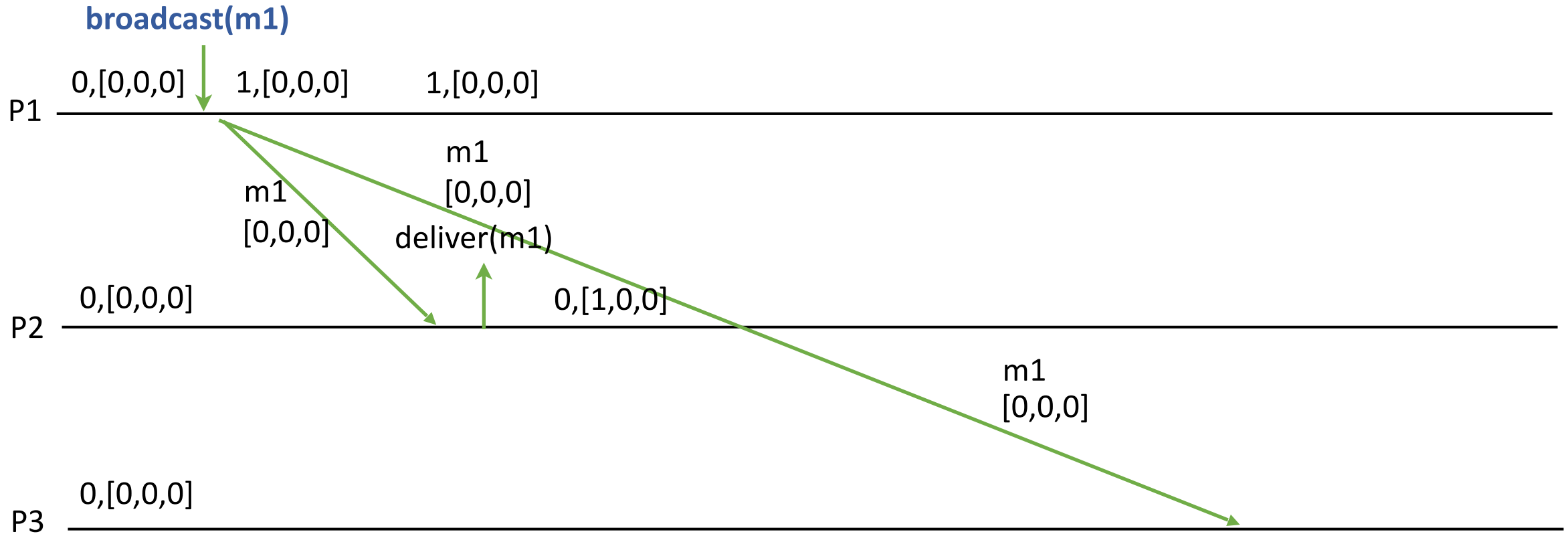
Example 2: FIFO Order



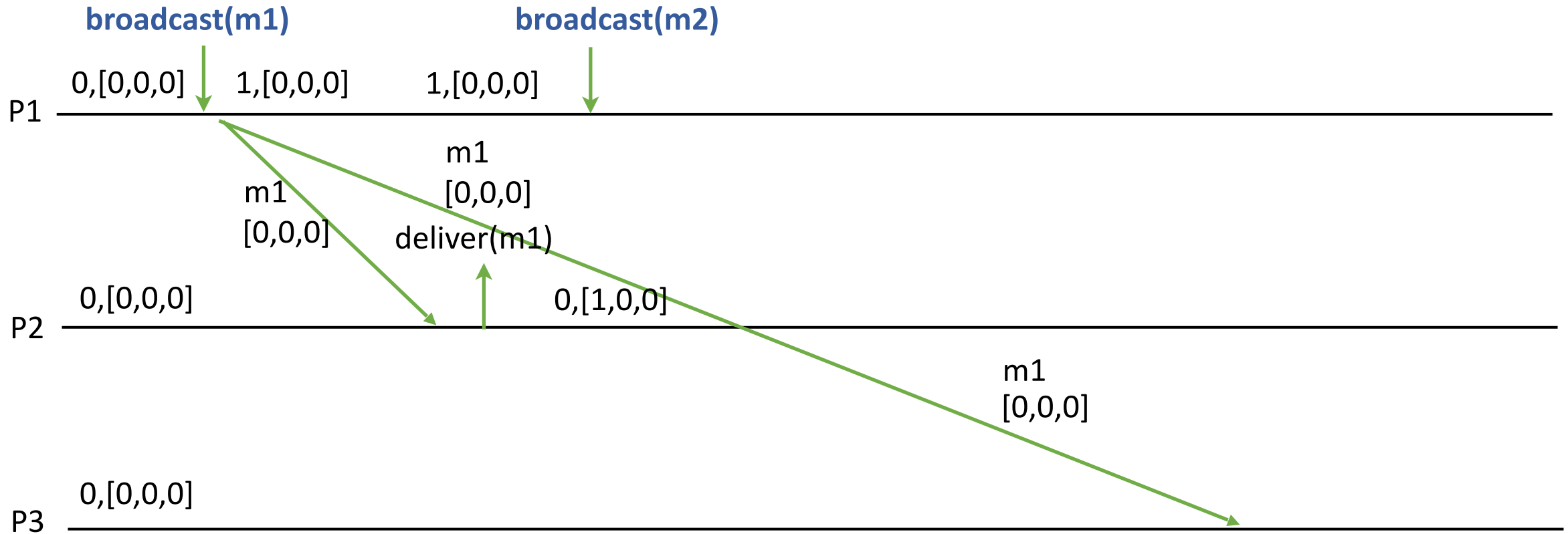
Example 2: FIFO Order



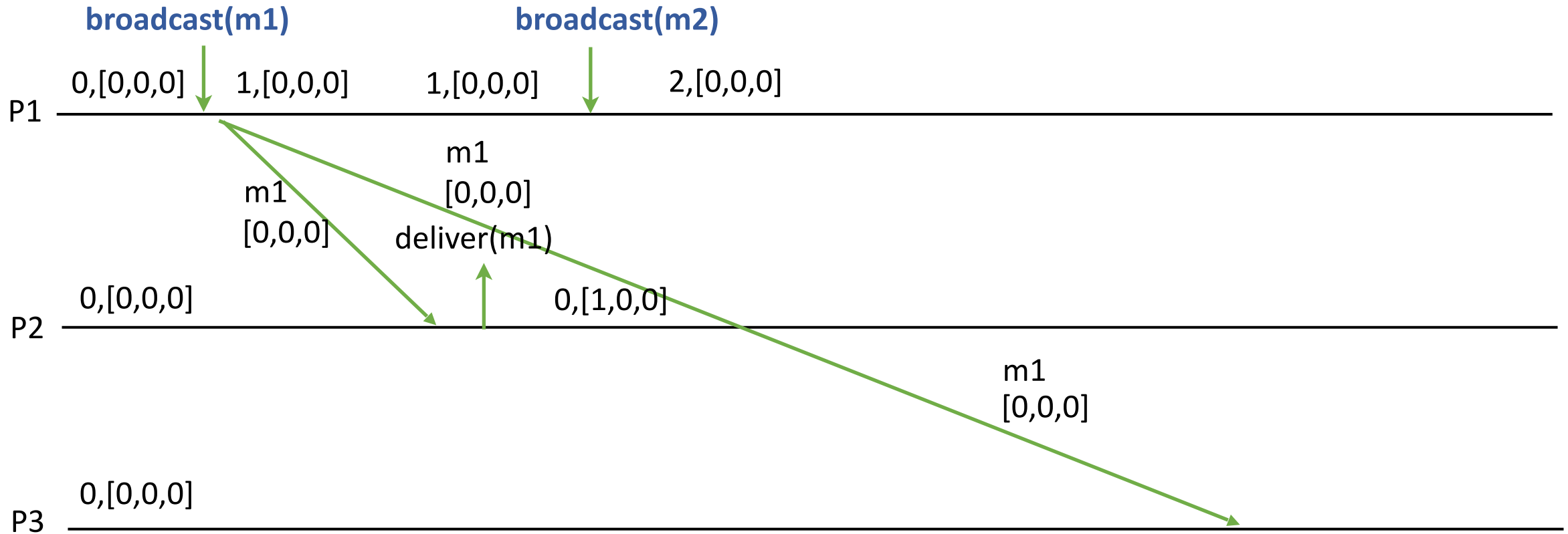
Example 2: FIFO Order



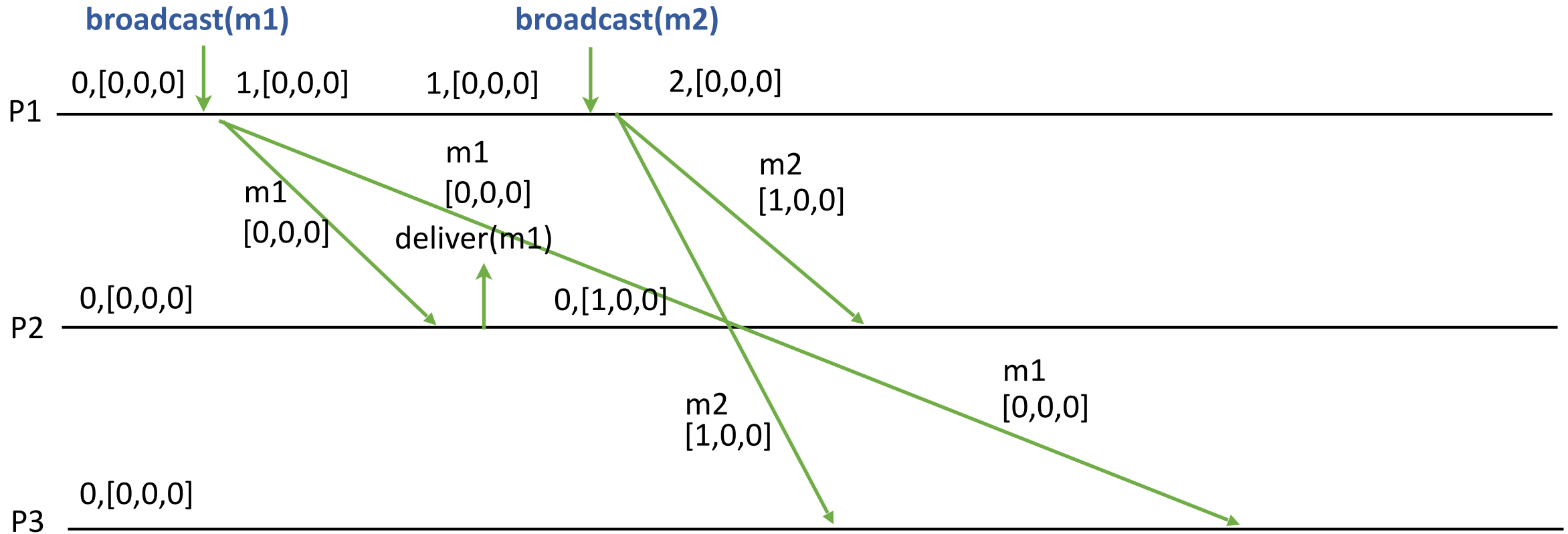
Example 2: FIFO Order



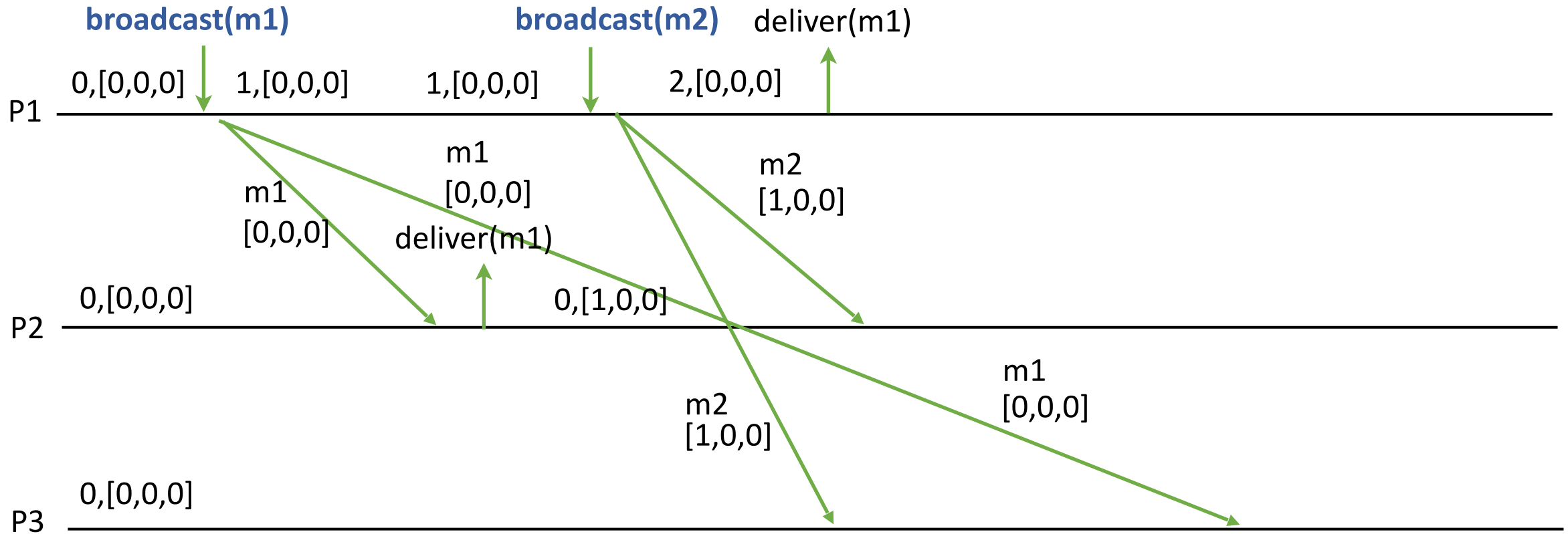
Example 2: FIFO Order



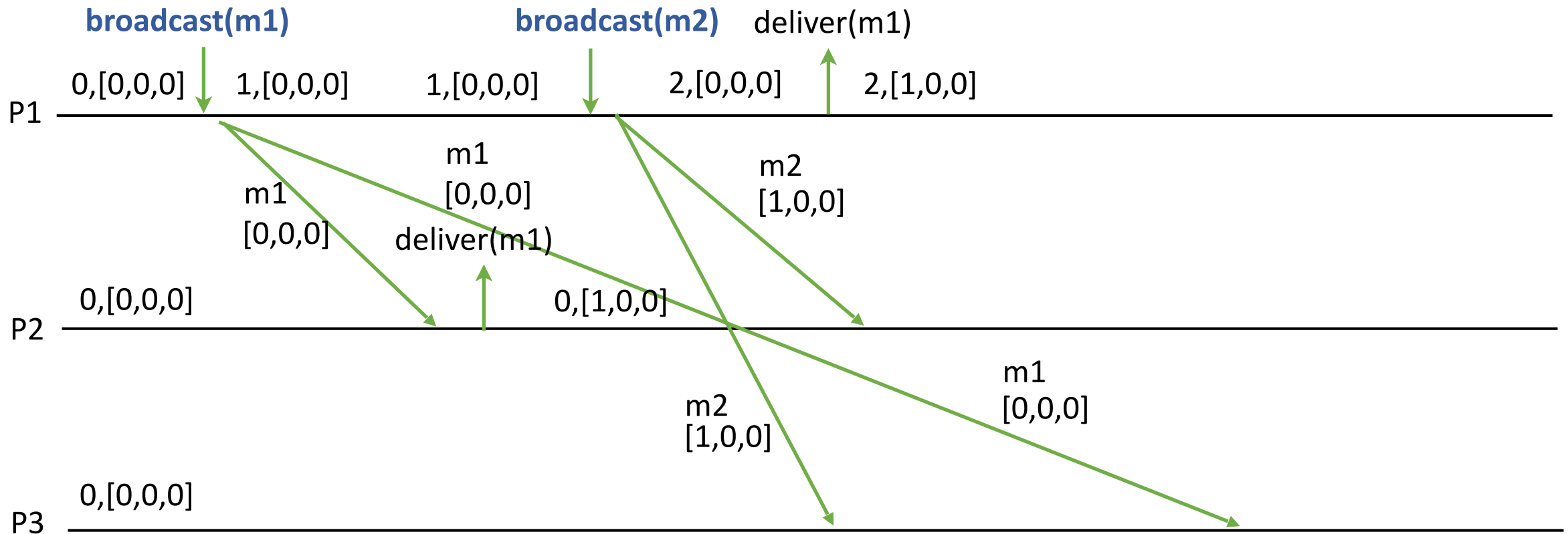
Example 2: FIFO Order



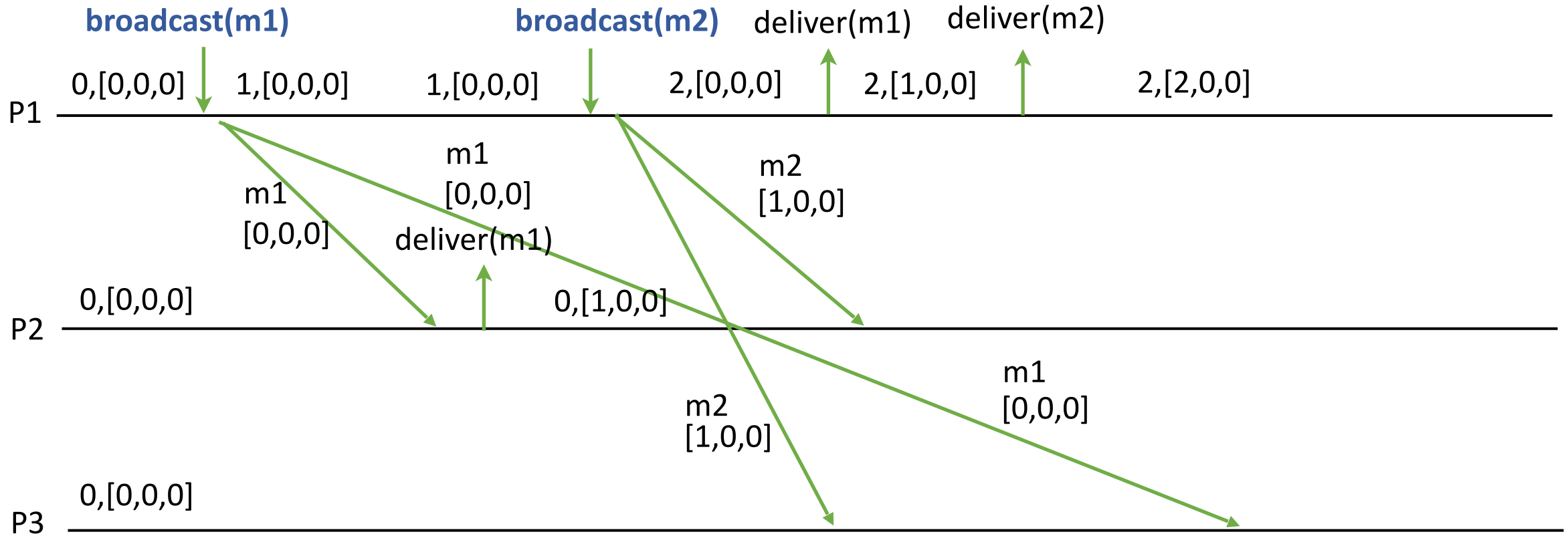
Example 2: FIFO Order



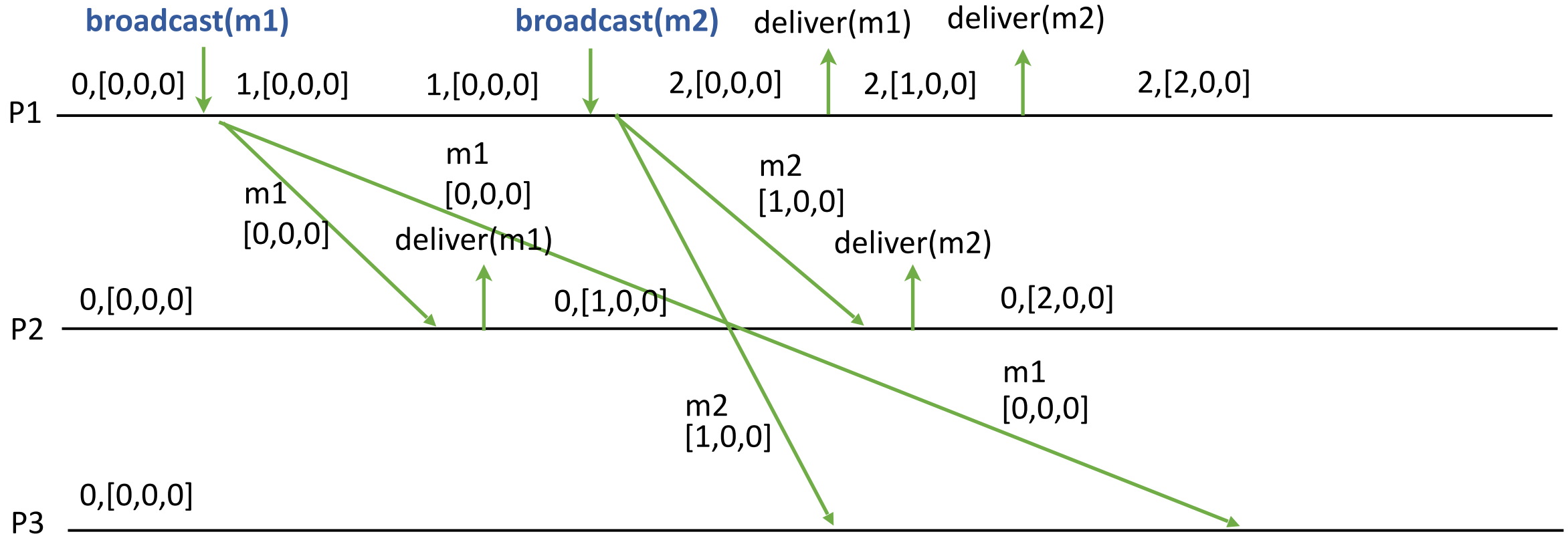
Example 2: FIFO Order



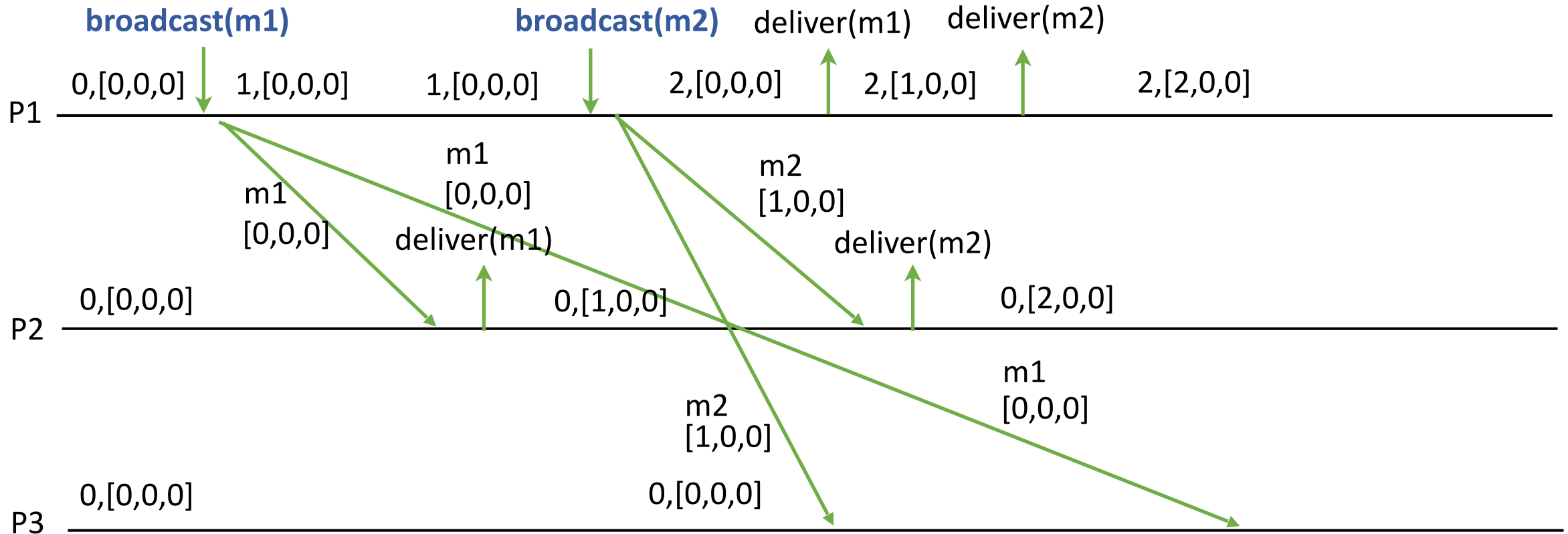
Example 2: FIFO Order



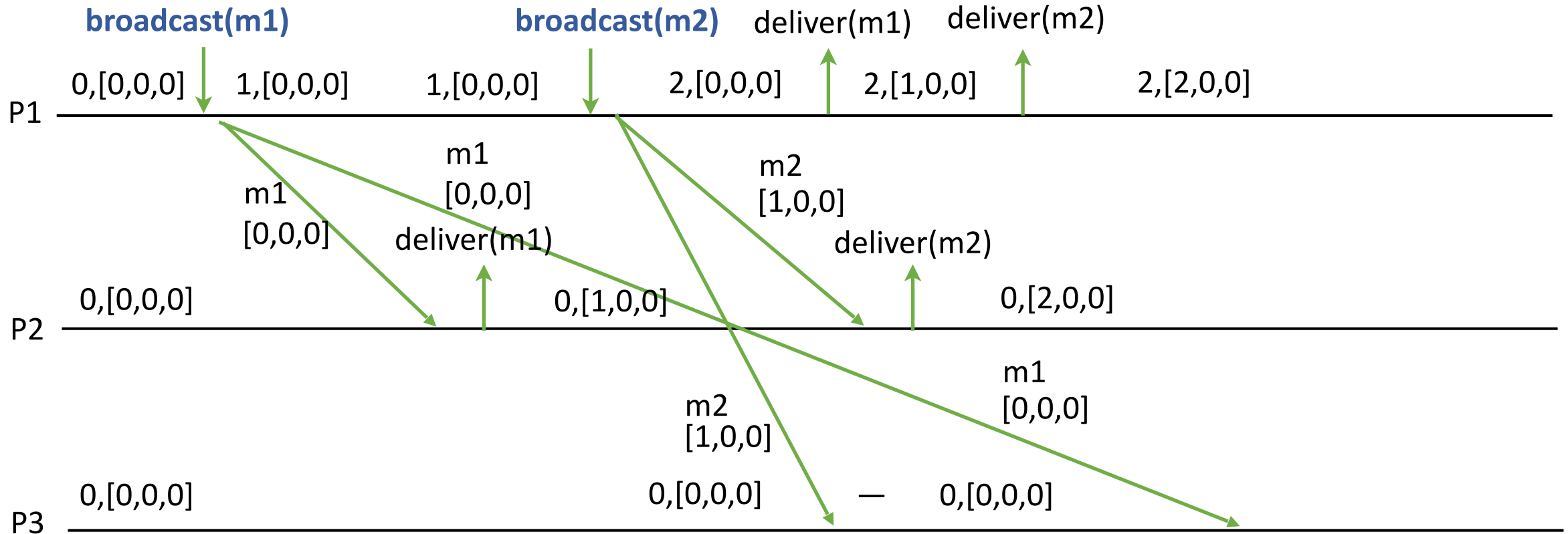
Example 2: FIFO Order



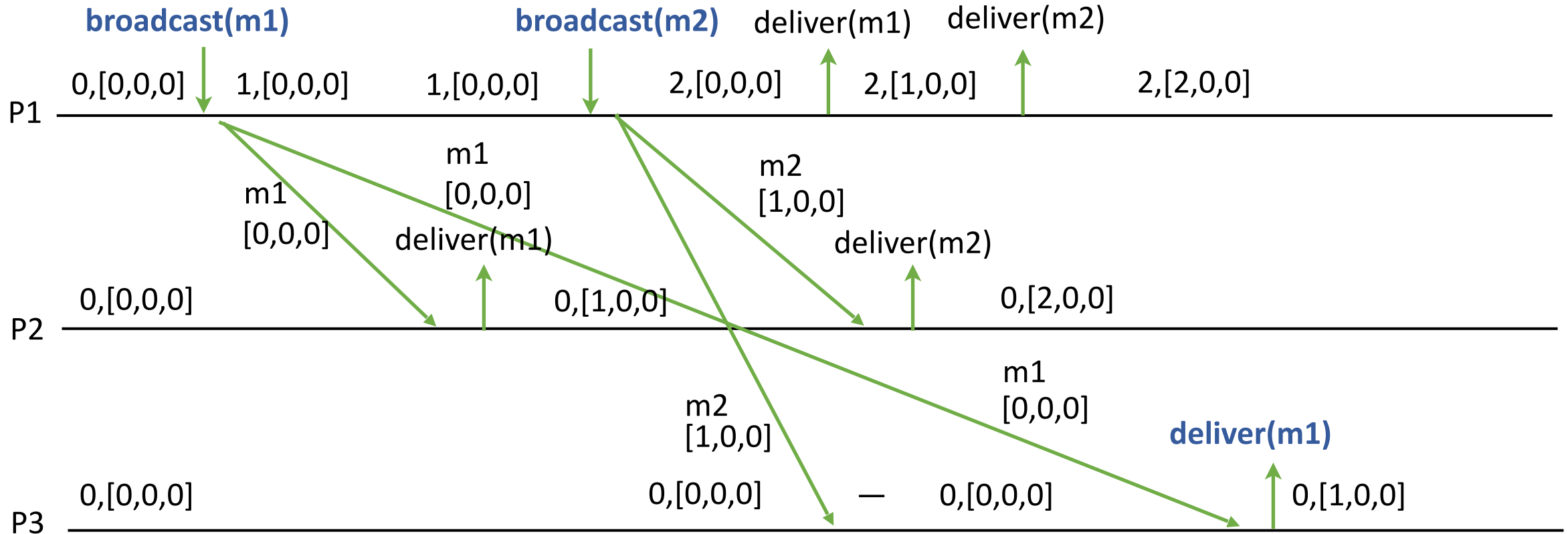
Example 2: FIFO Order



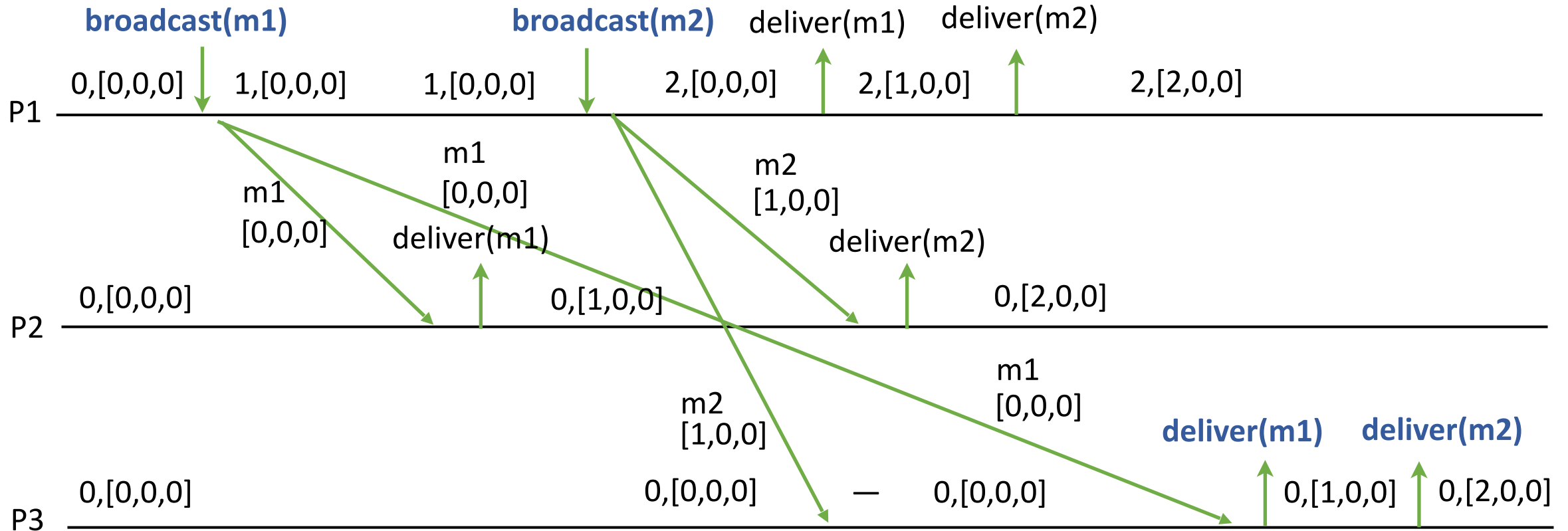
Example 2: FIFO Order



Example 2: FIFO Order



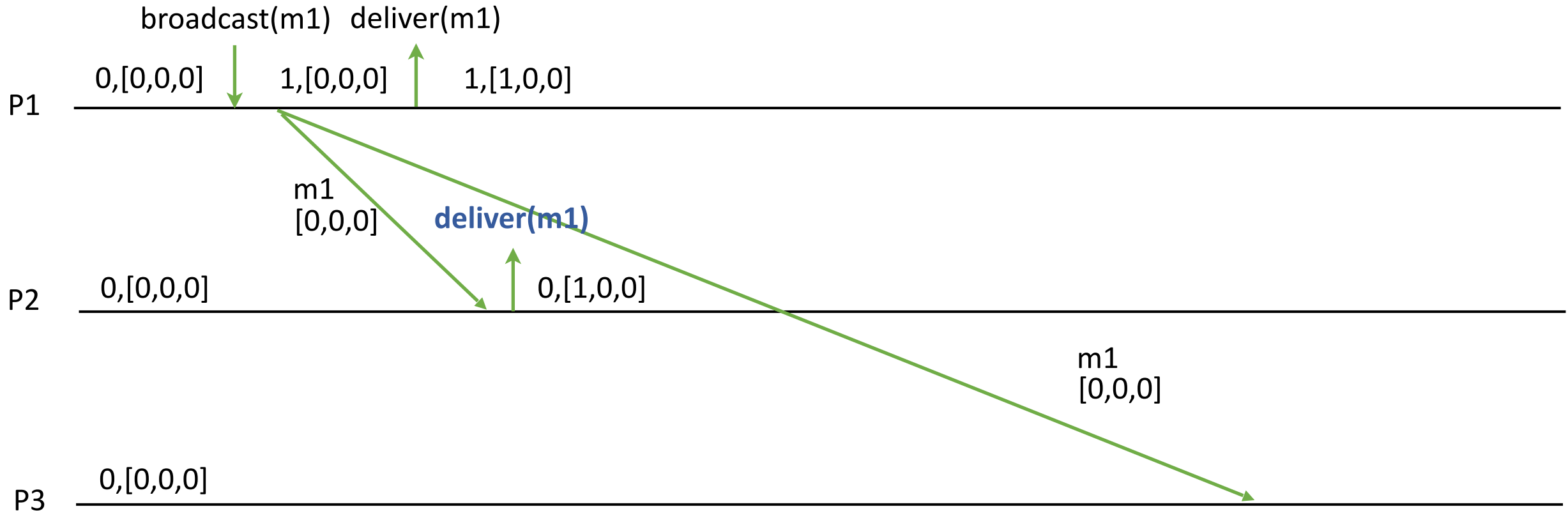
Example 2: FIFO Order



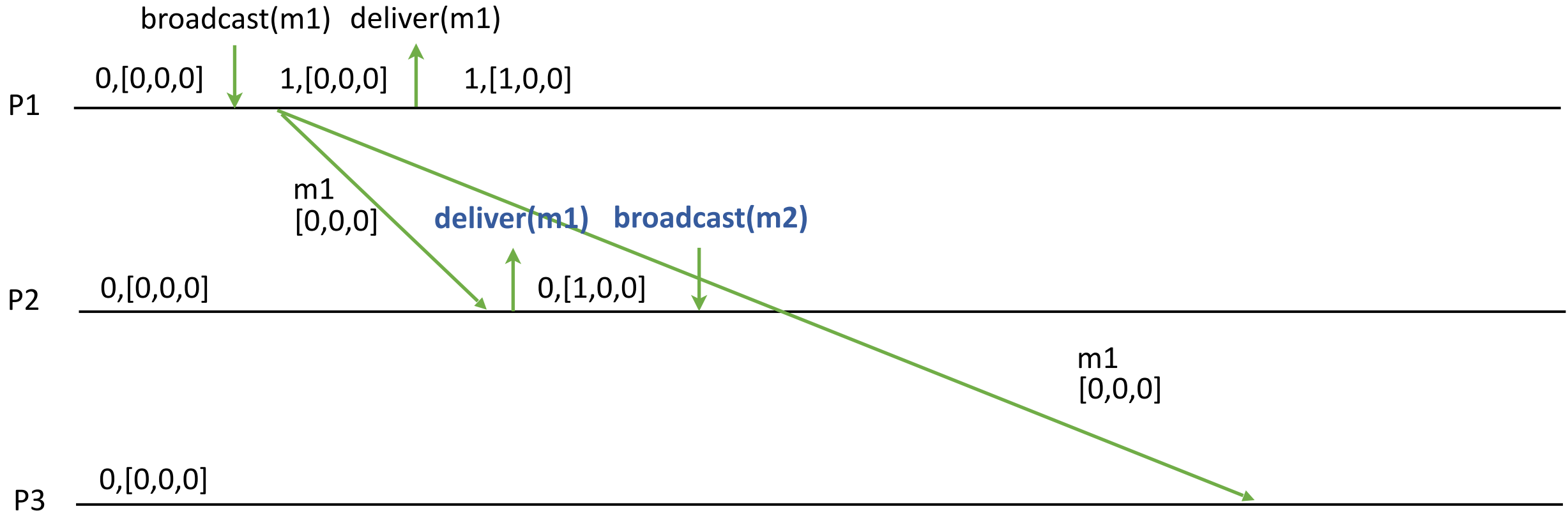
Example 3: InOut Order



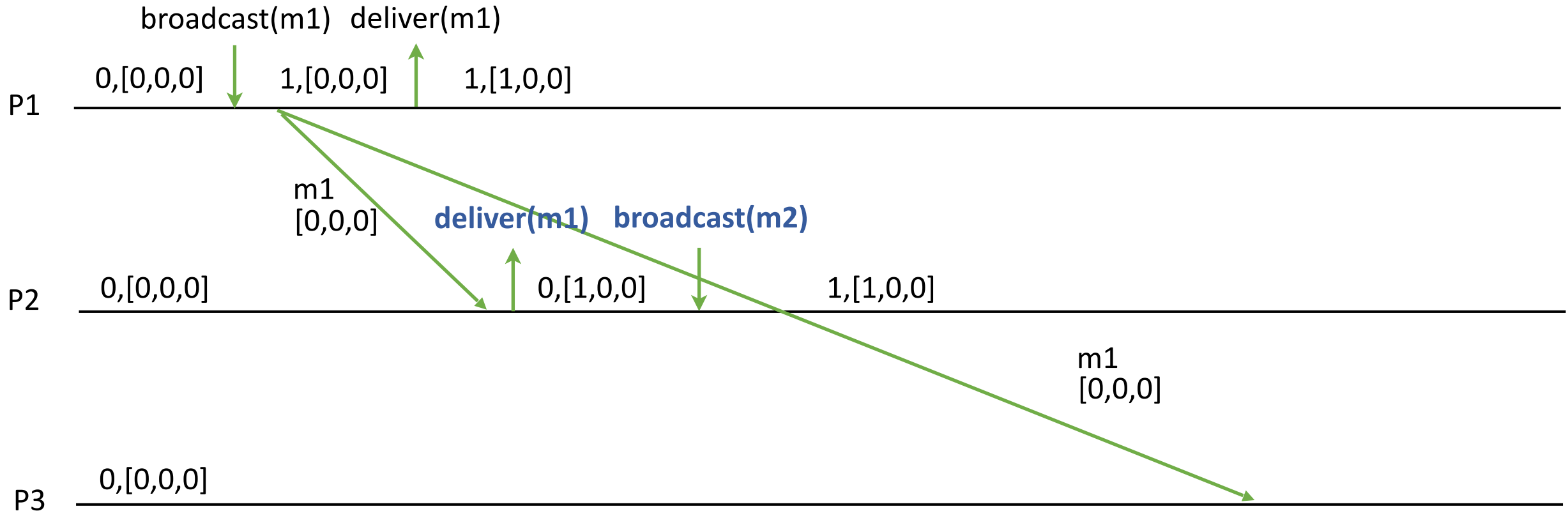
Example 3: InOut Order



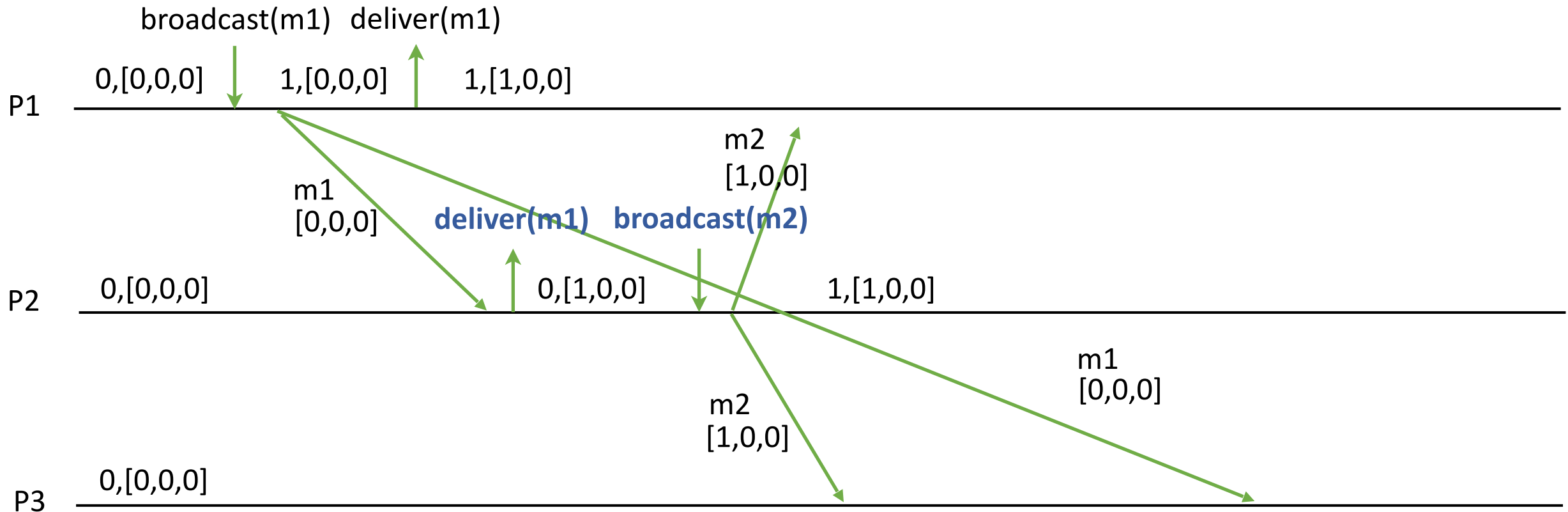
Example 3: InOut Order



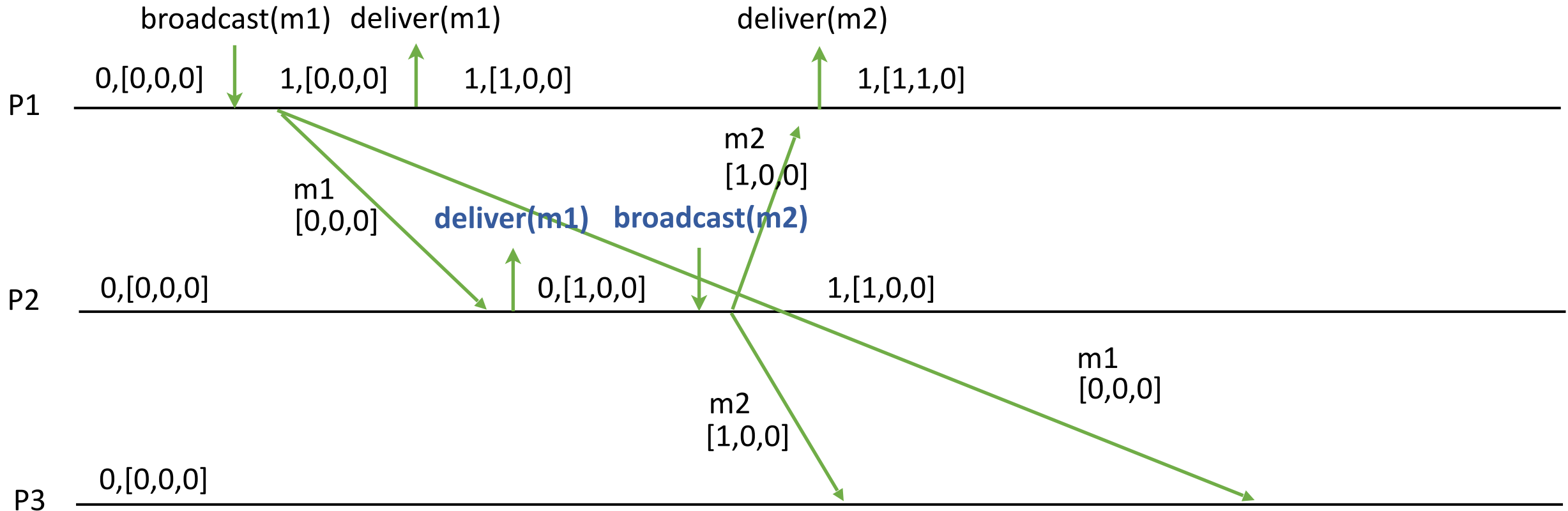
Example 3: InOut Order



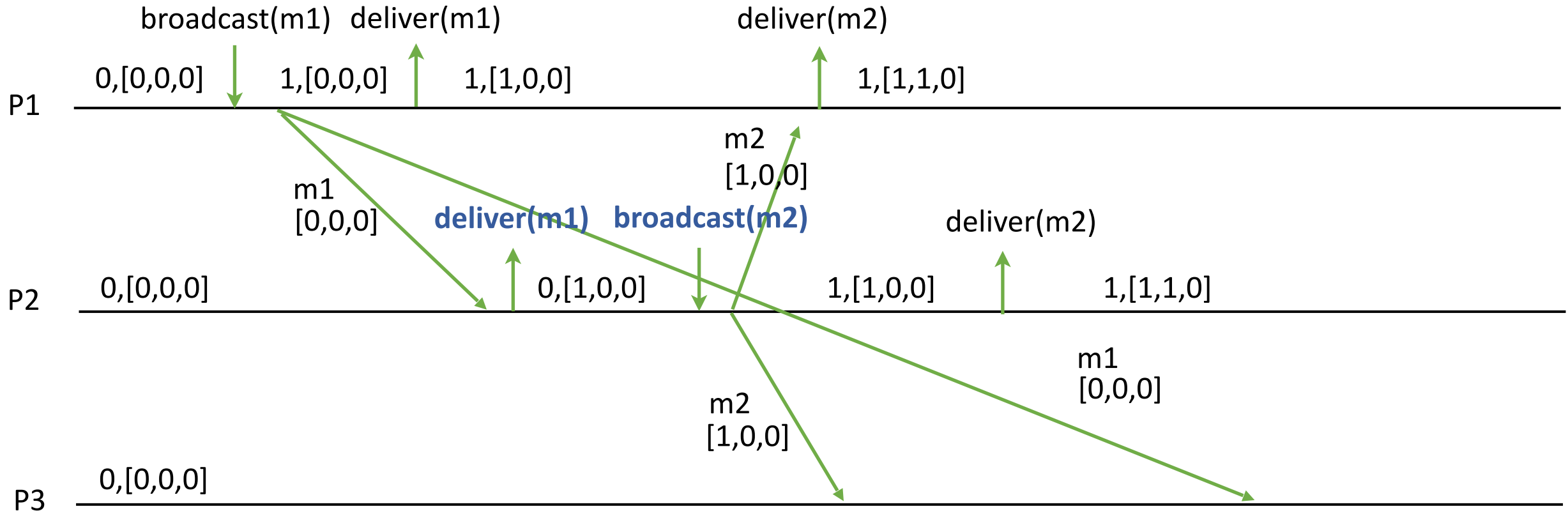
Example 3: InOut Order



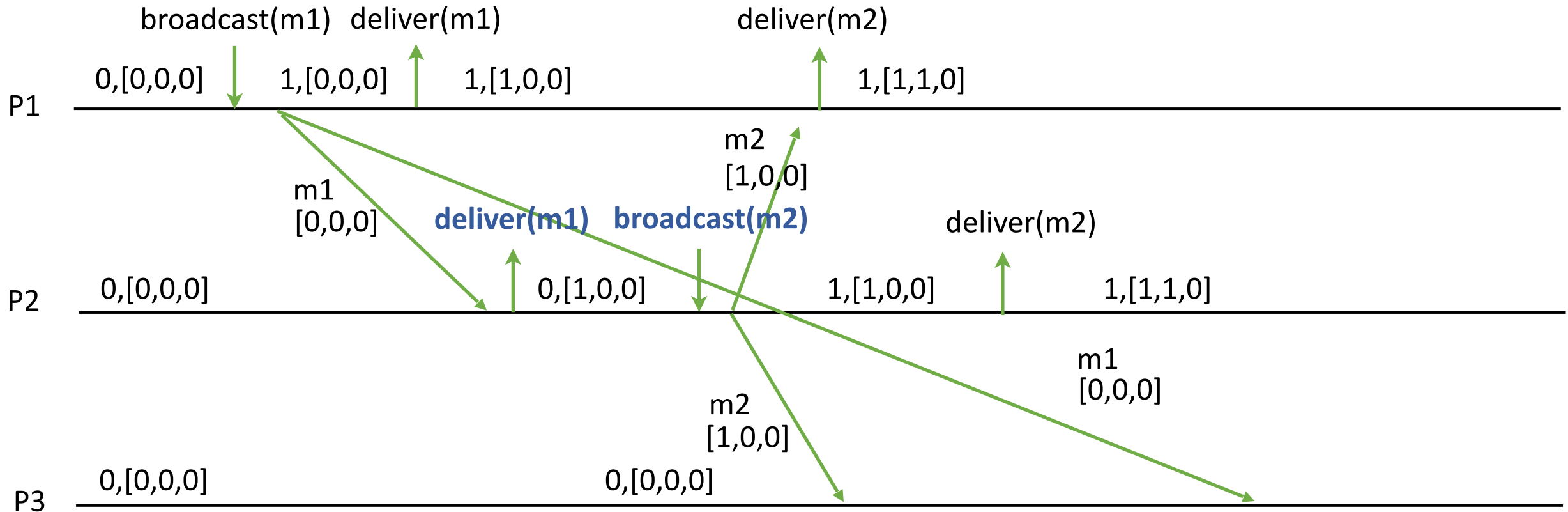
Example 3: InOut Order



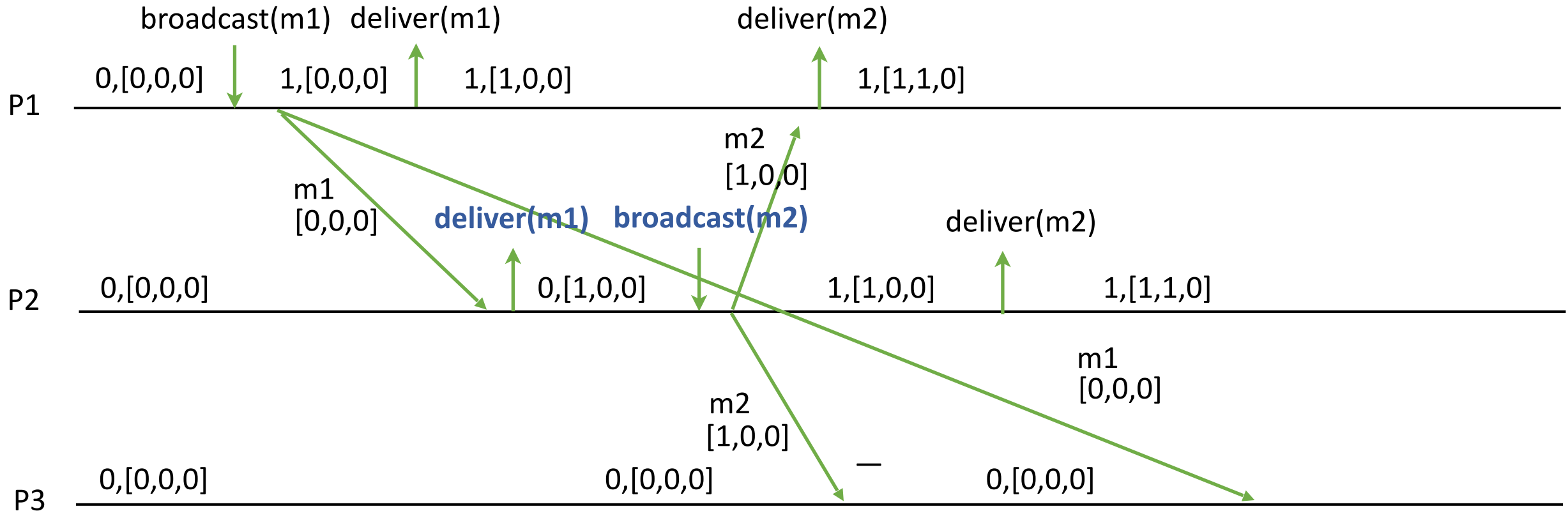
Example 3: InOut Order



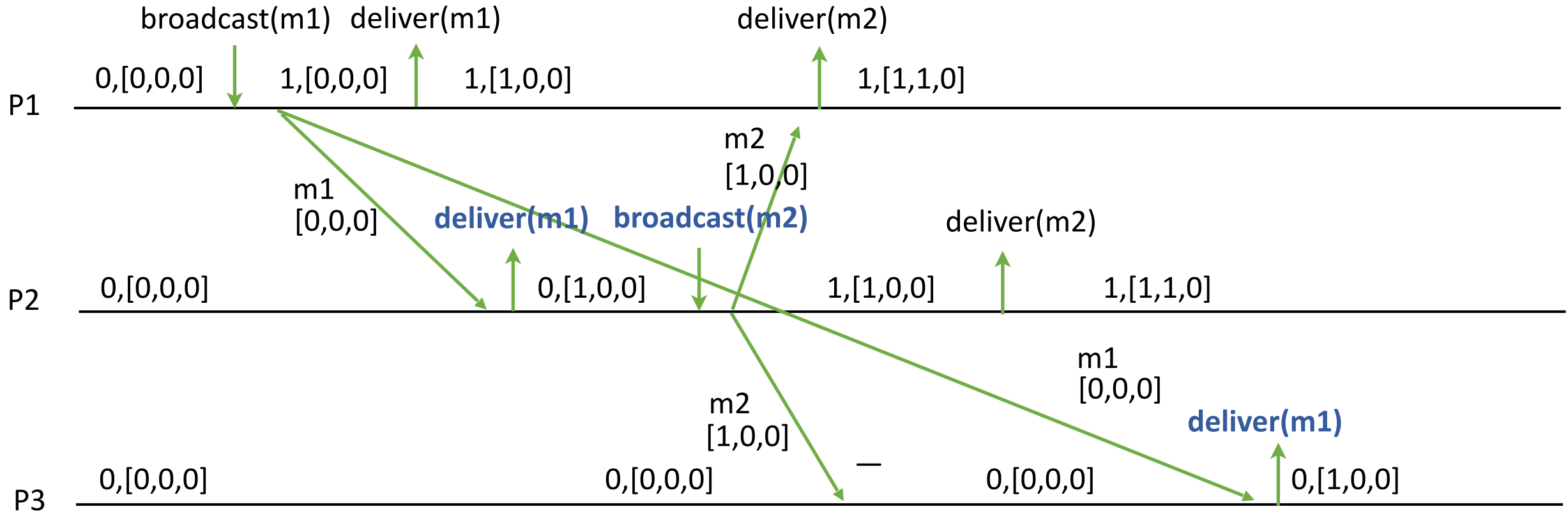
Example 3: InOut Order



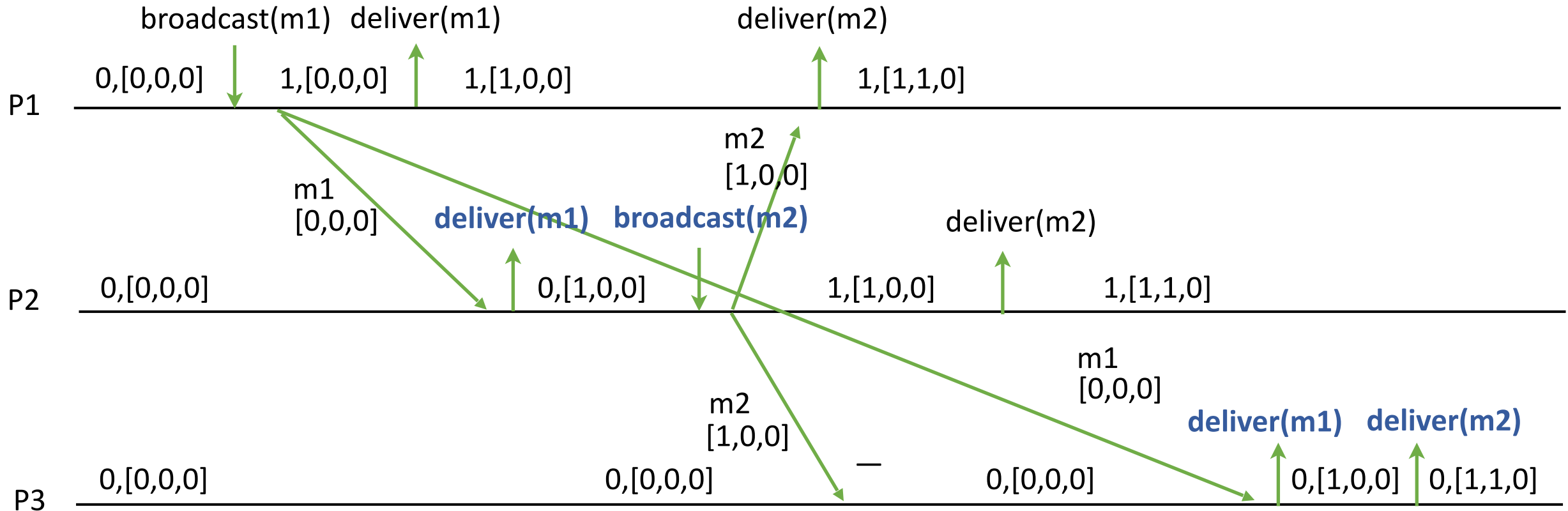
Example 3: InOut Order



Example 3: InOut Order



Example 3: InOut Order



Protocol 2

Implements: UniformCausalBroadcast (ucb).

Uses: UniformReliableBroadcast (urb).

upon event < Init > **do**

sq := 0

foreach pi in Π : VC[pi] := 0

Protocol 2

upon event < broadcast (m) > **do**

VC' = VC[self $\mapsto$ sq]

trigger < urb, broadcast ([VC', m]) >

sq = sq + 1

Protocol 2

upon event < broadcast (m) > **do**

$VC' = VC[self \mapsto sq]$

trigger < urb, broadcast ([VC', m]) >

$sq = sq + 1$

upon event < urb, deliver (pj, [VCm, m]) > **do**

wait until ($VC \geq VCM$)

trigger < deliver (pj, m) >

$VC[pj] := VC[pj] + 1$